

بازیابی اطلاعات

دانشگاه آزاد شیراز • دانشکده مهندسی کامپیوتر

ترم دوم • سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل پنجم: Index Compression

استاد درس

دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی)

امیرمحمد اسدجو

علی نیکوان

علی مجاهد

امیرحسین همتی

حمید نامجو

فهرست مطالب

5.1 Statistical properties of terms in information retrieval

5.1.1 Heaps' law: Estimating the number of terms

5.1.2 Zipf's law: Modeling the distribution of terms

5.2 Dictionary compression

5.2.1 Dictionary as a string

5.2.2 Blocked storage

5.3 Postings file compression

5.3.1 Variable byte codes

5.3.2 γ codes

5.4 Chapter 5 Summary

5.5 End of Chapter Exercises

فصل ۵: فشرده‌سازی ایندکس - هنر کوچک کردن داده‌ها بدون از دست دادن اطلاعات

هر روز میلیاردها جستجو در گوگل انجام می‌شود. برای پاسخ به این جستجوها در کسری از ثانیه، گوگل باید یک ایندکس عظیم از کل وب را مدیریت کند. این ایندکس آنقدر بزرگ است که اگر بخواهیم آن را بدون فشرده‌سازی ذخیره کنیم، به هزاران سرور نیاز داریم! اما گوگل چگونه این کار را انجام می‌دهد؟ پاسخ در فشرده‌سازی هوشمندانه ایندکس‌هاست.

فشرده‌سازی ایندکس یکی از مهم‌ترین تکنیک‌ها در سیستم‌های بازیابی اطلاعات است. در این فصل، با روش‌های فشرده‌سازی دیکشنری و ایندکس معکوس آشنا می‌شویم که برای سیستم‌های بازیابی اطلاعات کارآمد ضروری هستند.

💰 مزایای فشرده‌سازی: فراتر از صرفه‌جویی در فضا

اولین مزیت فشرده‌سازی که به ذهن می‌رسد، کاهش فضای ذخیره‌سازی است. با نسبت فشرده‌سازی 1:4، می‌توانیم هزینه ذخیره‌سازی ایندکس را 75% کاهش دهیم. اما دو مزیت مهم دیگر هم وجود دارد:

۱. **افزایش استفاده از حافظه کش:** تصور کنید در یک کتابخانه دیجیتال بزرگ، کاربران مدام به دنبال کلمه «کرونا» می‌گردند. اگر لیست ارسال این کلمه را در حافظه اصلی ذخیره کنیم، پاسخ به این جستجوها بسیار سریع خواهد بود. با فشرده‌سازی، می‌توانیم اطلاعات بیشتری را در حافظه اصلی نگه داریم و در نتیجه، پاسخ به جستجوها سریع‌تر انجام شود.

۲. **انتقال سریع‌تر داده از دیسک به حافظه:** الگوریتم‌های بازفشرده‌سازی مدرن آنقدر سریع هستند که زمان انتقال داده فشرده از دیسک و سپس بازفشرده‌سازی آن، معمولاً کمتر از زمان انتقال همان حجم داده به صورت فشرده‌نشده است. برای مثال، در یک سیستم جستجوی علمی مانند PubMed، بارگذاری لیست ارسال فشرده برای یک اصطلاح پزشکی رایج و سپس بازفشرده‌سازی آن، سریع‌تر از بارگذاری لیست ارسال فشرده‌نشده است.

📊 ویژگی‌های آماری واژه‌ها: رازهای آمار در خدمت فشرده‌سازی

برای فهم بهتر فشرده‌سازی، ابتدا باید با ویژگی‌های آماری واژه‌ها آشنا شویم. در مجموعه‌های متنی بزرگ، توزیع واژه‌ها از الگوهای خاصی پیروی می‌کند که می‌توانیم از آن‌ها برای فشرده‌سازی بهتر استفاده کنیم.

جدول زیر تأثیر پیش‌پردازش بر تعداد واژه‌ها و لیست‌های ارسال را در مجموعه Reuters-RCV1 نشان می‌دهد:

- حذف اعداد: کاهش ۲% در واژگان و ۸% در لیست‌های ارسال
- تبدیل حروف به حالت یکسان (case folding): کاهش ۱۷% در واژگان
- حذف ۳۰ واژه رایج: کاهش ۱۴% در لیست‌های ارسال
- حذف ۱۵۰ واژه رایج: کاهش ۳۰% در لیست‌های ارسال
- ریشه‌یابی (stemming): کاهش ۱۷% دیگر در واژگان

نکته جالب این است که حذف واژه‌های رایج باعث کاهش چشمگیر در تعداد postings می‌شود، اما تأثیر زیادی بر اندازه ایندکس فشرده ندارد، چون لیست‌های ارسال این واژه‌ها پس از فشرده‌سازی بسیار کوچک می‌شوند.

🔄 فشرده‌سازی بدون اتلاف در مقابل فشرده‌سازی با اتلاف

در این فصل، ما بر روی فشرده‌سازی بدون اتلاف (lossless compression) تمرکز می‌کنیم، جایی که تمام اطلاعات حفظ می‌شود. اما فشرده‌سازی با اتلاف (lossy compression) نیز کاربردهای مهمی دارد:

- **فشرده‌سازی بدون اتلاف:** تمام اطلاعات حفظ می‌شود. تکنیک‌هایی که در این فصل بررسی می‌شوند از این نوع هستند.

- **فشرده‌سازی با اتلاف:** بخشی از اطلاعات حذف می‌شود. تبدیل حروف به حالت یکسان، ریشه‌یابی، و حذف واژه‌های توقف نمونه‌هایی از این نوع هستند. همچنین مدل فضای برداری (فصل ۶) و تکنیک‌های کاهش ابعاد مانند نمایه‌سازی معنایی پنهان (فصل ۱۸) نیز نمونه‌هایی از فشرده‌سازی با اتلاف هستند.

در جستجوی وب، چون کاربران معمولاً فقط نتایج اول را می‌بینند، می‌توانیم از فشرده‌سازی با اتلاف استفاده کنیم بدون اینکه تأثیر منفی بر تجربه کاربر داشته باشیم.

📊 قانون هپس: چرا واژگان مجموعه‌های متنی آنقدر بزرگ هستند؟

شاید فکر کنید زبان‌ها واژگان محدودی دارند. برای مثال، دومین نسخه دیکشنری آکسفورد بیش از ۶۰۰۰۰۰ کلمه تعریف می‌کند. اما واژگان اکثر مجموعه‌های بزرگ بسیار بزرگ‌تر از دیکشنری آکسفورد است. چرا؟

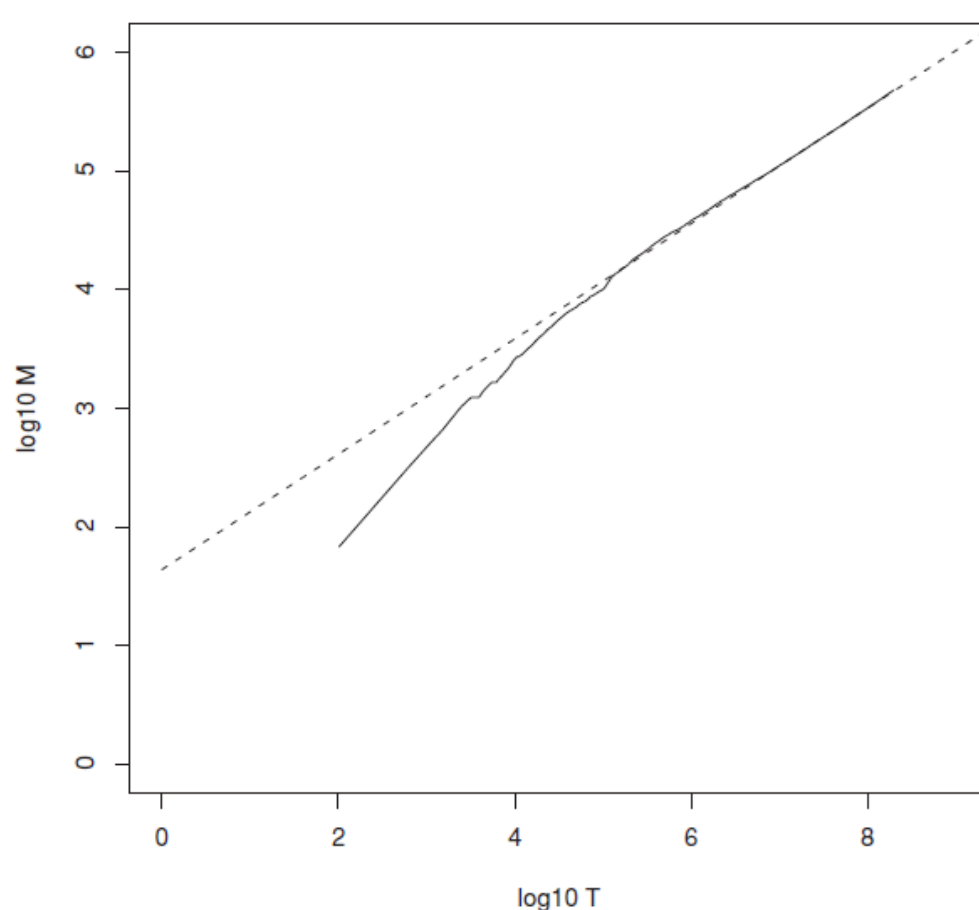
دیکشنری آکسفورد شامل اکثر نام افراد، مکان‌ها، محصولات یا موجودات علمی مانند ژن‌ها نمی‌شود. این نام‌ها باید در ایندکس معکوس گنجانده شوند تا کاربران بتوانند آن‌ها را جستجو کنند.

قانون هپس (Heaps' law) رابطه بین اندازه مجموعه (تعداد توکن‌ها) و اندازه واژگان (تعداد واژه‌های منحصر به فرد) را توصیف می‌کند. طبق این قانون، اندازه واژگان با افزایش اندازه مجموعه به صورت زیر رشد می‌کند:

$$M = k * T^b$$

که در آن M تعداد واژه‌های منحصر به فرد، T تعداد کل توکن‌ها، k و b ثابت‌هایی هستند که معمولاً k بین ۱۰ تا ۱۰۰ و b بین ۰.۴ تا ۰.۶ هستند.

برای مثال، در مجموعه Reuters-RCV1، بهترین برازش برای این رابطه $k = 44$ و $b = 0.49$ است. این یعنی با دو برابر کردن اندازه مجموعه، تعداد واژه‌های منحصر به فرد حدود ۴۰٪ افزایش می‌یابد، نه دو برابر!



► **Figure 5.1** Heaps' law. Vocabulary size M as a function of collection size T (number of tokens) for Reuters-RCV1. For these data, the dashed line $\log_{10} M = 0.49 * \log_{10} T + 1.64$ is the best least-squares fit. Thus, $k = 10^{1.64} \approx 44$ and $b = 0.49$.

شکل 5.1 – قانون هیپس. واژگان به اندازه M و تابعی از مجموعه ای به اندازه T (تعداد توکن ها) برای Reuters-RCV1. برای این داده ها، خط نقطه چین بهترین کوچکترین توان مناسب است.

در ادامه این فصل، با تکنیک‌های فشرده‌سازی دیکشنری (روش دیکشنری به صورت رشته و ذخیره‌سازی بلوکی) و فشرده‌سازی لیست‌های ارسال (کدگذاری بایت متغیر و کدگذاری گاما) آشنا خواهیم شد. این تکنیک‌ها به ما کمک می‌کنند تا سیستم‌های بازیابی اطلاعات کارآمدتر و سریع‌تری بسازیم.

۵.۱.۱ قانون هیپس: چرا واژگان اینقدر سریع رشد می‌کنند؟

🌐 یک مثال از دنیای واقعی: رشد واژگان در اینستاگرام

تصور کنید مدیر داده در اینستاگرام هستید و می‌خواهید پیش‌بینی کنید چقدر فضا برای ذخیره هشتگ‌های جدید نیاز دارید. در ابتدا، اینستاگرام فقط چند هزار هشتگ داشت، اما امروزه میلیون‌ها هشتگ منحصربه‌فرد وجود دارد. قانون هیپس به ما کمک می‌کند این رشد را پیش‌بینی کنیم!

برای تخمین تعداد واژه‌های متمایز M در یک مجموعه متنی با T توکن، از **قانون هیپس** استفاده می‌شود. این قانون رابطه‌ای تجربی بین اندازه مجموعه و اندازه واژگان ارائه می‌دهد:

$$M = k \cdot T^b$$

که در آن:

- T : تعداد توکن‌ها در مجموعه
- M : تعداد واژه‌های متمایز
- k : ضریب وابسته به نوع مجموعه (بین ۳۰ تا ۱۰۰)
- b : نمایی که معمولاً حدود ۰٫۵ است

🔍 چرا این قانون مهم است؟

قانون هیپس نشان می‌دهد که رشد واژگان با افزایش حجم متن متوقف نمی‌شود. این یعنی برای سیستم‌های بزرگی مانند گوگل یا اینستاگرام، همیشه باید برای واژگان جدید فضا در نظر گرفت. این دقیقاً همان چیزی است که فشرده‌سازی دیکشنری را برای سیستم‌های بازیابی اطلاعات حیاتی می‌کند.

در مجموعه Reuters-RCV1، برازش خطی زیر به‌دست آمده:

$$\log_{10} M = 0.49 \cdot \log_{10} T + 1.64$$

که معادل است با:

$$M = 44 \cdot T^{0.49}$$

مثال عددی: برای $T = 1,000,020$ توکن، قانون هیپس پیش‌بینی می‌کند:

$$M = 44 \cdot (1,000,020)^{0.49} \approx 38,323$$

در حالی که مقدار واقعی برابر با 38,365 واژه بوده — که بسیار نزدیک به پیش‌بینی است.

مقدار k وابسته به نوع مجموعه و نحوهٔ پیش‌پردازش آن است:

- استفاده از case folding و stemming باعث کاهش نرخ رشد واژگان می‌شود.
- وجود اعداد، اشتباهات املایی و اسامی خاص باعث افزایش واژگان می‌شود.

نتیجه‌گیری مهم:

- اندازهٔ واژگان با افزایش اسناد همچنان رشد می‌کند و به سقف نمی‌رسد.
- برای مجموعه‌های بزرگ، اندازهٔ دیکشنری بسیار زیاد خواهد بود.
- بنابراین، **فشرده‌سازی دیکشنری** برای کارایی سیستم بازیابی اطلاعات ضروری است.

۵.۱.۲ قانون زیپف: چرا برخی کلمات بسیار بیشتر از بقیه تکرار می‌شوند؟ 📊

🇮🇷 یک مثال از دنیای واقعی: تحلیل توییت‌های توییتر

تصور کنید محققان توییتر (X) می‌خواهند بفهمند کلمات در توییت‌های فارسی چگونه توزیع شده‌اند. آنها متوجه می‌شوند که کلمه «از» در میلیون‌ها توییت ظاهر می‌شود، در حالی که کلمه «الگوریتم» بسیار نادر است. این توزیع ناهمگون چیزی است که قانون زیپف توصیف می‌کند!

برای طراحی الگوریتم‌های فشرده‌سازی لیست‌های ارسال، باید بدانیم واژه‌ها چگونه در مجموعه اسناد توزیع شده‌اند. یکی از مدل‌های رایج برای این هدف، **قانون زیپف** است.

این قانون بیان می‌کند که اگر واژه t_1 رایج‌ترین واژه باشد، t_2 دومین واژه رایج، و همین‌طور تا t_i ، آنگاه فراوانی مجموعه‌ای (collection frequency) واژه t_i به صورت زیر است:

$$cf_i \propto \frac{1}{i}$$

به زبان ساده:

- اگر رایج‌ترین واژه cf_1 بار ظاهر شود
- واژه دوم تقریباً $cf_1/2$ بار
- واژه سوم تقریباً $cf_1/3$ بار
- و همین‌طور الگو ادامه می‌یابد...

🔍 اهمیت عملی قانون زیپف

در تحلیل داده‌های توییتر، محققان متوجه شدند کلماتی مانند «از»، «به» و «در» تقریباً در هر توییت ظاهر می‌شوند، در حالی که کلمات تخصصی مانند «الگوریتم» یا «فشرده‌سازی» بسیار نادر هستند. این توزیع ناهمگون به ما کمک می‌کند تا الگوریتم‌های فشرده‌سازی بهینه طراحی کنیم.

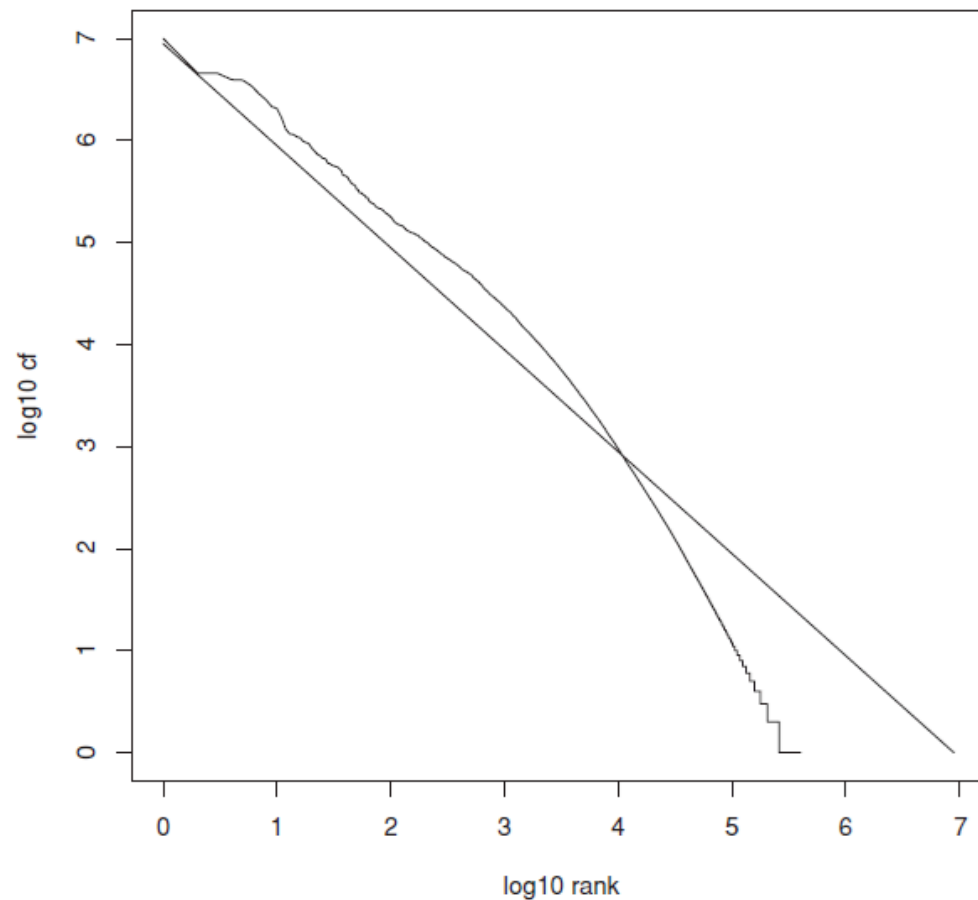
فرمول زیپف را می‌توان به صورت نمایی یا لگاریتمی نیز نوشت:

$$cf_i = c \cdot i^k \text{ with } k = -1$$

یا به صورت لگاریتمی:

$$\log cf_i = \log c + k \cdot \log i$$

که در آن c یک ثابت است و $k = -1$. این رابطه یک **قانون توانی (power law)** است که در بسیاری از پدیده‌های طبیعی و اجتماعی مشاهده می‌شود.



► **Figure 5.2** Zipf's law for Reuters-RCV1. Frequency is plotted as a function of frequency rank for the terms in the collection. The line is the distribution predicted by Zipf's law (weighted least-squares fit; intercept is 6.95).

نمودار بالا برای مجموعه **Reuters-RCV1**، فراوانی واژه‌ها را بر حسب رتبه‌شان نشان می‌دهد. خط با شیب 1- تقریب خوبی از داده‌ها ارائه می‌دهد، هرچند کاملاً دقیق نیست.

✿ کاربرد در فشرده‌سازی

قانون زیپف به ما می‌گوید که:

- بیشتر واژه‌ها بسیار کم‌تکرار هستند
- فشرده‌سازی لیست‌های ارسال برای واژه‌های رایج بسیار مؤثر است
- می‌توانیم الگوریتم‌های متفاوتی برای واژه‌های پرتکرار و کم‌تکرار طراحی کنیم

در بخش 5.3، خواهیم دید که چگونه این درک از توزیع واژه‌ها به طراحی الگوریتم‌های فشرده‌سازی بهینه کمک می‌کند.

■ ۵.۲ فشرده‌سازی دیکشنری: چرا حتی بخش کوچک هم مهم است؟

🛒 یک مثال از دنیای واقعی: جستجوی محصولات در آمازون

تصور کنید مدیر موتور جستجوی آمازون هستید. هر ثانیه، هزاران کاربر در حال جستجوی محصولات هستند. وقتی کاربری عبارت "لپ‌تاپ گیمینگ" را جستجو می‌کند، سیستم باید فوراً واژه‌های "لپ‌تاپ" و "گیمینگ" را در دیکشنری پیدا کند. اگر

دیکشنری در حافظه اصلی نباشد، هر جستجو نیازمند چندین دسترسی به دیسک خواهد بود که باعث تأخیر قابل توجهی می‌شود. در دنیای رقابتی تجارت الکترونیک، حتی تأخیر ۱۰۰ میلی‌ثانیه‌ای می‌تواند منجر به از دست رفتن مشتری شود!

در نگاه اول، دیکشنری نسبت به فایل لیست‌های ارسال (postings file) بسیار کوچک‌تر است. پس چرا باید آن را فشرده کنیم؟ پاسخ در **کارایی سیستم بازیابی اطلاعات** نهفته است.

یکی از عوامل اصلی در زمان پاسخ‌دهی سیستم IR، تعداد **disk seek**هایی است که برای پردازش یک پرسمان انجام می‌شود. اگر بخشی از دیکشنری روی دیسک باشد، هر جست‌وجو نیازمند چندین seek خواهد بود. بنابراین، هدف اصلی فشرده‌سازی دیکشنری این است که بتوان آن را به‌طور کامل (یا حداقل بخش بزرگی از آن) در حافظهٔ اصلی نگه داشت.

**مزایای فشرده‌سازی دیکشنری در دنیای واقعی**

در شرکت‌های بزرگی مانند گوگل یا مایکروسافت که باید میلیاردها سند را ایندکس کنند، فشرده‌سازی دیکشنری به آنها اجازه می‌دهد:

- کاهش تعداد disk seek در زمان جست‌وجو (هر seek معادل چند میلی‌ثانیه تأخیر است)
- افزایش throughput سیستم در پردازش پرسمان‌ها (مهم برای سرویس‌هایی با ترافیک بالا)
- امکان اجرای سیستم روی سخت‌افزارهای محدود (مثل موبایل یا کامپیوترهای جاسازی‌شده در خودروها)
- کاهش زمان راه‌اندازی سیستم (مهم برای سرویس‌های ابری که باید سریعاً راه‌اندازی شوند)
- هم‌زیستی بهتر با نرم‌افزارهای دیگر در حافظه (در محیط‌های اشتراکی)

شکل زیر یک ساختار ساده برای ذخیره‌سازی دیکشنری را نشان می‌دهد:

term	document frequency	pointer to postings list
a	656,265	→
aachen	65	→
...	...	...
zulu	221	→

در این ساختار، هر ورودی به‌صورت **fixed-width entry** ذخیره می‌شود. به‌طور مثال:

- term: 20 bytes
- document frequency: 4 bytes
- pointer to postings list: 4 bytes

**چالش در دنیای واقعی**

برای یک شرکت بزرگ بین‌المللی که اسناد را به زبان‌های مختلف ایندکس می‌کند، دیکشنری می‌تواند به راحتی از حافظه یک کامپیوتر استاندارد فراتر رود. در چنین مواردی، فشرده‌سازی دیکشنری نه یک انتخاب، بلکه یک ضرورت فنی است. در ادامه این بخش، با روش‌های مختلف فشرده‌سازی دیکشنری آشنا می‌شویم که به ما کمک می‌کنند تا حتی در محیط‌های محدود نیز سیستم‌های جستجوی کارآمد داشته باشیم.

🔍 یک مثال از دنیای واقعی: بهینه‌سازی جستجو در اسناد پزشکی

تصور کنید در شرکت IBM Watson Health کار می‌کنید و در حال طراحی سیستم جستجوی پزشکی هستید که به پزشکان کمک می‌کند در میلیون‌ها مقاله پزشکی جستجو کنند. در این حوزه، واژه‌های تخصصی و طولانی مانند "hydrochlorofluorocarbons" یا "immunohistochemistry" بسیار رایج هستند. اگر از روش با طول ثابت استفاده کنید، بسیاری از این واژه‌ها کوتاه نمی‌آیند و فضای زیادی هدر می‌رود. اینجاست که روش رشته‌ای به کمک شما می‌آید!

ساده‌ترین ساختار برای ذخیره‌سازی دیکشنری، استفاده از آرایه‌ای از ورودی‌های با طول ثابت (fixed-width) است. در این روش، واژگان به صورت مرتب‌شده الفبایی ذخیره می‌شوند و هر ورودی شامل موارد زیر است:

- term: 20 bytes
- document frequency: 4 bytes
- pointer to postings list: 4 bytes

برای مجموعه **Reuters-RCV1** با 400,000 واژه، حافظه موردنیاز برابر است با:

$$400,000 \times (20 + 4 + 4) = 11.2 \text{ MB}$$

⚠️ مشکلات روش با طول ثابت

این روش بسیار ناکارآمد است، چون:

- طول متوسط واژه‌ها در انگلیسی حدود ۸ کاراکتر است ← ۱۲ بایت هدر در هر ورودی
- واژه‌های طولانی‌تر از ۲۰ کاراکتر قابل ذخیره‌سازی نیستند

در سیستم جستجوی پزشکی Watson، این مشکل جدی‌تر است، چون بسیاری از واژه‌های پزشکی بسیار طولانی هستند و نمی‌توانند در ۲۰ کاراکتر جای بگیرند!

راه‌حل: ذخیره‌سازی واژه‌ها به صورت یک رشته طولانی از کاراکترها (dictionary-as-a-string). در این روش:

- همه واژه‌ها پشت سر هم در یک رشته ذخیره می‌شوند
- هر واژه با یک **term pointer** مشخص می‌شود (آدرس شروع واژه)
- جست‌وجو با **binary search** روی جدول اشاره‌گرها انجام می‌شود

🧮 محاسبه فضای ذخیره‌سازی

برای 400,000 واژه با طول متوسط ۸ کاراکتر، تعداد موقعیت‌های ممکن برابر است با:

$$400,000 \times 8 = 3.2 \times 10^6$$

برای آدرس‌دهی این تعداد موقعیت، به اشاره‌گرهایی با طول زیر نیاز داریم:

$$\log_2(3.2 \times 10^6) \approx 22 \text{ bits} \Rightarrow 3 \text{ bytes}$$

در این ساختار جدید، حافظه موردنیاز برابر است با:

- term: 8 bytes (میانگین)
- term pointer: 3 bytes

- document frequency: 4 bytes
- postings pointer: 4 bytes

$$400,000 \times (8 + 3 + 4 + 4) = 7.6 \text{ MB}$$

بنابراین، با استفاده از dictionary-as-a-string حدود ۳۳% صرفه‌جویی در حافظه نسبت به روش fixed-width خواهیم داشت.

نحوه کارکرد این روش

در روش dictionary-as-a-string، واژه‌ها به صورت یک رشته پیوسته ذخیره می‌شوند و اشاره‌گرها ابتدای هر واژه را مشخص می‌کنند. برای مثال، اگر سه واژه "syzygetic"، "systile" و "syzygial" را داشته باشیم، در رشته به این شکل ذخیره می‌شوند:

```
...s y s t i l e s y z y g e t i c s y z y g i a l...
```

اشاره‌گر اول به ابتدای "systile"، اشاره‌گر دوم به ابتدای "syzygetic" و اشاره‌گر سوم به ابتدای "syzygial" اشاره می‌کند. این ساختار به ما اجازه می‌دهد تا با جستجوی دودویی در جدول اشاره‌گرها، واژه مورد نظر را به سرعت پیدا کنیم.

۵.۲.۲ فشرده‌سازی بلوکی و کدگذاری پیشوندی

یک مثال از دنیای واقعی: بهینه‌سازی جستجو در اپلیکیشن موبایل

تصور کنید در تیم توسعه اپلیکیشن Spotify کار می‌کنید و مسئول بهینه‌سازی جستجوی آهنگ‌ها در نسخه موبایل هستید. اپلیکیشن شما باید بتواند نام صدها هزار هنرمند، آلبوم و آهنگ را در حافظه محدود موبایل ذخیره کند و همزمان جستجوی سریعی را ارائه دهد. فشرده‌سازی بلوکی و کدگذاری پیشوندی به شما اجازه می‌دهد تا دیکشنری بزرگ را در حافظه محدود موبایل جا دهید و همزمان سرعت جستجو را قابل قبول نگه دارید.

برای کاهش بیشتر اندازه دیکشنری، واژه‌ها را به بلوک‌هایی با اندازه k تقسیم می‌کنیم و فقط برای واژه اول هر بلوک یک اشاره‌گر ذخیره می‌کنیم. طول هر واژه نیز به صورت یک بایت در ابتدای آن ذخیره می‌شود.

برای $k = 4$:

- صرفه‌جویی در اشاره‌گرها: $9 \times (k - 1) = 9$ bytes
- هزینه اضافه برای طول واژه‌ها: $k = 4$ bytes
- صرفه‌جویی خالص: 5 bytes per block
- تعداد بلوک‌ها: $400,000 / 4 = 100,000$
- صرفه‌جویی کل: $100,000 \times 5 = 0.5 \text{ MB}$

حافظه نهایی: $7.6 - 0.5 = 7.1 \text{ MB}$

معامله بین فضا و سرعت

افزایش k باعث کاهش بیشتر حافظه می‌شود، اما زمان جست‌وجو افزایش می‌یابد. در اپلیکیشن Spotify، این معامله بسیار مهم است:

- جست‌وجو در دیکشنری بدون فشرده‌سازی: میانگین 1.6 مرحله
- جست‌وجو با $k = 4$: میانگین 2 مرحله ← افزایش 25%
- جست‌وجو با $k = 8$ یا $k = 16$: کندتر خواهد شد و تجربه کاربری را تحت تأثیر قرار می‌دهد

برای اپلیکیشن موبایل که باید پاسخگو باشد، تیم Spotify احتمالاً از $k = 4$ استفاده می‌کند تا تعادل خوبی بین فضا و سرعت برقرار شود.

ابو کدگذاری پیشوندی (Front Coding)

تکنیک **front coding** از اشتراک پیشوند بین واژه‌های متوالی استفاده می‌کند. در اپلیکیشن Spotify، بسیاری از نام هنرمندان با پیشوندهای مشترک شروع می‌شوند:

```
9The Beatles
10The Rolling Stones
8The Doors
8The Who
  ↓
3The*
6♦Beatles
12♦Rolling Stones
5♦Doors
3♦Who
```

در این روش، پیشوند مشترک با * علامت‌گذاری می‌شود و ادامه واژه‌ها با ♦ و طول مشخص می‌شوند. این تکنیک در مجموعه Reuters باعث صرفه‌جویی 1.2 MB دیگر شد.

📊 مقایسه نهایی ساختارهای دیکشنری

جدول زیر مقایسه‌ای از فضا مورد نیاز برای روش‌های مختلف فشرده‌سازی دیکشنری را نشان می‌دهد:

ساختار	حافظه (MB)
Fixed-width	11.2
Term pointers into string	7.6
Blocking (k = 4)	7.1
Blocking + Front coding	5.9

همانطور که می‌بینید، با ترکیب این تکنیک‌ها می‌توانیم تقریباً 50% در فضا صرفه‌جویی کنیم، که برای اپلیکیشن‌های موبایل با حافظه محدود بسیار حیاتی است.

📦 ۵.۳ فشرده‌سازی فایل لیست‌های ارسال: هنر ذخیره هوشمندانه

🔍 یک مثال از دنیای واقعی: بهینه‌سازی جستجو در گوگل

تصور کنید مهندس گوگل هستید و مسئول بهینه‌سازی ایندکس جستجو برای میلیاردها صفحه وب هستید. هر روز، میلیون‌ها صفحه جدید به ایندکس اضافه می‌شود و کاربران میلیاردها جستجو انجام می‌دهند. اگر هر لیست ارسال را بدون فشرده‌سازی ذخیره کنید، به هزاران سرور با حافظه عظیم نیاز خواهید داشت! فشرده‌سازی لیست‌های ارسال به گوگل اجازه می‌دهد تا با منابع کمتر، پاسخ‌های سریع‌تری به کاربران ارائه دهد.

در این بخش، هدف ما کاهش اندازه فایل لیست‌های ارسال (postings file) است. برای مجموعه **Reuters-RCV1**:

- تعداد اسناد: 800,000
- تعداد توکن‌ها در هر سند: 200
- طول هر توکن: 6 کاراکتر
- تعداد کل postings: 100,000,000

بنابراین:

- اندازه مجموعه: $800,000 \times 200 \times 6 = 960 \text{ MB}$
- اندازه فایل لیست‌های ارسال بدون فشرده‌سازی: $100,000,000 \times \frac{20}{8} = 250 \text{ MB}$

💡 ایده کلیدی: کدگذاری فاصله‌ها (Gap Encoding)

برای کاهش این حجم، از ایده **gap encoding** استفاده می‌کنیم. به جای ذخیره مستقیم docIDها، فاصله بین آن‌ها را ذخیره می‌کنیم:

term	docIDs	gaps
the	283042, 283043, 283044, 283045	1, 1, 1
computer	283047, 283154, 283159, 283202	43, 5, 107
arachnocentric	500100, 252000	248100, 252000

برای واژه‌های رایج مانند "the"، فاصله‌ها بسیار کوچک هستند (معمولاً ۱) و می‌توان آن‌ها را با بیت‌های بسیار کمتر ذخیره کرد. اما برای واژه‌های نادر مانند "arachnocentric"، فاصله‌ها بزرگ هستند و همچنان نیاز به بیت‌های کامل دارند.

🔪 روش‌های فشرده‌سازی

برای فشرده‌سازی این فاصله‌ها، از دو نوع روش استفاده می‌کنیم:

- Bytewise Compression**: فشرده‌سازی با حداقل تعداد بایت
- Bitwise Compression**: فشرده‌سازی با حداقل تعداد بیت

در گوگل، هر دو روش استفاده می‌شود. برای واژه‌های بسیار رایج که فاصله‌های کوچکی دارند، روش bitwise بسیار کارآمد است، زیرا می‌تواند فاصله ۱ را تنها با یک بیت ذخیره کند. برای واژه‌های کمتر رایج، روش bitwise ممکن است مناسب‌تر باشد.

در بخش‌های بعدی، دو الگوریتم اصلی از هر دسته را بررسی و پیاده‌سازی می‌کنیم:

- Variable Byte Encoding (bytewise)
- Gamma Encoding (bitwise)

در سیستم‌های بزرگ مانند گوگل یا آمازون، فشرده‌سازی لیست‌های ارسال فقط یک بهینه‌سازی کوچک نیست، بلکه یک ضرورت فنی است. بدون این تکنیک‌ها، هزینه ذخیره‌سازی و پردازش ایندکس‌ها به قدری بالا می‌رود که ارائه خدمات جستجوی سریع و مقرون‌به‌صرفه غیرممکن می‌شود. با فشرده‌سازی هوشمندانه، این شرکت‌ها می‌توانند میلیاردها سند را با هزینه کمتر ایندکس کرده و پاسخ‌های سریع‌تری به کاربران ارائه دهند.

۵.۳.۱ کدگذاری بایتی متغیر: هنر ذخیره هوشمندانه اعداد

یک مثال از دنیای واقعی: بهینه‌سازی جستجو در واتساپ

تصور کنید در تیم واتساپ کار می‌کنید و مسئول بهینه‌سازی جستجوی پیام‌ها هستید. هر روز میلیاردها پیام ارسال می‌شود و کاربران باید بتوانند سریعاً پیام‌های قدیمی خود را پیدا کنند. اگر هر شناسه پیام را با ۴ بایت ذخیره کنید، حجم ایندکس به سرعت افزایش می‌یابد. با استفاده از کدگذاری بایتی متغیر، واتساپ می‌تواند شناسه‌های پیام‌های نزدیک به هم را با فضای بسیار کمتری ذخیره کند و همزمان سرعت جستجو را بالا نگه دارد.

در روش کدگذاری بایتی متغیر (Variable Byte Encoding)، هر عدد (مثلاً فاصله بین docIDها) با تعداد متغیری از بایت‌ها ذخیره می‌شود. این روش از یک ایده ساده اما هوشمندانه استفاده می‌کند:

ساختار یک بایت در کدگذاری VB

هر بایت در این روش شامل دو بخش است:

- ۷ بیت داده (payload): بخشی از عدد را ذخیره می‌کند
- ۱ بیت ادامه (continuation bit): نشان می‌دهد آیا این بایت آخرین بایت عدد است یا خیر

اگر continuation bit برابر ۱ باشد ← این بایت آخرین بایت عدد است

اگر continuation bit برابر ۰ باشد ← عدد در بایت‌های بعدی ادامه دارد

برای مثال، عدد 214577 به صورت زیر کدگذاری می‌شود:

$214577 = 00001101\ 00001100\ 10110001 \rightarrow 3\ bytes: [13, 12, 177] \rightarrow continuation\ bits: 0, 0, 1$

فرآیند کدگذاری و رمزگشایی

برای کدگذاری یک عدد:

1. عدد را به مبنای 128 می‌بریم (چون 7 بیت برای داده داریم)
2. بایت‌ها را از پایین به بالا تولید می‌کنیم
3. به آخرین بایت، 128 اضافه می‌کنیم تا continuation bit آن 1 شود

برای رمزگشایی:

1. بایت‌ها را یکی یکی می‌خوانیم

2. تا زمانی که continuation bit صفر است، عدد را با ضرب در 128 ادامه می‌دهیم

3. وقتی continuation bit برابر ۱ شد، عدد کامل شده است

```
VBENCODENUMBER(n)
1  bytes ← {}
2  while true
3  do PREPEND(bytes, n mod 128)
4    if n < 128
5      then BREAK
6    n ← n div 128
7  bytes[LENGTH(bytes)] += 128
8  return bytes

VBENCODE(numbers)
1  bytestream ← {}
2  for each n ∈ numbers
3  do bytes ← VBENCODENUMBER(n)
4    bytestream ← EXTEND(bytestream, bytes)
5  return bytestream

VBDECODE(bytestream)
1  numbers ← {}
2  n ← 0
3  for i ← 1 to LENGTH(bytestream)
4  do if bytestream[i] < 128
5    then n ← 128 × n + bytestream[i]
6    else n ← 128 × n + (bytestream[i] − 128)
7    APPEND(numbers, n)
8    n ← 0
9  return numbers
```

► Figure 5.8 VB encoding and decoding. The functions div and mod compute integer division and remainder after integer division, respectively. PREPEND adds an element to the beginning of a list, for example, PREPEND($\langle 1, 2 \rangle$, 3) = $\langle 3, 1, 2 \rangle$. EXTEND extends a list, for example, EXTEND($\langle 1, 2 \rangle$, $\langle 3, 4 \rangle$) = $\langle 1, 2, 3, 4 \rangle$.

شکل 5.8 – کدگذاری و کدگشایی بایتی متغیر. توابع div و mod به ترتیب تقسیم صحیح و باقی مانده آن را محاسبه می‌کنند. Prepend یک عنصر

را به ابتدای لیست اضافه می‌کند. به عنوان مثال: $\text{Prepend}(\langle 1, 2 \rangle, 3) = \langle 3, 1, 2 \rangle$. Extend یک لیست را بسط می‌دهد. به عنوان مثال :

$\text{Extend}(\langle 1, 2 \rangle, \langle 3, 4 \rangle) = \langle 1, 2, 3, 4 \rangle$.

در آزمایش روی مجموعه Reuters-RCV1، اندازه فایل postings از 250MB به 116MB کاهش یافت – یعنی بیش از ۵۰٪ صرفه‌جویی در فضا.

مزایا و معایب

کدگذاری VB تعادلی عالی بین سرعت و فشردگی ارائه می‌دهد:

- **مزیت:** پیاده‌سازی ساده، سرعت رمزگشایی بالا، و فشردگی قابل قبول
- **معایب:** برای اعداد بسیار بزرگ (مانند فاصله‌های بزرگ در واژه‌های نادر) کارایی کمتری دارد

برای اکثر سیستم‌های بازیابی اطلاعات، این روش یک انتخاب عالی است. اما اگر فضا بسیار محدود است، می‌توان از روش‌های سطح بیت (مانند γ codes) که در بخش بعدی معرفی می‌شوند، استفاده کرد.

In [5]:

```
def vb_encode_number(n):
    """
    این تابع یک عدد را با استفاده از روش کدگذاری بایتی متغیر کدگذاری می‌کند.
    است (continuation bit) هر بایت شامل 7 بیت داده و 1 بیت ادامه
    (MSB=1). بیت ادامه برای تمام بایت‌ها به جز آخرین بایت تنظیم می‌شود.
    """
    # حالت خاص برای عدد صفر، یک بایت با مقدار صفر برمی‌گرداند
    if n == 0:
        return [0]

    bytes_ = []
    while True:
        # استخراج 7 بیت کم اهمیت
        byte = n & 0x7F
        n >>= 7

        # اگر هنوز بیت‌های باقی‌مانده وجود دارد، بیت ادامه را تنظیم کن
        if n > 0:
            byte |= 0x80

        bytes_.append(byte)
        if n == 0:
            break

    return bytes_

# (ها gap مثلاً) تابع کدگذاری لیستی از اعداد
def vb_encode(numbers):
    """
    این تابع لیستی از اعداد را با استفاده از روش کدگذاری بایتی متغیر کدگذاری می‌کند.
    """
    bytestream = []
    for n in numbers:
        bytestream.extend(vb_encode_number(n))
    return bytestream

# تابع رمزگشایی لیست بایت‌ها به لیست اعداد
def vb_decode(bytestream):
    """
    این تابع لیست بایت‌های کدگذاری شده را به لیست اعداد اصلی بازمی‌گرداند.
    """
    numbers = []
    n = 0
    shift = 0 # شیفต์ برای قرار دادن بایت‌ها در موقعیت صحیح

    for byte in bytestream:
        # استخراج 7 بیت داده
        data = byte & 0x7F
        # قرار دادن بیت‌ها در موقعیت صحیح
        n |= (data << shift)

        # Fixed: این آخرین بایت عدد است، (byte < 128) اگر بیت ادامه تنظیم نشده باشد
        if byte < 128:
            numbers.append(n)
            n = 0
            shift = 0
        else:
            # در غیر این صورت، برای بایت بعدی شیفต์ را 7 واحد افزایش می‌دهیم
            shift += 7

    return numbers

# تابع محاسبه حجم قبل و بعد از فشرده‌سازی
def compare_space(gaps):
    """
    ها را قبل و بعد از فشرده‌سازی مقایسه می‌کند gap این تابع حجم لیست
    """
    # حجم قبل از فشرده‌سازی (هر عدد 4 بایت)
    original_size = len(gaps) * 4

    # حجم بعد از فشرده‌سازی
    encoded = vb_encode(gaps)
    compressed_size = len(encoded)

    return original_size, compressed_size

# مثال با داده‌های واقعی از کتاب
gaps = [5, 214577]
encoded = vb_encode(gaps)
decoded = vb_decode(encoded)

print("Gaps:", gaps)
print("Encoded bytes:", encoded)
print("Decoded:", decoded)
print()

# مقایسه حجم قبل و بعد از فشرده‌سازی
original_size, compressed_size = compare_space(gaps)
print(f"Original size: {original_size} bytes")
print(f"Compressed size: {compressed_size} bytes")
print(f"Compression ratio: {compressed_size/original_size:.2f}")

# ها gap مثال با لیست بزرگتر از
large_gaps = [1, 1, 1, 1, 1, 107, 5, 43, 252000, 248100]
original_size, compressed_size = compare_space(large_gaps)
print()
print("Large gaps example:")
print(f"Original size: {original_size} bytes")
print(f"Compressed size: {compressed_size} bytes")
print(f"Compression ratio: {compressed_size/original_size:.2f}")
```

Gaps: [5, 214577]
Encoded bytes: [5, 177, 140, 13]
Decoded: [5, 214577]

Original size: 8 bytes
Compressed size: 4 bytes
Compression ratio: 0.50

Large gaps example:
Original size: 40 bytes
Compressed size: 14 bytes
Compression ratio: 0.35

۵.۳.۲ کدگذاری گاما: هنر فشرده‌سازی در سطح بیت ¹²₃₄

🔍 یک مثال از دنیای واقعی: بهینه‌سازی جستجو در گوگل

تصور کنید در گوگل کار می‌کنید و مسئول بهینه‌سازی ایندکس وب هستید. گوگل باید میلیاردها صفحه وب را ایندکس کند و برای هر کلمه، لیستی از شناسه‌های صفحات (docID) را ذخیره کند. برای کلمه‌ای مانند "HTML" که در صفحات زیادی وجود دارد، لیست docIDها بسیار طولانی است. اگر هر docID را با ۳۲ بیت ذخیره کنید، حجم ایندکس به ترابایت‌ها می‌رسد! با استفاده از کدهای گاما، گوگل می‌تواند فاصله بین docIDها را با بیت‌های بسیار کمتر ذخیره کند.

در روش کدگذاری گاما (γ encoding)، هر عدد (مثلاً فاصله بین docIDها) به دو بخش تقسیم می‌شود:

✂️ ساختار کد گاما

هر عدد G به دو بخش تقسیم می‌شود:

- **Offset:** نمایش دودویی G بدون بیت اول (مثلاً برای ۱۳ ← ۱۱۰۱ ← ۱۰۱)
- **Length:** طول offset به‌صورت کد unary (برای ۳ بیت ← ۱۱۱۰)

کد نهایی: اتصال length و offset ← برای ۱۳ می‌شود: ۱۱۱۰۱۰۱

بیاپید فرآیند کدگذاری و رمزگشایی را با یک مثال واقعی ببینیم. فرض کنید در ایندکس گوگل، docIDهای زیر برای کلمه "machine learning" وجود دارند:

docIDs: 824, 829, 215406

gaps: 5, 214577

🔄 فرآیند کدگذاری عدد ۲۱۴۵۷۷:

۱. عدد ۲۱۴۵۷۷ به دودویی تبدیل می‌شود: ۱۱۰۱۰۰۰۱۱۰۱۰۰۰۱
۲. بیت اول حذف می‌شود: ۱۰۱۰۰۰۱۱۰۱۰۰۰۱ (offset)
۳. طول offset محاسبه می‌شود: ۱۵ بیت
۴. طول ۱۵ به صورت unary نوشته می‌شود: ۱۵ بار ۱ و سپس ۰ ← ۱۱۱۱۱۱۱۱۱۱۱۱۱۱۰
۵. کد نهایی: ۱۱۱۱۱۱۱۱۱۱۱۱۱۱۱۱۱۰۱۰۱۰۰۰۱۱۰۱۰۰۰۱ = ۱۰۱۰۰۰۱۱۰۱۰۰۰۱ + ۱۱۱۱۱۱۱۱۱۱۱۱۱۱۱۱۱۰

برای رمزگشایی:

۱. ابتدا unary code تا رسیدن به ۰ می‌خوانیم (۱۵ بار ۱ و سپس ۰) ← طول offset = ۱۵ بیت
۲. سپس ۱۵ بیت بعدی را می‌خوانیم (offset)

ویژگی‌های برجسته کدهای گاما

کدهای گاما سه ویژگی کلیدی دارند که آنها را برای فشرده‌سازی ایندکس ایده‌آل می‌کند:

- **Universal:** برای هر توزیع احتمالاتی، نزدیک به کد بهینه است. این یعنی بدون اینکه توزیع فاصله‌ها را بدانیم، می‌توانیم از این کد استفاده کنیم.
- **Prefix-free:** هیچ کد گاما پیشوند دیگری نیست. این ویژگی رمزگشایی را ساده می‌کند، چون نیازی به جداکننده بین کدها نیست.
- **Parameter-free:** نیازی به تنظیم پارامتر ندارد. این ویژگی برای ایندکس‌های پویا که توزیع فاصله‌ها ممکن است تغییر کند، بسیار مهم است.

طول کد γ برای عدد G برابر است با:

$$\text{length} = 2 \cdot \lfloor \log_2 G \rfloor + 1$$

این کد همیشه طولی فرد دارد و در بدترین حالت، حداکثر دو برابر حد پایین اطلاعاتی است.

کارایی در دنیای واقعی

در مجموعه Reuters-RCV1، کدگذاری گاما باعث کاهش اندازه فایل postings از ۲۵۰MB به ۱۰۱MB شد – حدود ۱۵% بهتر از Variable Byte Encoding. این صرفه‌جویی در مقیاس وب، که گوگل باید میلیاردها صفحه را ایندکس کند، به ترابایت‌ها فضا می‌رسد!

برای تخمین اندازه ایندکس فشرده‌شده با γ ، از مدل Zipf و فرمول زیر استفاده می‌شود:

$$\text{total bits} \approx \sum_{j=1}^{M/L_c} 2NL_c \cdot \frac{\log_2 j}{j}$$

که برای $M = 400,000$ و $L_c = 15$ تقریباً برابر با ۲۲۴MB می‌شود – اما مقدار واقعی ۱۰۱MB است چون فرض‌های مدل ساده‌سازی شده‌اند.

کدهای گاما تعادل خوبی بین فشرده‌سازی و سرعت رمزگشایی ارائه می‌دهند. اگرچه رمزگشایی آنها از Variable Byte Encoding کندتر است، اما فشرده‌سازی بهتری ارائه می‌دهند. انتخاب بین این دو روش به نیازهای کاربرد بستگی دارد: اگر فضا محدود است، γ codes انتخاب بهتری هستند؛ اگر سرعت پردازش پرسمان مهم‌تر است، Variable Byte Encoding ممکن است مناسب‌تر باشد.

```
In [7]: # Gamma Encoding (تابع کدگذاری یک عدد با استفاده از روش گاما)
def gamma_encode(n):
    """
    این تابع یک عدد صحیح را با استفاده از روش کدگذاری گاما کدگذاری می‌کند.
    فرایند به این صورت است:
    1. عدد را به فرم دودویی تبدیل می‌کنیم.
    2. (offset) بیت اول را حذف کرده.
    3. (length) کدگذاری می‌کنیم unary را محاسبه کرده و به صورت offset طول.
    4. length و offset را به هم متصل می‌کنیم.
    """
    if n <= 0:
        raise ValueError("کدگذاری گاما فقط برای اعداد صحیح مثبت پشتیبانی می‌شود")

    # مرحله 1: تبدیل به دودویی و حذف پیشوند 0
    binary = bin(n)[2:]

    # مرحله 2: حذف بیت اول برای ساخت
```

```
offset = binary[1:]

# length برای unary و ساخت کد offset مرحله 3: محاسبه طول
length = '1' * len(offset) + '0'

# offset و length مرحله 4: اتصال
return length + offset

# تابع رمزگشایی یک کد گاما
def gamma_decode(code):
    """
    این تابع یک کد گاما را به عدد اصلی بازمی‌گرداند.
    فرآیند به این صورت است:
    1. (offset طول) تعداد 1های اولیه را می‌شمارد تا به 0 برسیم.
    2. را می‌خوانیم offset به تعداد بیت.
    3. اضافه کرده و به دودویی تبدیل می‌کنیم offset یک 1 به ابتدای.
    """

    # offset مرحله 1: شمارش تعداد 1ها برای پیدا کردن طول
    length = 0
    i = 0
    while i < len(code) and code[i] == '1':
        length += 1
        i += 1

    # offset مرحله 2: خواندن
    offset = code[i+1:i+1+length]

    # مرحله 3: بازسازی عدد اصلی
    return int('1' + offset, 2)

# تابع رمزگشایی یک جریان از کدهای گاما
def gamma_decode_stream(bit_stream):
    """
    این تابع یک جریان از بیت‌ها را که شامل چندین کد گاما است را رمزگشایی می‌کند.
    این تابع برای نمایش نحوه کار با لیست‌های ارسال فشرده‌شده مفید است.
    """

    numbers = []
    i = 0
    while i < len(bit_stream):
        # پیدا کردن طول کد بعدی
        length = 0
        while i < len(bit_stream) and bit_stream[i] == '1':
            length += 1
            i += 1

        if i >= len(bit_stream):
            break

        # offset خواندن
        offset = bit_stream[i+1:i+1+length]

        # بازسازی عدد و اضافه به لیست
        numbers.append(int('1' + offset, 2))

        # حرکت به ابتدای کد بعدی
        i += 1 + length

    return numbers

# تست توابع با اعداد نمونه از کتاب و مثال‌های واقعی
test_numbers = [1, 2, 3, 4, 9, 13, 24, 511, 1025, 5, 214577]

print("تست کدگذاری و رمزگشایی:")
print("-" * 60)
print(f"{'رمزگشایی':<10}>: {'10'} | {'طول کد':<10}>: {'10'} | {'کد گاما':<10}>: {'25'} | {'عدد':<10}>: {'10'}}")
print("-" * 60)

for n in test_numbers:
    encoded = gamma_encode(n)
    decoded = gamma_decode(encoded)
    print(f"{n:<10} | {encoded:<25} | {len(encoded):<10} | {decoded:<10}")

print("\n\nرمزگشایی جریان بیت‌ها\n")
print("-" * 60)
# ایجاد یک جریان از کدهای گاما برای تست
stream = ''.join([gamma_encode(n) for n in test_numbers])
decoded_stream = gamma_decode_stream(stream)
print(f"جریان بیت‌ها: {stream[:50]}...")
print(f"اعداد رمزگشایی شده: {decoded_stream}")

# محاسبه و نمایش طول کد برای چند عدد نمونه
print("\n\nتحلیل طول کدها\n")
print("-" * 60)
print(f"{'عدد':<10}>: {'10'} | {'طول کد گاما':<10}>: {'15'} | {'2*floor(log2(n))+1':<15}>: {'10'}}")
print("-" * 60)

for n in [1, 2, 3, 4, 9, 13, 24, 511, 1025]:
    encoded = gamma_encode(n)
    theoretical_length = 2 * (len(bin(n)) - 3) + 1
    print(f"{n:<10} | {len(encoded):<15} | {theoretical_length:<15}")
```

تست کدگذاری و رمزگشایی

عدد				کد گاما				طول کد				رمزگشایی			
1				1				0				1			
2				3				100				2			
3				3				101				3			
4				5				11000				4			
9				7				1110001				9			
13				7				1110101				13			
24				9				111101000				24			
511				17				1111111101111111				511			
1025				21				11111111110000000001				1025			
5				5				11001				5			
214577				35				11111111111111111010100011000110001				214577			

تست رمزگشایی جریان بیت‌ها

...جریان بیت‌ها: 010010111100011100011110101111101000111111110111111
اعداد رمزگشایی شده: [1, 2, 3, 4, 9, 13, 24, 511, 1025, 5, 214577]

تحلیل طول کدها

عدد*floor(log2(n))+1			طول کد گاما			2		
1			1			1		
2			3			3		
3			3			3		
4			5			5		
9			7			7		
13			7			7		
24			9			9		
511			17			17		
1025			21			21		

جمع‌بندی فصل ۵: فشرده‌سازی ایندکس

در این فصل:

- دلایل فنی و عملی فشرده‌سازی ایندکس – از کاهش فضای دیسک تا افزایش سرعت جست‌وجو – را بررسی کردیم.
- ویژگی‌های آماری واژه‌ها را با قانون‌های Heaps و Zipf مدل‌سازی کردیم تا رفتار واژگان در مجموعه‌های بزرگ را تحلیل کنیم.
- روش‌های فشرده‌سازی دیکشنری را معرفی کردیم: ذخیره‌سازی رشته‌ای، بلوکی (blocking)، و کدگذاری پیشوندی (front coding).
- دو الگوریتم اصلی فشرده‌سازی لیست‌های ارسال را پیاده‌سازی کردیم: کدگذاری بایتی متغیر (Variable Byte) و کدگذاری گاما (γ).
- مقایسهٔ دقیق بین نرخ فشرده‌سازی، سرعت رمزگشایی، و پیچیدگی پیاده‌سازی این الگوریتم‌ها را انجام دادیم.

این فصل پایه‌های **مهندسی فشرده‌سازی داده‌ها در سیستم‌های بازیابی اطلاعات** را فراهم می‌کند. در فصل‌های بعدی، با مفاهیمی مانند:

- مدل‌های رتبه‌بندی و وزن‌دهی واژه‌ها (فصل ۶)
- ارزیابی کیفیت نتایج جست‌وجو (فصل ۸)
- جست‌وجوی وب و ایندکس‌سازی در مقیاس جهانی (فصل ۲۰)

آشنا خواهیم شد – که همگی بر همین زیرساخت فشرده‌سازی دقیق و قابل توسعه بنا شده‌اند.

End of Chapter Exercises

مسئله 5.1: رشد تعداد واژه‌های متمایز با افزایش حجم متن

آیا با افزایش تعداد کل واژه‌های یک مجموعه متنی، تعداد واژه‌های متمایز نیز افزایش پیدا می‌کند؟ آیا این افزایش به صورت خطی است؟ با استفاده از رابطه **Heaps' Law** توضیح دهید.

مسئله 5.2: توزیع فراوانی واژه‌ها در مجموعه‌های متنی

در یک مجموعه متنی بزرگ، معمولاً مشاهده می‌شود که تعداد کمی از واژه‌ها بسیار پرتکرار هستند و در مقابل تعداد زیادی از واژه‌ها فراوانی بسیار کمی دارند. این پدیده یکی از ویژگی‌های مهم در تحلیل مجموعه‌های متنی و طراحی سامانه‌های بازیابی اطلاعات محسوب می‌شود.

توضیح دهید این ویژگی چه تأثیری می‌تواند بر موارد زیر داشته باشد:

(الف) طراحی واژه‌نامه (**Dictionary**)

(ب) اندازه نمایه (**Index**)

(ج) سرعت و کارایی جست‌وجو در سیستم‌های بازیابی اطلاعات

مسئله 5.3: تحلیل توزیع واژگان

در اکثر مجموعه‌های متنی توزیع فراوانی کلمات از مدل توزیع نرمال پیروی نمی‌کند

با تکیه بر مفاهیم آماری دلیل این پدیده را توضیح داده و نقش کلمات با فراوانی یک (Hapax Legomena) را در شکل‌گیری «دم بلند» (Long Tail) نمودار توزیع توصیف کنید

مسئله 5.4: تحلیل مفهومی قانون هیپ و رشد واژه‌نامه

در سیستم‌های بازیابی اطلاعات، قانون هیپ برای تخمین چه پارامتری استفاده می‌شود؟

پارامترهای **k** و **b** در فرمول $M = k \times T^b$ به چه عواملی در دنیای واقعی بستگی دارند و چرا مقدار **b** همواره کمتر از 1 است؟

مسئله 5.5: تخمین اندازه واژه‌نامه با استفاده از قانون هیپ

یک موتور جستجو در حال پردازش یک مجموعه دادگان جدید است. پس از پردازش 1000000 توکن اولیه، واژه‌نامه دارای 50000 کلمه یکتا است.

با فرض اینکه قانون هیپ با ثابت $b = 0.5$ برای این زبان برقرار باشد، ثابت **k** را محاسبه کرده و تخمین بزنید پس از پردازش 4000000 توکن، واژه‌نامه چند کلمه یکتای جدید را به سیستم اضافه خواهد کرد؟

مسئله 5.6: توزیع فرکانس کلمات بر اساس قانون زیپف

طبق قانون زیپف، فرکانس تکرار **i** -امین کلمه پرتکرار در یک مجموعه دادگان با کسر $1/i$ متناسب است.

اگر پرتکرارترین کلمه در یک متن 1000000 بار ظاهر شده باشد، مجموع دفعات تکرار کلماتی که در رتبه‌های 2 تا 5 قرار دارند چقدر است؟

مسئله 5.7: تحلیل ناهنجاری در رشد واژه‌نامه

شما در حال بررسی لاگ‌های یک موتور جستجو هستید. در گزارش ماه اول، تعداد توکن‌های پردازش‌شده برابر با 100 میلیون و اندازه واژه‌نامه برابر با 400000 کلمه بوده است.

در ماه دوم، با اضافه شدن اسناد جدید، تعداد کل توکن‌ها به 120 میلیون می‌رسد، اما اندازه واژه‌نامه در سیستم 380000 گزارش می‌شود.

با استفاده از ساختار قانون هیپ، این پدیده را با ریاضی تحلیل کنید. آیا سیستم دچار خطای محاسباتی شده است یا توجیه منطقی در حوزه پردازش متن برای آن وجود دارد؟

مسئله 5.8: شناسایی و فیلتر کلمات توقف بر پایه قانون زیپف

قانون زیپف نشان می‌دهد که بخش عظیمی از توکن‌های یک متن به تعداد بسیار کمی از کلمات (**Stop words**) اختصاص دارد. ساختار منطقی الگوریتمی را توصیف کنید که با دریافت یک دیکشنری از فرکانس کلمات، کلماتی که مجموع تکرار آن‌ها درصد مشخصی (مثلاً **10 درصد**) از کل توکن‌ها را تشکیل می‌دهند را به عنوان کلمات توقف شناسایی و فیلتر کند.

مسئله 5.9: محاسبه حافظه در ساختار آرایه با طول ثابت

یک واژه‌نامه شامل **200000** کلمه است. میانگین طول حروف هر کلمه **6** بایت است. اگر بخواهیم این واژه‌نامه را با روش **Fixed-Length Array** ذخیره کنیم و طول هر بلوک کلمه را **15** بایت در نظر بگیریم، مجموعاً چند **MB** حافظه برای ذخیره کلمات مصرف می‌شود؟

مسئله 5.10: فشردن سازی واژه‌نامه به روش رشته پیوسته

واژه‌نامه بررسی‌شده در مسئله پیشین را این بار با روش **Dictionary as a String** فشرده می‌کنیم. در این روش، علاوه بر خود کاراکترها، برای هر کلمه یک اشاره‌گر **3** بایتی نیاز داریم. میزان صرفه‌جویی در حافظه را نسبت به روش **Fixed-Length Array** بر حسب **MB** محاسبه کنید.

مسئله 5.11: معماری حافظه در ذخیره‌سازی بلوکی

در تکنیک ذخیره‌سازی بلوکی با اندازه بلوک **k = 4** ، ساختار فیزیکی ذخیره‌سازی را برای کلمات **[system, systematic, systems, systemic]** ترسیم کرده و نشان دهید چگونه طول کلمات در ابتدای هر بخش از رشته یکپارچه ذخیره می‌شود.

مسئله 5.12: بررسی کارایی ذخیره‌سازی بلوکی

فرض کنید در حال بهینه‌سازی واژه‌نامه یک موتور جستجوی تخصصی برای رشته‌های ژنتیکی هستید که کلمات آن تنها از توالی‌های بسیار کوتاه **2** الی **3** کاراکتری تشکیل شده است. تیم مهندسی پیشنهاد می‌دهد برای کاهش حجم دیکشنری، از روش **Blocked Storage** با اندازه بلوک **8** استفاده شود. با فرض اینکه هر اشاره‌گر **4** بایت و هر کاراکتر **1** بایت فضا اشغال می‌کند، ثابت کنید که این پیشنهاد برای این دامنه خاص از داده‌ها، منجر به اتلاف فضا خواهد شد.

مسئله 5.13: جستجوی دودویی در واژه‌نامه فاقد مرز

در روش ذخیره‌سازی دیکشنری به عنوان یک رشته (**Dictionary as a String**)، کلمات در حافظه به صورت متوالی و بدون جداکننده صریح ذخیره می‌شوند. تشریح کنید که سیستم در زمان دریافت یک پرسمان، چگونه عملیات **Binary Search** را روی این ساختار یکپارچه و فاقد مرزهای مشخص پیاده‌سازی می‌کند.

مسئله 5.14: رمزگذاری فواصل با استفاده از کدهای بایت متغیر

لیست پستینگ با شناسه‌های [120, 135, 263, 10503] به یک کلمه خاص اختصاص دارد. ابتدا لیست فواصل را برای این آرایه استخراج کنید. سپس این فواصل را با استفاده از Variable-Byte Codes به معادل باینری آن‌ها تبدیل کنید (بیت پر ارزش هر بایت را به عنوان Continuation Bit در نظر بگیرید).

مسئله 5.15: مهندسی معکوس و خطایابی جریان بیتی کدهای گاما

شما در حال بررسی و خطایابی یک فایل پستینگ هستید که با Gamma Codes فشرده شده است. جریان بیتی استخراج‌شده از حافظه به این صورت است 111111100101 ; تلاش کنید این جریان را رمزگشایی کرده و شناسه‌های اسناد را استخراج کنید. در طول فرآیند رمزگشایی چه بن‌بست منطقی رخ می‌دهد و نقص ساختاری کدهای پیشوندی (Prefix-free) در این جریان بیتی کجاست؟

مسئله 5.16: نسبت فراوانی واژه‌ها بر اساس قانون زیف

فرض کنید فراوانی واژه‌ای که در رتبه 1 قرار دارد برابر با F باشد. طبق قانون زیف، فراوانی واژه‌ای که در رتبه 10 قرار دارد تقریباً چه نسبتی با F خواهد داشت؟

مسئله 5.17: تأثیر فشرده‌سازی واژه‌نامه

فشرده‌سازی واژه‌نامه چه تأثیری بر موارد زیر دارد:

(الف) مصرف حافظه

(ب) سرعت دسترسی به واژه‌ها

(ج) مقیاس‌پذیری سیستم‌های بازیابی اطلاعات

مسئله 5.18: ارزیابی تکنیک‌های فشرده‌سازی در سطح سخت‌افزار

دو تکنیک Variable-Byte Codes و Gamma Codes را از دو منظر زیر ارزیابی و مقایسه کنید:

(الف) نرخ فشرده‌سازی برای زمانی که فواصل بین اسناد بسیار بزرگ باشند.

(ب) سرعت پردازش و عملیات خروج از حالت فشرده (Decompression) در معماری پردازنده‌های مدرن 32 یا 64 بیتی.

مسئله 5.19: تحلیل بدترین حالت حافظه در ذخیره‌سازی نمایه‌ها

یک سیستم در حال ذخیره‌سازی لیست پستینگ برای کلمه‌ای است که در M سند از کل N سند مجموعه وجود دارد.

یکی از طراحان سیستم ادعا می‌کند:

«در بدترین حالت ممکن، بزرگترین فاصله بین دو سند مجاور می‌تواند معادل N باشد. بنابراین کد گامای این تک فاصله به فضای $2 * \log_2(N) + 1$ نیاز دارد. پس بدترین حالت حافظه برای کل لیست پستینگ زمانی رخ می‌دهد که حجم آن به $M * (2 * \log_2(N) + 1)$ برسد.»

با استفاده از اصول ریاضیات گسسته و تعریف مجموع فواصل در یک آرایه، اشتباه منطقی این ادعا را اثبات کرده و کران بالای واقعی برای حجم این لیست پستینگ را به دست آورید.

مسئله 5.20: مقایسه پارامترهای رشد در مدل هیپس

قانون هیپس نرخ رشد دایره واژگان (M) را بر اساس تعداد کل توکن‌ها (T) با فرمول $M = k * T^{\text{beta}}$ پیش‌بینی می‌کند. اگر در دو پیکره متنی متفاوت در اولی مقدار ثابت k بزرگتر و در دومی توان beta بزرگتر باشد، تحلیل کنید که با افزایش حجم اسناد کدام مجموعه در نهایت دیکشنری بزرگتری خواهد داشت؟

مسئله 5.21: محاسبات تخمینی ابعاد دیکشنری

فرض کنید یک سیستم بازیابی اطلاعات مجموعه‌ای شامل 100 میلیون توکن ($T = 10^8$) را پردازش کرده است با در نظر گرفتن مقادیر میانگین $k = 50$ و $\text{beta} = 0.5$

الف) تعداد تقریبی کلمات یکتای این مجموعه را محاسبه نمایید

ب) چنانچه برای ذخیره‌سازی هر واژه یک شناسه عددی 4 بایتی تخصیص یابد حداقل فضای مورد نیاز برای نگهداری شناسه‌های واژگان چند مگابایت خواهد بود؟

مسئله 5.22: روابط معکوس در قانون زیپف

طبق قانون زیپف حاصل ضرب رتبه یک واژه در فرکانس تکرار آن مقداری تقریباً ثابت است اگر واژه‌ای با رتبه 10 دارای فرکانس 5000 باشد

الف) فرکانس مورد انتظار برای واژه‌ای با رتبه 50 را به دست آورید

ب) واژه‌ای که فرکانس آن تنها 50 گزارش شده است حدوداً در چه رتبه‌ای از لیست واژگان قرار می‌گیرد؟

مسئله 5.23: تعیین ثابت تناسب سی

در مجموعه‌ای با 1000 واژه یکتا و مجموعاً 100 هزار توکن مقدار ثابت سی (c) را در مدل زیپف محاسبه کنید

برای محاسبه مجموع معکوس رتبه‌ها از تقریب سری هارمونیک استفاده کنید $\text{Sum}(1/i) \approx \ln(n) + 0.577$

مسئله 5.24: ارزیابی ساختارهای ذخیره‌سازی

مزایا و معایب مدل «آرایه‌ای با طول ثابت» را در مقایسه با مدل «رشته متصل» (`Dictionary as a string`) از منظر هدررفت حافظه (`Internal Fragmentation`) بررسی کنید

مسئله 5.25: ذخیره‌سازی بلوکی واژه‌های واژه‌نامه

فرض کنید واژه‌های واژه‌نامه در بلوک‌هایی شامل 4 واژه ذخیره می‌شوند. توضیح دهید این روش چه مزیتی نسبت به ذخیره‌سازی کاملاً مستقل هر واژه دارد و چه تأثیری بر فرآیند جست‌وجوی واژه خواهد داشت.

مسئله 5.26: ایده اصلی کدهای گاما

ایده اصلی **γ-codes** برای نمایش اعداد چیست؟ مراحل کلی کدگذاری یک عدد را توضیح دهید.

مسئله 5.27: محاسبه ظرفیت حافظه در مدل رشته‌ای

یک دیکشنری شامل 50 هزار واژه با میانگین طول 8 کاراکتر است

اگر برای هر ورودی یک اشاره‌گر 4 بیتی به رشته اصلی و یک فیلد 4 بیتی برای فرکانس در نظر گرفته شود مجموع بایت‌های مصرفی را محاسبه کنید

مسئله 5.28: بهینه‌سازی فضا با تکنیک بلوک‌بندی

در مسئله 5.21 چنانچه از استراتژی بلوک‌بندی با گام $k = 4$ استفاده شود (ذخیره تنها یک اشاره‌گر برای هر 4 واژه) میزان صرفه‌جویی نهایی در حافظه را به درصد بیان کنید و به‌دست آورید

مسئله 5.29: کدگذاری پیشوند (Front Coding)

توالی واژگان زیر که به صورت الفبایی مرتب شده‌اند را با استفاده از روش **Front Coding** فشرده کنید

automation ، automate ، automatic ، automata

فرمت خروجی را به صورت «طول پیشوند مشترک - باقی‌مانده کلمه» نمایش دهید

مسئله 5.30: تحلیل موازنه کارایی (Trade off)

تأثیر افزایش اندازه بلوک (k) را بر «نرخ فشرده‌سازی» و «زمان پاسخ‌گویی سیستم» (**Latency**) تحلیل نمایید
در چه شرایطی طراح سیستم ترجیح می‌دهد مقدار k را کوچک انتخاب کند؟

مسئله 5.31: مفهوم ذخیره‌سازی تفاضلی

چرا در لیست‌های ارجاع به جای ذخیره شناسه مستقیم اسناد (**DocID**) از مفهوم شکاف (**Gap**) استفاده می‌شود؟
این تکنیک چگونه پیش‌زمینه را برای استفاده از کدهای طول متغیر فراهم می‌کند؟

مسئله 5.32: کدگذاری بایت متغیر (VB)

اعداد صحیح 82 ، 215 و 12450 را که نشان‌دهنده شکاف‌های یک لیست ارجاع هستند به فرمت **Variable Byte** تبدیل کنید

وضعیت (**Continuation Bit**) را در بایت‌های خروجی مشخص کنید

مسئله 5.33: مکانیسم کدگذاری گاما

عدد 53 را با استفاده از **Gamma Code** کدگذاری کنید

سپس تفکیک عدد به دو بخش «طول» (در قالب Unary Code) و «آفست» (در قالب Binary) را به دقت نمایش دهید

مسئله 5.34: مقایسه تطبیقی VB و Gamma

کدگذاری گاما بر مبنای «بیت» و کدگذاری VB بر مبنای «بایت» طراحی شده است

تفاوت این دو رویکرد را از نظر «تراکم فشرده‌سازی در اعداد کوچک» و «سرعت کدگذاری در CPUهای مدرن» مقایسه کنید

مسئله 5.35: طراحی بهینه برای لیست‌های ارجاع

برای یک لیست شامل اسناد به شماره‌های 100 ، 102 ، 110 ، 310 و 315

الف) لیست شکاف‌ها (Gaps) را استخراج کنید

ب) با توجه به میانگین اندازه شکاف‌ها، استدلال کنید کدام روش (VB یا Gamma) برای این لیست خاص کارآمدتر است؟

مسئله 5.36: تحلیل الگوریتم جست‌وجو در بلوک ★

در دیکشنری‌های بلوک‌بندی شده (با فرض $k = 8$) چرا امکان استفاده مستقیم از جست‌وجوی دودویی برای تمام کلمات وجود ندارد؟

مراحل جست‌وجوی یک واژه را از لایه ایندکس تا پیمایش خطی داخل بلوک شرح دهید

مسئله 5.37: بازدهی روش بایت متغیر ★

تحلیل کنید در چه صورتی استفاده از کدگذاری VB نسبت به ذخیره‌سازی 32 بیتی استاندارد منجر به صرفه‌جویی 50 درصدی در فضا می‌شود؟

نقش اندازه شکاف‌ها را در این تحلیل تعیین کنید

مسئله 5.38: اثر پیش‌پردازش بر نمودارهای توانی ★

حذف کلمات توقف (Stop words) یکی از مراحل اصلی پیش‌پردازش است

تحلیل کنید که حذف 100 کلمه پرتکرار اول چه تأثیری بر شیب و شکل نمودار زیپف در مختصات «دو-لگاریتمی» Log-Log خواهد داشت

مسئله 5.39: نقض فرض یکنواختی فاصله‌ها در فشرده‌سازی γ ★

در هنگام تخمین فضای مورد نیاز برای یک نمایه که با روش γ-encoding فشرده‌سازی شده است، گاهی فرض می‌شود که فاصله‌های بین شناسه‌های اسناد (docID gaps) تقریباً اندازه‌ای یکسان دارند. چند نمونه از واژه‌هایی (terms) را ذکر کنید که این فرض را نقض می‌کنند. همچنین توضیح دهید ویژگی‌های کلی این واژه‌ها چیست و چرا توزیع فاصله‌های آن‌ها یکنواخت نیست.

مسئله 5.40: بهینگی کد یکانی برای یک توزیع خاص ★

کد **Unary Code** از دیدگاهی که پیش‌تر تعریف شد، یک **Universal Code** محسوب نمی‌شود. با این حال، توزیعی بر روی فاصله‌ها (**Gaps**) وجود دارد که کد یکانی برای آن توزیع، بهینه است. این توزیع کدام است؟

مسئله 5.41: تحلیل معماری نمایه‌سازی و فشرده‌سازی در موتور جست‌وجوی «آرکو» ★

فرض کنید موتور جست‌وجوی «آرکو» برای نمایه‌سازی مجموعه‌ای بسیار بزرگ از لاگ‌های تولیدشده توسط سامانه‌های هوشمند طراحی شده است. این لاگ‌ها، برخلاف متون طبیعی، با یک زبان ماشینی استاندارد و محدود تولید می‌شوند؛ بنابراین تعداد کل واژه‌های ممکن در این زبان ثابت و از پیش مشخص است. با این حال، تعداد کل توکن‌های تولیدشده در این مجموعه با سرعت زیادی در حال افزایش است و پیش‌بینی می‌شود در سال آینده چندین برابر شود.

همچنین تحلیل داده‌ها نشان می‌دهد که توزیع فراوانی واژه‌ها در این مجموعه از قانون **Zipf** معمولی پیروی نمی‌کند، بلکه اگر رتبه واژه با **i** نشان داده شود، فراوانی آن متناسب با $i^3 / 1$ است. در نتیجه، تنها چند واژه نخست سهم بسیار بزرگی از کل متن را تشکیل می‌دهند و بخش عمده‌ای از واژه‌ها بسیار نادر هستند.

(الف)

تیم مهندسی پیشنهاد کرده است که به دلیل رشد شدید حجم داده‌ها، ساختار واژه‌نامه باید به‌طور جدی فشرده شود و به همین علت، استفاده از **Dictionary as a String** کنار گذاشته شده و به‌جای آن از **Blocked Storage** با بلوک‌های بزرگ استفاده شود. با تکیه بر قانون هیپس و رابطه زیر،

$$M = k \times T^b$$

تحلیل کنید که در این سناریو، رشد واقعی **M** نسبت به **T** چگونه است. سپس توضیح دهید که در بلندمدت، نسبت اندازه واژه‌نامه به اندازه **postings file** چه تغییری می‌کند و آیا تمرکز اصلی بر فشرده‌سازی واژه‌نامه، از دیدگاه معماری، تصمیم درستی است یا نه.

(ب)

برای فشرده‌سازی **postings file** دو پیشنهاد مطرح شده است: استفاده یکنواخت از **Variable Byte Coding** برای همه واژه‌ها، یا استفاده یکنواخت از **Gamma Coding** برای همه واژه‌ها. با توجه به توزیع فراوانی واژه‌ها که متناسب با $i^3 / 1$ است، نشان دهید چرا هر دو تصمیم، اگر به‌صورت یکسان برای همه واژه‌ها اعمال شوند، می‌توانند نادرست باشند.

تحلیل خود را بر پایه رفتار **gap**‌ها برای واژه‌های بسیار پرتکرار و نیز واژه‌های بسیار نادر ارائه دهید و در پایان، یک راه‌حل ترکیبی پیشنهاد کنید که مشخص کند برای چه دسته‌ای از واژه‌ها کدام روش مناسب‌تر است و این انتخاب بر چه مبنایی انجام می‌شود.

سوال 5.42: تحلیل رشد واژگان، توزیع Zipf و فشرده‌سازی ایندکس ★

یک شرکت تولیدکننده‌ی قطعات الکترونیکی، برای دفترچه‌های راهنمای تعمیر، گزارش‌های خطا و پرسش‌های متداول تکنسین‌ها، یک موتور جست‌وجوی آفلاین طراحی کرده است. این موتور جست‌وجو باید بر روی تبلت‌هایی با توان پردازشی محدود و حافظه‌ی کم اجرا شود؛ از این‌رو، هم اندازه‌ی ایندکس و هم سرعت پاسخ‌گویی به پرس‌وجو از دیدگاه طراحی سیستم اهمیت اساسی دارند. واژه‌های این سامانه را می‌توان به دو دسته‌ی اصلی تقسیم کرد.

دسته‌ی نخست، **term**‌های عمومی و پرتکرار هستند؛ مانند:

error ، **charge** ، **battery** ، **reset** ، **screen**

دسته‌ی دوم، term‌های تخصصی، شناسه‌مانند و کم‌تکرار هستند؛ مانند:

err9007 ، qz11pro ، bat4x ، xr17

مدیر فنی شرکت ادعای زیر را مطرح می‌کند:

«از آنجا که توزیع فراوانی term‌های این سامانه تقریباً از قانون **Zipf** پیروی می‌کند، رشد دیکشنری نیز ذاتاً قابل کنترل است. بنابراین کافی است دیکشنری به‌صورت فشرده ذخیره شود و برای کل فایل **postings** نیز تنها یک روش فشرده‌سازی یکنواخت در نظر گرفته شود.»

مطلوب است این ادعا را از منظر نظریه‌ی بازیابی اطلاعات و نیز از دیدگاه طراحی عملی سیستم تحلیل و ارزیابی کنید.

اطلاعات آماری corpus

در پایان ماه **2**، وضعیت corpus به‌صورت زیر گزارش شده است:

تعداد کل token‌ها برابر است با:

$$N_1 = 1,200,000$$

تعداد term‌های متمایز دیکشنری برابر است با:

$$M_1 = 41,000$$

در پایان ماه **4**، وضعیت corpus به‌صورت زیر درآمده است:

تعداد کل token‌ها برابر است با:

$$N_2 = 4,800,000$$

تعداد term‌های متمایز دیکشنری برابر است با:

$$M_2 = 67,000$$

در این سؤال، برای مدل‌سازی رشد اندازه‌ی واژگان از قانون **Heaps** استفاده می‌شود. فرم این قانون به‌صورت زیر است:

$$M = kN^b$$

که در آن:

- **M** تعداد term‌های متمایز در دیکشنری است،

- **N** تعداد کل token‌ها در corpus است،

- **k** یک ثابت مثبت است،

- و **b** پارامتری است که معمولاً در بازه‌ی **0 < 1 < b** قرار می‌گیرد.

برای دو مقطع زمانی داده‌شده، رابطه‌ی زیر برقرار است:

$$M_2 / M_1 = (N_2 / N_1)^b$$

اطلاعات مربوط به توزیع فراوانی term‌ها

در پایان ماه **4**، فراوانی نسبی چند term پرتکرار مطابق داده‌های زیر گزارش شده است:

- term با رتبه‌ی **1** دارای فراوانی نسبی **0.080** است.

- term با رتبه‌ی **2** دارای فراوانی نسبی **0.040** است.

- term با رتبه‌ی **4** دارای فراوانی نسبی **0.020** است.

- term با رتبه‌ی **8** دارای فراوانی نسبی **0.010** است.

برای مدل‌سازی توزیع فراوانی termها، از فرم ساده‌شده‌ی قانون **Zipf** استفاده می‌شود:

$$f_r \propto 1 / r$$

که در آن:

- **r** رتبه‌ی term برحسب فراوانی است،

- و **f_r** فراوانی نسبی term با رتبه‌ی **r** است.

افزون بر این، گزارشی دقیق‌تر درباره‌ی termهای جدیدی که بین ماه **2** و ماه **4** وارد دیکشنری شده‌اند، در اختیار شما قرار گرفته است:

- **64%** از termهای جدید تنها یک بار در کل corpus مشاهده شده‌اند.

- از میان همین termهای تک‌رخداد، **72%** شبیه شناسه‌ی محصول، کد قطعه، مدل دستگاه یا کد خطا هستند.

- این termها در queryها به‌ندرت ظاهر می‌شوند، اما حذف آن‌ها از دیکشنری موجب می‌شود بخشی از queryهای تخصصی دیگر نتوانند پاسخ صحیح را بازایی کنند.

اطلاعات مربوط به دیکشنری

بخشی از دیکشنری مرتب‌شده‌ی سامانه به‌صورت زیر است:

bat4x
battery
batterycase
batteryerror
charge
charger
charging
checksum
err9007
error
reset
screen
xr17

در این سؤال، برای فشرده‌سازی دیکشنری فقط مجازید از دو ایده‌ی زیر استفاده کنید:

1. Dictionary as a string

در این روش، termها به‌جای آنکه هر یک به‌صورت یک شیء یا ساختار مستقل ذخیره شوند، پشت‌سرهم و در قالب یک رشته‌ی پیوسته نگه‌داری می‌شوند.

2. Blocked storage

در این روش، termها در قالب بلوک‌های چندتایی سازمان‌دهی می‌شوند تا تعداد pointerها و سایر اطلاعات سربار مربوط به دسترسی مستقیم کاهش یابد. روشن است که کاهش تعداد pointerها می‌تواند اندازه‌ی دیکشنری را کم کند؛ با این حال، جست‌وجوی یک term درون یک بلوک بزرگ‌تر ممکن است هزینه‌ی بیشتری ایجاد کند.

اطلاعات مربوط به postings file

برای دو term نمونه، posting listها به‌صورت زیر داده شده‌اند. این اعداد، docIDهای مرتب‌شده هستند.

برای term **charge** :

10 ، 11 ، 12 ، 13 ، 14 ، 300 ، 301 ، 302 ، 9000

برای term **xr17** :

8 ، 620 ، 621 ، 20000

در فایل postings، به‌جای ذخیره‌ی مستقیم docIDها، از نمایش **d-gap** استفاده می‌شود. اگر posting list یک term برابر باشد با:

$d_1, d_2, d_3, \dots, d_n$

آنگاه d-gapها به‌صورت زیر تعریف می‌شوند:

$g_1 = d_1$

و برای هر $i > 1$ خواهیم داشت:

$g_i = d_i - d_{i-1}$

برای فشرده‌سازی d-gapها فقط مجازید از دو روش زیر استفاده کنید:

1. Variable-byte codes

در این روش، هر عدد با یک یا چند بایت نمایش داده می‌شود. در هر بایت، معمولاً 7 بیت برای نمایش مقدار و 1 بیت برای مشخص‌کردن ادامه‌دار بودن یا پایان کد به‌کار می‌رود. طول تقریبی کد **Variable-byte** برای عدد مثبت **x** با رابطه‌ی زیر مدل می‌شود:

$L_{VB}(x) = 8 \times \text{ceil}((\text{floor}(\log_2 x) + 1) / 7)$

2. y-codes / y-codes

در این روش، اعداد کوچک معمولاً فشرده‌تر نمایش داده می‌شوند؛ اما فرایند decode کردن آن‌ها عموماً پرهزینه‌تر از **Variable-byte** است. طول کد **y** برای عدد مثبت **x** به‌صورت زیر در نظر گرفته می‌شود:

$L_y(x) = 2\text{floor}(\log_2 x) + 1$

الگوی queryهای سامانه

بررسی query log نشان می‌دهد که بیشتر queryهای این سامانه از دو term تشکیل شده‌اند و غالباً ساختاری مشابه نمونه‌های زیر دارند:

error xr17

charge bat4x

battery err9007

reset qz11pro

به بیان دیگر، در بیشتر queryها:

- یک term عمومی و پرتکرار وجود دارد،

- و در کنار آن، یک term تخصصی، کم‌تکرار یا شناسه‌مانند نیز ظاهر می‌شود.

سوال

بخش الف

با استفاده از قانون‌های **Heaps** و **Zipf**، ادعای مدیر فنی را تحلیل کنید و نشان دهید کدام بخش از استدلال او قابل دفاع و کدام بخش گمراه‌کننده است.

در پاسخ خود، حتماً به موارد زیر بپردازید:

1. با استفاده از داده‌های ماه **2** و ماه **4**، مقدار تقریبی پارامتر **b** را در رابطه‌ی Heaps به‌دست آورید. برای این منظور از رابطه‌ی زیر استفاده کنید:

$$M_2 / M_1 = (N_2 / N_1)^b$$

سپس توضیح دهید که مقدار به‌دست‌آمده چه برداشتی درباره‌ی آهنگ رشد واژگان ایجاد می‌کند.

2. بررسی کنید که آیا داده‌های مربوط به رتبه‌های **1**، **2**، **4** و **8** با مدل ساده‌ی Zipf سازگار هستند یا خیر. توضیح دهید این سازگاری دقیقاً چه چیزی را نشان می‌دهد.

3. مهم‌ترین دام مفهومی این بخش را تحلیل کنید: اگر بخش **head** توزیع فراوانی termها با Zipf سازگار باشد، آیا می‌توان نتیجه گرفت که بخش **tail** دیکشنری نیز پایدار، قابل پیش‌بینی و کم‌هزینه است؟ پاسخ شما باید روشن کند که چگونه ممکن است به‌طور هم‌زمان دو گزاره‌ی زیر برقرار باشند:

Head of the distribution is Zipf-like

و در عین حال:

Tail of the dictionary is unstable and full of rare new terms

4. توضیح دهید چرا برآورد رشد واژگان با استفاده از قانون Heaps و تنها بر پایه‌ی دو نقطه‌ی زمانی، در این سناریو می‌تواند گمراه‌کننده باشد؛ به‌ویژه هنگامی که بخش بزرگی از termهای جدید از نوع شناسه‌ها و کدهای فنی باشند.

5. در پایان، درباره‌ی گزاره‌ی زیر داوری کنید:

«چون **Zipf** برقرار است، رشد دیکشنری دیگر مشکل مهمی نیست.»

پاسخ شما باید تحلیلی، دقیق و مستدل باشد.

بخش ب

برای همین سامانه، یک طراحی فشرده‌سازی برای دیکشنری و **postings file** پیشنهاد دهید. در طراحی شما فقط مجازید از مفاهیم زیر استفاده کنید:

- **Dictionary as a string**

- **Blocked storage**

- **Variable-byte codes**

- **γ-codes / y-codes**

در پاسخ خود، حتماً به موارد زیر بپردازید:

1. توضیح دهید چرا ذخیره‌سازی دیکشنری به صورت یک رشته‌ی پیوسته می‌تواند نسبت به ذخیره‌سازی term ها به صورت جداگانه، باعث کاهش مصرف حافظه شود.

2. توضیح دهید **blocked storage** چگونه می‌تواند سربار ناشی از pointer ها و اطلاعات جانبی را کاهش دهد، و در عین حال چرا انتخاب اندازه‌ی بلوک یک **trade-off** ایجاد می‌کند. پاسخ شما باید این دو جهت را به صورت شفاف توضیح دهد:

Bigger block $\Rightarrow$ lower memory overhead

اما هم‌زمان:

Bigger block $\Rightarrow$ higher search cost inside the block

3. برای دو **posting list** داده‌شده، ابتدا **d-gap** ها را به طور کامل محاسبه کنید. سپس توضیح دهید که شکل این **d-gap** ها چه اثری بر مناسب بودن **Variable-byte** یا **y-code** دارد.

4. دام اصلی این بخش را تحلیل کنید: آیا انتخاب یک روش فشرده‌سازی یکنواخت برای کل **postings file** تصمیم درستی است؟ یا در این سامانه باید بین term های پرتکرار و term های کم‌تکرار تفاوت قائل شد؟ پاسخ شما باید با الگوی query های سامانه سازگار باشد؛ یعنی با این واقعیت که بیشتر query ها ترکیبی از یک term پرتکرار و یک term نادر هستند.

5. در پایان، یک طراحی نهایی و دفاع‌پذیر ارائه دهید و توضیح دهید که در این سناریو هدف اصلی طراحی باید چه باشد: آیا باید صرفاً اندازه‌ی ایندکس را حداقل کرد، یا باید بین اندازه‌ی ایندکس و **latency** جست‌وجو تعادل برقرار شود؟ پاسخ شما باید نشان دهد چرا یک طراحی کاملاً یکنواخت، هرچند ساده، لزوماً انتخاب بهینه‌ای برای این سامانه نیست.

مسئله 5.43) تحلیل رشد واژگان، توزیع فراوانی و انتخاب طراحی ایندکس ★

یک فروشگاه آنلاین بزرگ در حال توسعه‌ی موتور جست‌وجوی خود است. در وضعیت فعلی، مجموعه‌ی اسناد شامل 40M توکن و حدود 600K **term** متمایز است. شرکت می‌خواهد دو منبع جدید را نیز به سیستم اضافه کند:

1) **داده‌های کاربران** شامل 40M توکن جدید که از نظر زبانی تقریباً مشابه داده‌های فعلی‌اند و رشد واژگان آن‌ها را می‌توان با **Heaps' Law** و $\beta = 0.5$ تقریب زد.

2) **کاتالوگ فنی تأمین‌کنندگان** شامل 10M توکن که در آن شناسه‌ها و کدهای فنی زیادی مانند **AX-19K-PRO** وجود دارد. این منبع دارای تعداد زیادی **term** نادر است و به دلیل نیازهای کسب‌وکار، حذف **stopword** یا نرمال‌سازی تهاجمی روی آن مجاز نیست.

تیم زیرساخت برای طراحی ایندکس دو انتخاب برای **Dictionary** و دو انتخاب برای **Postings** دارد:

- **D1**: ذخیره‌ی مستقل هر واژه با اشاره‌گرهای معمولی

- **D2: dictionary as a string**: همراه با **blocked storage**

- **P1**: فشرده‌سازی **variable-byte coding**

- **P2**: فشرده‌سازی **gamma-code**

الگوی کوئری‌های سیستم نیز چنین است: 60% جست‌وجوها ترکیبی از یک **term** پرتکرار و یک **term** نادرند، 25% فقط شامل چند **term** پرتکرارند و 15% عمدتاً روی **term** های نادر اجرا می‌شوند.

بخش اول) تحلیل رشد و توزیع

مدیر محصول ادعا می‌کند چون رشد واژگان با **Heaps' Law** زیرخطی است و طبق **Zipf** بیشتر فراوانی روی تعداد کمی واژه

متمرکز می‌شود، پس رشد **vocabulary** مشکل اصلی نیست و کافی است فقط برای **postings** واژه‌های پرتکرار بهینه‌سازی

(1) با فرض اینکه رشد منبع اول مشابه داده‌های فعلی باشد، تخمین بزنید پس از رسیدن کل مجموعه به 90M توکن، تعداد نهایی **term** های متمایز تقریباً چقدر خواهد شد.

(2) این ادعا را نقد کنید و توضیح دهید چرا منبع دوم می‌تواند رشد **dictionary** را شدیدتر از چیزی کند که از **Heaps** برای داده‌ی یکنواخت انتظار داریم. همچنین روشن کنید چرا عوامل غالب در رشد **dictionary** با عوامل غالب در رشد **postings** یکسان نیستند.

بخش دوم) انتخاب طراحی نهایی

شما باید یکی از چهار ترکیب **D2+P1** ، **D1+P2** ، **D1+P1** یا **D2+P2** را انتخاب کنید و از انتخاب خود دفاع نمایید.

(1) توضیح دهید چرا انتخاب شما با الگوی کوئری‌های داده‌شده، به‌ویژه ترکیب **term** پرتکرار + نادر، سازگار است.

(2) توضیح دهید چرا «فشرده‌ترین» طراحی لزوماً بهترین انتخاب نیست و چه **trade-off** ی میان فشرده‌سازی، سرعت دسترسی و هزینه‌ی **CPU** وجود دارد.

(3) توضیح دهید چرا عبارت « **tail** اهمیت ندارد» در این مسئله گمراه‌کننده است و طراحی شما چگونه با **term** های نادر کاتالوگ فنی برخورد می‌کند.

(4) یک وضعیت مشخص را بیان کنید که در آن طراحی انتخابی شما ممکن است عملکرد مطلوبی نداشته باشد.

(5) صریحاً بگویید کدام هزینه را آگاهانه قربانی کرده‌اید: فضای **dictionary** ، اندازه‌ی **postings** یا **latency/CPU** .

مسئله 5.44: معمار کلمات (The Interstellar Indexer) ★

سناریوی اصلی و بستر مسئله

شما به عنوان مهندس ارشد نرم‌افزار، وظیفه توسعه سیستم ایندکسر برای یک کاوشگر فضایی را بر عهده دارید. این سیستم تعبیه‌شده (**Embedded System**) دارای محدودیت‌های فوق‌العاده شدیدی در زمینه حافظه و پردازش است. در طراحی این سیستم، شما با دو پدیده علمی در توزیع داده‌ها روبه‌رو هستید:

رشد لغت‌نامه (قانون Heaps): با دریافت لاگ‌های بیشتر، حجم لغت‌نامه (**Vocabulary**) پیوسته افزایش می‌یابد. برای

مقابله با این مشکل، تصمیم دارید لغت‌نامه را به جای آرایه‌ای از کلمات مجزا، به صورت یک رشته متنی پیوسته (

Dictionary as a String) با استفاده از تکنیک ذخیره‌سازی بلوکی (**Blocked Storage**) پیاده‌سازی کنید.

توزیع چوله کلمات (قانون Zipf): توزیع کلمات در لاگ‌ها متوازن نیست. تعداد معدودی از کلمات به شدت پرتکرار هستند و

فاصله حضور آن‌ها در اسناد (**d-gap**) بسیار کوچک است. در مقابل، کلمات نادر فواصل بسیار بزرگی دارند.

برای فشرده‌سازی فایل پستینگ (**Postings File**)، باید یک سیستم هیبریدی پیاده‌سازی کنید:

دقیقاً **T** کلمه اول (پرکاربردترین کلمات) باید با استفاده از کدهای در سطح بیت یا کدهای گاما (**Gamma Codes**) فشرده شوند.

سایر کلمات (کلمات نادرتر) باید با استفاده از کدهای در سطح بایت یا بایت-متغیر (**Variable Byte**) فشرده‌سازی شوند.

بخش الف) معماری هسته و چالش اشاره‌گرها

در این بخش باید کلاسی طراحی کنید که لیستی از جفت‌های (**docID, term**) را دریافت کرده و ساختار ایندکس را شکل دهد.

نکات مهم و دام‌های طراحی:

تداخل معماری حافظه: کدهای گاما منحصراً در سطح بیت (**Bit-aligned**) و کدهای بایت-متغیر در سطح بایت (

Byte-aligned) پردازش می‌شوند.

چالش اصلی شما: شما اجازه ندارید کدهای گاما را با صفرهای اضافی پر کنید (`Padding`)، زیرا تمام مزیت فشرده‌سازی سیستم از بین می‌رود. همچنین اجازه استفاده از متغیرهای اضافی (`Flag`) برای تشخیص نوع فشرده‌سازی کلمات را ندارید. شما باید معماری حافظه و آرایه اشاره‌گرها (`postings_ptrs`) را به گونه‌ای ایزوله و هوشمندانه طراحی کنید که بدون هدررفت حتی یک بیت، سیستم به طور خودکار تشخیص دهد که آفست ذخیره شده برای هر کلمه، یک آفست بیتی است یا یک آفست بایتی.

بخش ب) پیاده‌سازی موتور فشرده‌ساز

در این بخش باید دو مازول برای دریافت فواصل اسناد (`d-gap`) و تبدیل آن‌ها به فرمت‌های فشرده بنویسید.

نکات مهم و دام‌های محاسباتی:

مدیریت کدهای گاما: دقت کنید که اگر یک کلمه در دو سند متوالی تکرار شود، فاصله (`d-gap`) آن برابر با 1 خواهد بود. در فرمول‌های استاندارد، استخراج آفست از عدد 1 منجر به خطای ریاضی و محاسباتی می‌شود. سیستم شما باید این استثنای منطقی را بهینه‌ترین شکل ممکن و تنها با اختصاص یک بیت مدیریت کند.

مدیریت کدهای بایت‌متغیر: عملیات‌های انتقال و شیفت روی فواصل بزرگ باید به دقت انجام شود؛ به طوری که دقیقاً 7 بیت به داده اصلی (`Payload`) و 1 بیت با ارزش (`MSB`) به پرچم ادامه (`Continuation Bit`) اختصاص یابد.

پیچیدگی زمانی: کدهای شما باید برای حجم عظیمی از داده‌ها بهینه‌سازی شده باشند. استفاده از روش‌های ناکارآمد در ساخت رشته‌ها که منجر به خطای (`Time Limit Exceeded (TLE)`) شود، نمره این بخش را از بین خواهد برد.

ساختار ورودی برنامه

برنامه شما باید داده‌ها را به شکل زیر از ورودی استاندارد دریافت کند:

خط اول: شامل سه عدد صحیح است که با فاصله از هم جدا شده‌اند:

عدد **N**: نشان‌دهنده تعداد کل جفت‌های (سند، کلمه) در جریان داده.

عدد **T**: نشان‌دهنده تعداد کلمات پرتکرار که باید با کد گاما فشرده شوند.

عدد **k**: نشان‌دهنده اندازه هر بلوک برای ذخیره‌سازی لغت‌نامه.

خطوط بعدی (N خط): در هر خط، یک جفت `docID term` قرار دارد که با فاصله از هم جدا شده‌اند. (این داده‌ها از قبل بر اساس `docID` مرتب شده‌اند).

ساختار خروجی برنامه

برنامه شما باید دقیقاً 5 خط خروجی به شکل زیر چاپ کند:

خط اول (لغت‌نامه فشرده): رشته کامل و پیوسته دیکشنری. (برای سهولت در تصحیح، طول هر کلمه باید به صورت یک عدد دقیقاً قبل از همان کلمه درج شود، مانند: `4term3the`).

خط دوم (اشاره‌گرهای بلوک): چاپ یک آرایه شامل اعداد صحیح که نشان‌دهنده آدرس‌های شروع هر بلوک در رشته دیکشنری هستند.

خط سوم (اشاره‌گرهای پستینگ): چاپ آرایه `postings_ptrs` که آفست‌های دسترسی به فایل‌های فشرده را نگه می‌دارد.

خط چهارم (خروجی گاما): رشته‌ای متشکل از صفر و یک‌ها، که نشان‌دهنده حافظه اختصاص‌یافته به ناحیه فشرده‌سازی گاما است.

خط پنجم (خروجی بایت‌متغیر): لیستی از اعداد صحیح (بین 0 تا 255) که نشان‌دهنده بایت‌های تولید شده در ناحیه فشرده‌سازی `Variable Byte` است.

مسئله 5.45: مهندسی دیکشنری و پستینگ‌ها ★

در طراحی یک موتور جستجوی توزیع‌شده برای لاگ‌های دستگاه‌های IoT، با چالش‌های اساسی حافظه روبرو هستیم. با توجه به قانون هیپ (Heaps' Law)، تعداد کلمات یکتا (Terms) با نرخ بالایی رشد می‌کنند و طبق قانون زیف (Zipf's Law)، توزیع فراوانی کلمات به گونه‌ای است که کلمات نادر فواصل (Gaps) بسیار بزرگی در لیست‌های پستینگ ایجاد می‌کنند. شما باید دو مازول اصلی فشرده‌سازی را با رعایت دقیق محدودیت‌های زیر پیاده‌سازی کنید.

بخش الف: ساختار دیکشنری فشرده و جستجوی بهینه

هدف، ذخیره‌سازی تمامی کلمات در یک رشته‌ی پیوسته (String) بدون استفاده از ساختارهای داده‌ای اضافی مانند لیست‌ها یا دیکشنری‌های پایتون برای ذخیره کلمات است.

الزام فنی 1 (ساختار داده): دیکشنری باید به صورت یک رشته‌ی کاراکتری طولانی (Dictionary-as-a-String) ذخیره شود. برای پیمایش بین کلمات، تنها مجاز به ذخیره‌ی ایندکس (Pointer) برای اولین کلمه در هر بلاک (به طول k) هستید. **چالش (Ambiguity):** بسیاری از کلمات لاگ با اعداد شروع می‌شوند (مانند 404error). هرگونه استفاده از جداکننده‌های کاراکتری (Delimiter) برای تشخیص طول کلمات ممنوع است. شما باید راهکاری ابداع کنید که بدون مصرف فضای اضافه، طول هر کلمه را در دیکشنری ذخیره کرده و در زمان جستجو بتوانید بدون ابهام آن را استخراج کنید.

الزام عملکردی: با توجه به حجم بالای کلمات، هرگونه استفاده از عملگر += برای اتصال رشته‌ها در حلقه اصلی ممنوع است. الگوریتم ساخت باید با پیچیدگی زمانی $O(N)$ پیاده‌سازی شود.

وظیفه: تابع `build_and_search_dict(sorted_terms, k, query)` را بنویسید که کلمات را در بلاک‌های k تایی بسته‌بندی کرده و عملیات جستجو را با استفاده از جستجوی دودویی بر روی پوینترهای بلاک و جستجوی خطی در بلاک هدف انجام دهد.

بخش ب: فشرده‌سازی هیبریدی پستینگ‌ها (Hybrid Compression)

هدف، انتخاب بهینه‌ترین روش فشرده‌سازی برای هر فاصله (Gap) بین DocID ها، جهت حداقل‌سازی فضای مصرفی است.

چالش (Decodability): برای اینکه سیستم دی‌کُد کننده بتواند در جریانِ بیتیِ فشرده شده تشخیص دهد که هر Gap با چه الگوریتمی (Gamma یا Variable Byte) فشرده شده است، شما ملزم هستید برای هر Gap یک بیت پرچم (Flag Bit) در ابتدای آن اضافه کنید (مثلاً 0 برای VB و 1 برای گاما).

وظیفه ریاضی و الگوریتمی: تابع `hybrid_encode_postings(gaps)` را پیاده‌سازی کنید. این تابع باید برای هر Gap، فرمول دقیق طول بیت‌های مصرفی هر دو روش (شامل بیت پرچم اضافه شده) را محاسبه کرده و روشی را انتخاب کند که کمترین تعداد بیت را اشغال می‌کند.

نکته مهم: شما باید نشان دهید که در چه محدوده‌هایی از مقادیر Gap، الگوریتم VB با وجود بیت پرچم، همچنان کارآمدتر از کد گاما عمل می‌کند.

نکته برای دانشجو:

دقت کنید که پاسخ‌های شما نه تنها باید از نظر منطقی درست باشند، بلکه باید محدودیت‌های حافظه (RAM) و پیچیدگی زمانی (Time Complexity) را در محیط‌های واقعی IR لحاظ کنند. پاسخ‌هایی که از توابع کتابخانه‌ای یا ساختارهای داده‌ای غیرمؤثر استفاده کنند، نمره کامل دریافت نخواهند کرد.

مسئله 5.46: فشرده‌سازی در موتور جستجو ★

توضیحات سیستم:

یک موتور جستجو برای ذخیره واژگان خود از ساختار `Dictionary as a string` به همراه `Blocked storage` با اندازه بلاک `k = 4` استفاده می‌کند. این سیستم با استفاده از قانون هیپ (`Heaps' law`) تعداد کل لغات را تخمین می‌زند. همچنین بر اساس قانون زیپف (`Zipf's law`)، کلمات به دو دسته تقسیم شده‌اند. برای فشرده‌سازی فایل پستینگ (`Postings File`)، سیستم کلمات پرکاربرد را با کدهای گاما (`Gamma codes`) و کلمات کم‌کاربرد را با کدهای بایت متغیر (`Variable Byte codes`) فشرده کرده است.

پرسش‌ها

بخش الف:

با توجه به قانون زیپف، فقط در یک خط توضیح دهید که چرا کلمات پرکاربرد با کدهای گاما فشرده می‌شوند، اما برای کلمات کم‌کاربرد از کدهای بایت متغیر (`VB`) استفاده می‌کنیم؟

بخش ب:

تابعی بنویسید که فرکانس تکرار یک کلمه و یک حد آستانه (`Threshold`) را به عنوان ورودی بگیرد. اگر فرکانس کلمه از حد آستانه بیشتر بود (یعنی جزو کلمات پرکاربرد قانون زیپف بود)، پستینگ‌لیست آن را با استفاده از تابع `decode_gamma` دیکد کند و در غیر این صورت، از تابع `decode_vb` استفاده کند.

مسئله 5.47: طراحی هسته موتور جستجوی تعبیه‌شده (Embedded Search Engine) ★

شما به عنوان مهندس ارشد داده، وظیفه توسعه یک مینی‌موتور جستجو برای یک گجت پوشیدنی (ساعت هوشمند) را بر عهده دارید. به دلیل محدودیت‌های شدید سخت‌افزاری، مدیریت بهینه حافظه `RAM` در بخش ذخیره‌سازی دیکشنری و فایل‌های پستینگ (`Postings`) حیاتی است. یک جریان متنی (`Corpus`) با مشخصات آماری و سیستمی زیر به سیستم شما وارد شده است:

پارامتر سیستم (System Parameter)	مقدار (Value)	توضیحات فنی (Technical Description)
N	10^6	تعداد کل توکن‌ها در پیکره متنی
K, β	$K = 40, \beta = 0.5$	پارامترهای ثابت برای قانون هیپ ($M = K \times N^\beta$)
Avg. Term Length	8 Bytes	میانگین فضای اشغالی توسط هر کلمه
k (Block Size)	4	اندازه بلاک برای فشرده‌سازی دیکشنری
MAX_RAM	400 KB	حداکثر حافظه تخصیص‌یافته برای بخش دیکشنری

بخش اول: مدیریت حافظه دیکشنری بر اساس قوانین آماری

تابع الگوریتمی `build_optimized_dict` را بنویسید که آرایه‌ای از کلمات یکتا و فرکانس آن‌ها را به عنوان ورودی دریافت می‌کند. سیستم شما باید مراحل زیر را پیاده‌سازی کند:

- ارزیابی حافظه :** ابتدا با استفاده از قانون هیپ (`Heaps' law`) تخمین بزنید که آیا دیکشنری با روش `Dictionary-as-a-String` با `Blocked Storage` در `MAX_RAM` مشخص شده جا می‌شود یا خیر. (فرض کنید هر پوینتر 3 بایت و نشانگر طول کلمه 1 بایت فضا نیاز دارد). اگر حافظه کافی نبود، سیستم باید خطای حافظه (`Exception`) پرتاب کند.

2. بهینه‌سازی کش (Cache): در صورت کفایت حافظه، کلمات باید قبل از بلاک‌بندی، بر اساس قانون زیپ (Zipf's law) تحلیل شده و بر اساس فرکانس نزولی مرتب شوند تا کلمات پرتکرار در ابتدای رشته حافظه قرار گیرند.
3. خروجی: یک رشته یکپارچه (String) طولانی و آرایه پوینترها (Pointers).

بخش دوم: فشرده‌سازی هیبریدی Postings

برای ذخیره لیست سندها، تابعی به نام `encode_postings` بنویسید که یک آرایه مرتب از `doc_ids` و رتبه زیپ (Zipf Rank) آن کلمه را دریافت کرده و فشرده‌سازی تطبیقی (Adaptive) زیر را اعمال کند:

1. مدیریت فاصله‌ها (Gaps): الگوریتم باید به درستی `Gap` ها را محاسبه کند (دقت کنید که رفتار اولین شناسه سند با بقیه متفاوت است).

2. فشرده‌سازی دوگانه (تله الگوریتمی):

برای کلمات بسیار پرتکرار (رتبه $r \leq 10$ در قانون زیپ) که فاصله‌های کوچکی دارند، باید در سطح بیت (Bit-level) از کدهای گاما (γ -codes) استفاده کنید. این بیت‌ها باید در بایت‌های 8 بیتی پک (Pack) شوند.

برای سایر کلمات (رتبه $r > 10$)، از `Variable Byte Codes` (در سطح بایت) استفاده کنید.

3. خروجی: یک آرایه یکپارچه از بایت‌های فشرده‌شده.

مسئله 5.48: معماری موتور جستجوی لاگ‌های توزیع‌شده (سناریو محور) ★

سناریو:

یک استارت‌آپ در حوزه اینترنت اشیا (IoT)، در حال توسعه یک موتور جستجوی بلادرنگ برای تحلیل لاگ‌های متنی ارسالی از میلیون‌ها سنسور است. به دلیل محدودیت شدید حافظه `RAM` در سرورهای `Edge`، تیم معماری باید ساختار ایندکس (Inverted Index) را بهینه‌سازی کند. کل توکن‌های پردازش شده تاکنون $N = 10^8$ کلمه است.

در فاز اول تست، مهندسين داده دو گزارش گرافیکی و آماری زیر را از ساختار متون استخراج کرده‌اند:

نمودار شماره 1: رفتار رشد کلمات یکتا (Heaps' Law)

این نمودار، رشد اندازه واژه‌نامه (M) نسبت به رشد کل توکن‌ها (N) را در مقیاس لگاریتمی ($\log_{10}$) نشان می‌دهد.

⚙ (0, 2) ⚙ (4, 8)

0 2 4 6 8

1 2 3 4 5

$\log_{10}(M)$ [Vocabulary Size] $\log_{10}(N)$ [Collection Size]

جدول شماره 1: توزیع فرکانس کلمات بر اساس قانون زیپف (Zipf's Law)

توزیع کلمات از قانون استاندارد زیپف پیروی می‌کند. (فرکانس آامین کلمه پرکاربرد در کل مجموعه).

رتبه زیپف (i)	توکن (Term)	فرکانس در مجموعه (cf _i)
1 🏆	[system_OK]	10,000,000
2 🏆	[auth_fail]	5,000,000
3 🏆	[temp_warn]	3,333,333
...	...	...
🚨 50,000	[kernel_panic_0x...]	*(نیاز به محاسبه)*

بخش اول (الف): معماری واژه‌نامه و دام مقیاس‌پذیری

تیم فنی قصد دارد برای فاز نهایی پروژه، کل واژه‌نامه را با ترکیب روش‌های Dictionary-as-a-String و Blocked Storage (با اندازه بلوک $k = 4$) در حافظه رم فشرده کند. با تحلیل دقیق نمودار شماره 1، ثابت کنید که چرا اجرای این روش فشرده‌سازی برای این مجموعه داده، یک اشتباه محاسباتی فاجعه‌بار است و کل معماری موتور جستجو را با شکست مواجه می‌کند؟

راهنمایی: به پارامترهای پنهان در شیب خط نمودار و ماهیت فیزیکی قانون Heaps دقت کنید. محاسبات ریاضی الزامی است.

Postings بخش دوم (ب): پارادوکس فشرده‌سازی

برای فشرده‌سازی فایل Postings، مهندس ارشد تیم یک قانون قطعی وضع کرده است:

که $i = 50,000$ (مثل رتبه) برای 100 کلمه اول (پرکاربردترین‌ها) از Variable Byte استفاده کنید، اما برای کلمات نادر» استفاده کنید؛ زیرا فشرده‌سازی در γ (γ -codes) فاصله‌های (Gaps) بسیار بزرگی در اسناد دارند، الزاماً و قطعاً از کدهای «سطح بیت (Bit-Level) برای اعداد نجومی، همیشه فضای کمتری نسبت به سطح بایت می‌گیرد

با استفاده از جدول قانون زیپف، میانگین فاصله (Average Gap) را برای کلمه‌ای با رتبه 50,000 محاسبه کنید. سپس با محاسبه دقیق طول بیت برای هر دو روش Variable Byte و γ -code، ثابت کنید که استدلال مهندس ارشد یک دام و تصور غلط است یا یک تصمیم هوشمندانه؟

پاسخنامه مسئله 5.1: رشد تعداد واژه‌های متمایز با افزایش حجم متن

بخش اول: رابطه بین تعداد کل واژه‌ها و واژه‌های متمایز

در یک مجموعه متنی، با افزایش تعداد کل واژه‌ها (Tokens) معمولاً تعداد واژه‌های متمایز یا واژگان یکتا (Distinct Terms) نیز افزایش پیدا می‌کند. دلیل این موضوع آن است که با اضافه شدن اسناد یا جملات جدید به مجموعه، احتمال ظهور واژه‌های جدید نیز وجود دارد.

با این حال، این افزایش همیشه با همان سرعت اولیه ادامه نمی‌کند. در ابتدای شکل‌گیری مجموعه متنی، تعداد زیادی واژه جدید مشاهده می‌شود، اما با بزرگ‌تر شدن متن، بسیاری از واژه‌ها در واقع تکرار واژه‌هایی هستند که قبلاً دیده شده‌اند.

بنابراین رشد تعداد واژه‌های متمایز معمولاً کندتر از رشد تعداد کل واژه‌ها است.

بخش دوم: قانون هیپ (Heaps' Law)

رابطه بین تعداد کل واژه‌ها و تعداد واژه‌های متمایز معمولاً با استفاده از قانون هیپس مدل‌سازی می‌شود. این قانون رابطه زیر را بیان می‌کند:

$$M = k \times T^\beta$$

در این رابطه:

M تعداد واژه‌های متمایز در مجموعه متنی است.

T تعداد کل واژه‌ها در مجموعه اسناد است.

k یک ضریب ثابت است که به زبان و نوع مجموعه داده وابسته است.

β نمای رشد است که مقدار آن معمولاً در بازه $0.4 - 0.6$ قرار دارد.

بخش سوم: تحلیل رفتار رشد

در رابطه قانون هیپس، مقدار β کمتر از 1 است. این موضوع نشان می‌دهد که رشد تعداد واژه‌های متمایز نسبت به رشد تعداد کل واژه‌ها از نوع **Sublinear** یا زیرخطی است.

اگر مقدار β برابر با 1 بود، رابطه به شکل زیر در می‌آمد:

$$M = k \times T$$

که نشان‌دهنده یک رشد خطی است. اما در عمل چون $\beta < 1$ است، تعداد واژه‌های متمایز با سرعتی کمتر از رشد تعداد کل واژه‌ها افزایش پیدا می‌کند.

به بیان ساده، در ابتدای مجموعه متنی واژه‌های جدید زیادی ظاهر می‌شوند، اما با بزرگ‌تر شدن مجموعه اسناد، بیشتر واژه‌ها تکرار واژه‌های قبلی خواهند بود. بنابراین نرخ ورود واژه‌های جدید به تدریج کاهش می‌یابد.

بخش چهارم: تفسیر در سیستم‌های بازیابی اطلاعات

این رفتار در سیستم‌های بازیابی اطلاعات اهمیت زیادی دارد. با افزایش اندازه مجموعه اسناد، اندازه واژه‌نامه **Dictionary** نیز افزایش پیدا می‌کند، اما این افزایش نسبت به رشد تعداد کل واژه‌ها کندتر است.

در نتیجه حتی در مجموعه‌های بسیار بزرگ نیز اندازه واژه‌نامه قابل مدیریت باقی می‌ماند و سیستم‌های بازیابی اطلاعات می‌توانند این ساختار را به شکل کارآمد نگهداری و پردازش کنند.

پاسخ نهایی

با افزایش تعداد کل واژه‌های یک مجموعه متنی، تعداد واژه‌های متمایز نیز افزایش پیدا می‌کند. با این حال، این افزایش از نوع خطی نیست. طبق قانون هیپس:

$$M = k \times T^\beta$$

از آنجا که مقدار β معمولاً کمتر از 1 است، رشد تعداد واژه‌های متمایز نسبت به افزایش تعداد کل واژه‌ها به صورت زیرخطی اتفاق می‌افتد؛ یعنی با بزرگ‌تر شدن مجموعه متنی، سرعت اضافه شدن واژه‌های جدید به تدریج کاهش پیدا می‌کند.

پاسخنامه مسئله 5.2: توزیع فراوانی واژه‌ها در مجموعه‌های متنی

بخش اول: ویژگی توزیع فراوانی واژه‌ها

در مجموعه‌های متنی بزرگ، توزیع فراوانی واژه‌ها معمولاً به صورت یکنواخت نیست. در عمل مشاهده می‌شود که تعداد کمی از واژه‌ها بسیار پرتکرار هستند، در حالی که بخش بزرگی از واژه‌ها تنها تعداد کمی بار در کل مجموعه ظاهر می‌شوند. این پدیده یکی از ویژگی‌های بنیادی داده‌های متنی محسوب می‌شود.

این رفتار با قانونی به نام **Zipf's Law** یا قانون زیپف توضیح داده می‌شود. طبق این قانون، فراوانی یک واژه با رتبه آن در فهرست واژه‌ها رابطه‌ای معکوس دارد؛ به این معنا که اگر واژه‌ها بر اساس فراوانی مرتب شوند، فراوانی واژه با رتبه i تقریباً متناسب با $1/i$ خواهد بود.

این توزیع نامتوازن اثر قابل توجهی بر طراحی ساختارهای داده در سیستم‌های **Information Retrieval** دارد.

الف) تأثیر بر طراحی واژه‌نامه (Dictionary)

در ساختار واژه‌نامه یک سیستم بازیابی اطلاعات، تمام واژه‌های یکتای مجموعه ذخیره می‌شوند. با توجه به توزیع زیپفی، تعداد زیادی از واژه‌ها فراوانی بسیار کمی دارند. در نتیجه واژه‌نامه شامل تعداد زیادی مدخل خواهد بود.

برای مدیریت این حجم بزرگ از واژه‌ها، معمولاً از روش‌های مختلف فشرده‌سازی و بهینه‌سازی استفاده می‌شود. به عنوان نمونه می‌توان به حذف یا نادیده گرفتن **Stop Words** (کلمات بسیار پرتکرار)، و همچنین استفاده از روش‌های فشرده‌سازی رشته مانند **Front Coding** یا **Blocked Storage** اشاره کرد.

در بسیاری از سیستم‌ها تلاش می‌شود واژه‌نامه در حافظه اصلی **RAM** نگهداری شود تا دسترسی به آن سریع‌تر انجام شود.

بنابراین توزیع نامتوازن واژه‌ها باعث می‌شود طراحی واژه‌نامه به سمت استفاده از روش‌های **فشرده‌سازی و مدیریت مؤثر تعداد زیاد واژه‌های کم‌تکرار** حرکت کند.

ب) تأثیر بر اندازه نمایه (Index)

در ساختار **Inverted Index** برای هر واژه یک فهرست از شناسه اسناد مرتبط ذخیره می‌شود که به آن **Postings List** گفته می‌شود.

به دلیل توزیع زیپفی، واژه‌های بسیار پرتکرار دارای فهرست‌های بسیار طولانی هستند، زیرا در تعداد زیادی از اسناد ظاهر می‌شوند. در مقابل، واژه‌های نادر دارای فهرست‌های بسیار کوتاه خواهند بود.

در نتیجه بخش قابل توجهی از فضای نمایه توسط فهرست‌های طولانی مربوط به واژه‌های پرتکرار اشغال می‌شود. برای کاهش اندازه نمایه معمولاً از روش‌های فشرده‌سازی مانند **Gap Encoding**، **Variable Byte Coding** و **Gamma Coding** استفاده

همچنین در برخی سیستم‌ها کلمات بسیار پرتکرار (**Stop Words**) اصلاً در نمایه ذخیره نمی‌شوند تا اندازه نمایه کاهش یابد.

بنابراین توزیع زیپفی باعث ایجاد **لیست‌های بسیار طولانی برای واژه‌های پرتکرار** و **لیست‌های کوتاه برای واژه‌های نادر** می‌شود و نیاز به فشرده‌سازی نمایه را افزایش می‌دهد.

ج) تأثیر بر سرعت و کارایی جست‌وجو

این توزیع نامتوازن همچنین اثر مهمی بر کارایی عملیات جست‌وجو در سیستم‌های بازیابی اطلاعات دارد. اگر یک پرس‌وجو شامل واژه‌های بسیار پرتکرار باشد، فهرست اسناد مرتبط با آن واژه‌ها بسیار بزرگ خواهد بود و پردازش آن‌ها در عملیات‌هایی مانند تقاطع فهرست‌ها زمان بیشتری نیاز دارد.

در مقابل، واژه‌های کم‌تکرار معمولاً فهرست‌های کوتاه‌تری دارند و پردازش آن‌ها سریع‌تر انجام می‌شود.

به همین دلیل بسیاری از سیستم‌های بازیابی اطلاعات ابتدا واژه‌هایی را پردازش می‌کنند که فراوانی کمتری دارند. این کار باعث می‌شود اندازه مجموعه اسناد کاندید در مراحل اولیه کاهش یابد و سرعت پردازش پرس‌وجو افزایش پیدا کند.

همچنین استفاده از حذف کلمات توقف و روش‌های رتبه‌بندی نتایج نیز از جمله راهکارهایی است که برای بهبود کارایی جست‌وجو به کار گرفته می‌شود.

پاسخ نهایی

در مجموعه‌های متنی بزرگ، توزیع فراوانی واژه‌ها معمولاً نامتوازن است و مطابق قانون **Zipf's Law** تعداد کمی از واژه‌ها بسیار پرتکرار و تعداد زیادی از واژه‌ها کم‌تکرار هستند.

این ویژگی پیامدهای مهمی دارد: در طراحی واژه‌نامه نیاز به روش‌های فشرده‌سازی و مدیریت واژه‌های کم‌تکرار ایجاد می‌شود؛ در ساختار نمایه باعث شکل‌گیری فهرست‌های بسیار طولانی برای واژه‌های پرتکرار می‌شود که نیازمند فشرده‌سازی هستند؛ و در فرآیند جست‌وجو نیز واژه‌های پرتکرار می‌توانند هزینه پردازشی بیشتری ایجاد کنند، در حالی که واژه‌های نادر معمولاً جست‌وجوی سریع‌تری دارند.

پاسخنامه مسئله 5.3: تحلیل توزیع واژگان

بخش 1) صورت مسئله و هدف سؤال

در بسیاری از مجموعه‌های متنی (**Corpus**) توزیع فراوانی کلمات (**Word/Term Frequency**) شبیه توزیع نرمال (**Normal/Gaussian**) نیست

هدف سؤال این است که:

1) با تکیه بر مفاهیم آماری توضیح دهیم چرا فراوانی کلمات «نرمال» توزیع نمی‌شود

2) نقش کلماتی که فقط یک‌بار ظاهر می‌شوند (**Hapax Legomena**) را در شکل‌گیری «دم بلند» (**Long Tail**) توزیع بیان کنیم

بخش 2) مفاهیم پایه و واژگان کلیدی (Definitions)

1) **فراوانی واژه** (**Term Frequency/TF**) : تعداد دفعاتی که یک واژه در متن/پیکره تکرار می‌شود

2) **توزیع نرمال (Normal Distribution)** : توزیعی زنگوله‌ای و متقارن حول میانگین (mean) که با دور شدن از میانگین، احتمال خیلی سریع کاهش پیدا می‌کند

3) **دم بلند (Long Tail)** : بخشی از توزیع که در آن تعداد زیادی رخداد/آیتم با فراوانی بسیار کم وجود دارد و کاهش فراوانی کند است؛ به‌طوری که دنباله توزیع طولانی می‌شود

4) **Hapax Legomena** : واژه‌هایی که دقیقاً یک بار در کل پیکره رخ داده‌اند

بخش 3) چرا توزیع فراوانی کلمات نرمال نیست؟ (Statistical Reasoning)

گام 1: نرمال شدن معمولاً از «جمع اثرهای مستقل» می‌آید CLT

یکی از مهم‌ترین دلایل شکل‌گیری توزیع نرمال در بسیاری از پدیده‌ها قضیه حد مرکزی (Central Limit Theorem) است ایده ساده **CLT** این است: اگر یک کمیت نتیجه «جمع» تعداد زیادی عامل تقریباً مستقل و هم‌رفتار باشد، توزیع خروجی به نرمال نزدیک می‌شود

اما فراوانی واژه‌ها معمولاً این شرایط را ندارند

گام 2: رخداد واژه‌ها مستقل (Independent) نیست

در متن واقعی کلمات به‌شدت به موارد زیر وابسته‌اند:

- موضوع متن (Topic)

- سبک نوشتار و نویسنده (Style/Author)

- هم‌رخدادی واژه‌ها و ساختار جمله (Co-occurrence)

مثال: اگر متن درباره «شبکه» باشد، احتمال دیدن کلماتی مثل «پروتکل»، «مسیریابی» و ... افزایش می‌یابد. این یعنی رخدادها مستقل نیستند و یکی از ستون‌های اصلی نرمال شدن فرو می‌ریزد.

گام 3: داده‌ها ناهمگن (Heterogeneous) و مخلوط (Mixture) هستند

پیکره‌ها معمولاً از چندین زیرمجموعه با موضوعات مختلف تشکیل می‌شوند (مثلاً اخبار، متن علمی، شبکه اجتماعی، وبلاگ). در هر زیرمجموعه احتمال استفاده از واژه‌ها فرق می‌کند؛ بنابراین در سطح کل پیکره با «مخلوط توزیع‌ها» روبه‌رو هستیم (Mixture distributions).

مخلوط توزیع‌ها غالباً شکل زنگوله‌ای و متقارن نرمال تولید نمی‌کند.

گام 4: فراوانی یک متغیر شمارشی و کراندار از پایین است

فراوانی نمی‌تواند منفی باشد؛ حداقل آن صفر است.

اما توزیع نرمال روی کل خط حقیقی تعریف می‌شود و از نظر نظری مقادیر منفی هم دارد.

پس از دید مدل‌سازی آماری، از همان ابتدا «نرمال» مدل طبیعی برای شمارش‌ها نیست معمولاً مدل‌های شمارشی مثل

Poisson یا Negative Binomial مطرح می‌شوند، هرچند آن‌ها هم به‌تنهایی heavy-tail بودن متن را کامل توضیح نمی‌دهند.

گام 5: شکل واقعی داده: تعداد کمی بسیار پرتکرار + تعداد بسیار زیاد کم‌تکرار

در زبان طبیعی:

- چند واژه بسیار پرتکرارند (اغلب کلمات دستوری/پرمصرف؛ مثل stopwords)

- اما تعداد بسیار زیادی واژه فقط چند بار یا یک بار دیده می‌شوند

این ساختار باعث نامتقارن شدن شدید توزیع (Skewness بالا) و ایجاد دم سنگین/دم بلند می‌شود؛ چیزی که با ویژگی‌های یک توزیع نرمال سازگار نیست.

بخش 4) نقش Hapax Legomena در ایجاد «دم بلند» (Long Tail)

گام 1: چرا Hapax ها زیاد تولید می‌شوند؟

دلایل اصلی زیاد بودن واژه‌های با فراوانی یک عبارت‌اند از:

- وجود واژه‌های تخصصی، اصطلاحات جدید و نام‌ها (Named Entities)

- شکل‌های صرفی و ترکیبی (Morphology, Compounds)

- اعداد، کدها، URLها و هشتگ‌ها در داده‌های وب

- خطاهای تایپی یا توکن‌سازی (Tokenization errors) در داده‌های خام

گام 2: Hapax ها «تعداد واژه‌های یکتا» (Types) را زیاد می‌کنند

هر Hapax فقط یک Token به کل متن اضافه می‌کند، اما یک Type جدید به واژگان پیکره می‌افزاید.

پس اگر تعداد Hapax ها زیاد باشد، تعداد بسیار زیادی واژه داریم که دقیقاً فراوانی 1 دارند.

گام 3: چگونه این موضوع دم بلند می‌سازد؟

اگر نمودار را این‌طور ببینیم: «برای هر مقدار فراوانی f ، چند واژه وجود دارد که دقیقاً فراوانی f دارند؟»

در داده واقعی معمولاً تعداد واژه‌های با $f = 1$ بسیار زیاد است.

سپس برای $f = 2, 3, 4, \dots$ تعداد واژه‌ها کمتر می‌شود.

اما این کاهش مثل نرمال سریع و متقارن نیست؛ بلکه دنباله‌ای بلند از واژه‌های نادر باقی می‌ماند. بنابراین Hapax ها عملاً

بزرگ‌ترین توده در سمت «کم‌فراوانی» هستند و بخش اصلی دم بلند را شکل می‌دهند.

بخش 5) نتیجه‌گیری نهایی پاسخ سؤال (Final Conclusion)

توزیع فراوانی واژه‌ها در متن‌ها معمولاً نرمال نیست؛ چون شرایط آماری لازم برای نرمال شدن (مانند استقلال، همگنی و ساختار

جمعی ساده) برقرار نیست.

متن‌ها موضوع‌محور و ناهمگن‌اند، رخداد کلمات به‌شدت وابسته است، داده‌ها شمارشی و نامتقارن‌اند، و در عمل تعداد کمی واژه

بسیار پرتکرار و تعداد بسیار زیادی واژه کم‌تکرار داریم (heavy-tail).

Hapax Legomena با ایجاد تعداد بسیار زیاد واژه‌های با فراوانی 1، مهم‌ترین جزء ناحیه کم‌فراوانی هستند و بخش اصلی «دم

بلند» توزیع را می‌سازند

مسئله 5.4: تحلیل مفهومی قانون هیپ و رشد واژه‌نامه

بخش اول: قانون هیپ چه پارامتری را تخمین می‌زند؟

قانون هیپ (Heaps' Law) یک رابطه تجربی در حوزه **Information Retrieval** و زبان‌شناسی آماری است که رشد تعداد

واژه‌های منحصر به فرد را بر حسب اندازه مجموعه متنی توصیف می‌کند. این قانون بیان می‌کند که اگر **T** تعداد کل واژه‌های

مشاهده‌شده در یک مجموعه متنی و **M** تعداد واژه‌های متمایز (واژه‌نامه) باشد، رابطه میان آن‌ها به صورت زیر قابل تقریب

است:

$$M = k \times T^b$$

در این رابطه، **M** بیانگر اندازه واژه‌نامه، **T** نشان‌دهنده حجم کل متن، و پارامترهای **k** و **b** ضرایب تجربی وابسته به ویژگی‌های مجموعه متنی هستند. بنابراین، قانون هیپ ابزاری برای تخمین **حجم واژه‌نامه (Vocabulary Size)** بر اساس اندازه پیکره متنی است.

بخش دوم: عوامل مؤثر بر پارامترهای **k** و **b**

پارامتر **k** ضریب مقیاس رشد واژه‌نامه است. هرچه مقدار آن بزرگ‌تر باشد، برای حجم مشخصی از متن، تعداد واژه‌های متمایز بیشتری خواهیم داشت. این پارامتر تحت تأثیر دامنه موضوعی متن، تنوع منابع، و سبک نگارش قرار دارد. مجموعه‌های تخصصی (مانند متون علمی یا پزشکی) معمولاً دارای **k** بزرگ‌تری هستند، زیرا شامل اصطلاحات فنی متعددی‌اند. همچنین، پیکره‌هایی با نویسندگان متعدد یا سبک‌های متنوع، واژه‌نامه گسترده‌تری تولید می‌کنند.

پارامتر **b** نرخ رشد واژه‌نامه را تعیین می‌کند و معمولاً در بازه **(0.4 – 0.6)** قرار دارد. این پارامتر بیانگر آن است که رشد واژه‌نامه نسبت به رشد حجم متن چگونه کاهش می‌یابد. عوامل مؤثر بر آن شامل توزیع فراوانی واژه‌ها، میزان تکراری بودن محتوا، و ویژگی‌های ساختاری زبان است. برای مثال، متونی با تکرار زیاد واژه‌ها (مانند اخبار مشابه) معمولاً دارای **b** کوچک‌تری هستند.

بخش سوم: چرا مقدار **b < 1** است؟

کمتر بودن مقدار **b** از **1** نشان می‌دهد که رشد واژه‌نامه نسبت به رشد متن، **زیرخطی (Sublinear)** است. این ویژگی ناشی از ساختار آماری زبان طبیعی است. در زبان، تعداد محدودی واژه بسیار پرتکرار هستند، در حالی که تعداد زیادی واژه کم‌تکرار وجود دارند. با افزایش **T**، بخش عمده‌ای از واژه‌های افزوده‌شده تکراری‌اند و تنها بخش کوچکی واژه‌های جدید هستند.

از دید احتمالاتی، هرچه حجم متن افزایش یابد، احتمال مشاهده واژه‌ای کاملاً جدید کاهش پیدا می‌کند. بنابراین، اگر **T** دو برابر شود، مقدار **M** کمتر از دو برابر خواهد شد. به عنوان نمونه، اگر **b ≈ 0.6** باشد:

$$2^{0.6} \approx 1.5$$

این رفتار بازتابی از توزیع‌های قانون توانی (**Power-law**) در زبان طبیعی است که موجب می‌شود رشد واژه‌نامه با آهنگی کاهنده ادامه یابد.

پاسخ نهایی

قانون هیپ (Heaps' Law) رابطه‌ای تجربی برای تخمین تعداد واژه‌های منحصر به فرد (**M**) بر حسب حجم کل متن (**T**) است و به صورت $M = k \times T^b$ بیان می‌شود. پارامتر **k** به دامنه، تنوع و سبک متن وابسته است، در حالی که **b** نرخ زیرخطی رشد واژه‌نامه را مشخص می‌کند و همواره کوچکتر از **1** است. دلیل این امر، توزیع نامتوازن فراوانی واژه‌ها در زبان طبیعی و کاهش احتمال مشاهده واژه‌های جدید با افزایش حجم متن است. در نتیجه، رشد واژه‌نامه همواره کندتر از رشد خود متن خواهد بود.

پاسخنامه مسئله 5.5: تخمین اندازه واژه‌نامه با استفاده از Heaps' Law

بخش 1) صورت مسئله و فرمول پایه

$$M = k \times T^b$$

عبارت ریاضی فوق، فرمول **قانون هیپ (Heaps' Law)** است که در این مسئله برای تخمین اندازه واژه‌نامه یک موتور جستجو استفاده می‌شود. در این سناریو، موتور جستجو در حال پردازش یک مجموعه دادگان جدید است. پس از بررسی 1,000,000 توکن اولیه، اندازه واژه‌نامه به 50,000 کلمه یکتا رسیده است. با فرض اینکه پارامتر رشد واژه‌نامه (b) برای این زبان برابر با 0.5 باشد، هدف این مسئله استخراج ثابت مقیاس (k)، تخمین اندازه واژه‌نامه پس از رسیدن حجم پردازش به 4,000,000 توکن، و در نهایت محاسبه تعداد کلمات یکتای جدیدی است که به سیستم اضافه خواهند شد.

بخش 2) تعریف متغیرهای مسئله

$$\begin{aligned}T &= 1,000,000 \\M &= 50,000 \\b &= 0.5\end{aligned}$$

در این متغیرها، **M** نشان‌دهنده اندازه واژه‌نامه (تعداد کلمات یکتا)، **T** نشان‌دهنده تعداد کل توکن‌های پردازش‌شده، **b** توان رشد واژه‌نامه و متغیر مجهول **k** نشان‌دهنده ثابت مقیاس است. بر اساس داده‌های گام نخست مسئله، موتور جستجو تاکنون 1,000,000 توکن (**T**) را پردازش کرده و توانسته است 50,000 کلمه متمایز (**M**) را با توان رشد 0.5 (**b**) شناسایی و ثبت کند.

بخش 3) محاسبه ثابت k

$$\begin{aligned}M &= k \times T^b \\50,000 &= k \times (1,000,000)^{0.5} \\50,000 &= k \times \sqrt{1,000,000} \\50,000 &= k \times 1,000 \\k &= \frac{50,000}{1,000} = 50\end{aligned}$$

برای استخراج مقدار ثابت **k**، پارامترهای اولیه را در فرمول هیپ جای‌گذاری می‌کنیم. از نظر ریاضی، به توان 0.5 رساندن یک عدد دقیقاً معادل محاسبه ریشه دوم (جذر) آن است. بنابراین، جذر 1,000,000 برابر با 1,000 می‌شود. با ساده‌سازی معادله و تقسیم 50,000 (تعداد کلمات) بر 1,000 (جذر توکن‌ها)، مقدار ثابت تناسب برای این زبان و مجموعه داده به صورت دقیق برابر با **50** محاسبه می‌گردد.

بخش 4) تخمین اندازه واژه‌نامه پس از پردازش 4000000 توکن

$$\begin{aligned}T_{new} &= 4,000,000 \\M_{new} &= 50 \times (4,000,000)^{0.5} \\M_{new} &= 50 \times \sqrt{4,000,000} \\M_{new} &= 50 \times 2,000 = 100,000\end{aligned}$$

در مرحله دوم پردازش، حجم توکن‌های مجموعه دادگان به 4,000,000 می‌رسد. اکنون با استفاده از ثابت مقیاسی که در مرحله قبل محاسبه کردیم ($k = 50$)، می‌توانیم حجم جدید واژه‌نامه را تخمین بزنیم. جذر 4,000,000 برابر با 2,000 است. با ضرب این مقدار در ضریب مقیاس، اندازه کل واژه‌نامه تقریباً برابر با **100,000 کلمه یکتا** پیش‌بینی می‌شود.

بخش 5) محاسبه تعداد کلمات یکتای جدید اضافه‌شده

$$\begin{aligned}M_{initial} &= 50,000 \\M_{final} &= 100,000 \\N_{new} &= M_{final} - M_{initial} \\N_{new} &= 100,000 - 50,000 = 50,000\end{aligned}$$

هدف نهایی، یافتن تعداد کلمات کاملاً جدیدی است که طی این ارتقای حجم وارد سیستم شده‌اند. برای این کار، اندازه واژه‌نامه پیش‌بینی‌شده در مرحله دوم (M_{final}) را از اندازه واژه‌نامه در مرحله اول ($M_{initial}$) کسر می‌کنیم. نتیجه نشان می‌دهد که با پردازش 3,000,000 توکنِ مازاد، تعداد **50,000 کلمه یکتای جدید** کشف و به واژه‌نامه موتور جستجو اضافه خواهد شد.

بخش 6) نکته مفهومی مهم درباره نتیجه

$$\begin{aligned}T_2 &= 4 \times T_1 \\ \sqrt{T_2} &= \sqrt{4 \times T_1} = 2 \times \sqrt{T_1} \\ M_2 &= 2 \times M_1\end{aligned}$$

ارتباط ریاضی نمایش‌داده‌شده بیانگر ماهیت **رشد ریشه‌ای** قانون هیپ است. زمانی که نمای رشد (b) معادل 0.5 باشد، افزایش اندازه واژه‌نامه با جذر افزایش تعداد توکن‌ها متناسب خواهد بود. در این سناریو، اگرچه حجم پردازش متن (T) معادل **4 برابر** شده است، اما واژه‌نامه موتور جستجو تنها به میزان جذر آن، یعنی **2 برابر**، گسترش یافته است. این امر نشان‌دهنده کاهش نرخ ورود کلمات جدید در متون بسیار طولانی است.

بخش 7) پاسخ نهایی مسئله

$$\begin{aligned}k &= 50 \\M_{final} &= 100,000 \\N_{new} &= 50,000\end{aligned}$$

به عنوان جمع‌بندی؛ با استفاده از رابطه ریاضی هیپ ($M = k \times T^b$)، مقدار ثابت سیستم برابر با **50** استخراج شد. اندازه کل واژه‌نامه پس از رسیدن به 4,000,000 توکن، **100,000 کلمه یکتا** تخمین زده می‌شود و نهایتاً، در این فاز توسعه، سیستم **50,000 کلمه کاملاً متمایز جدید** را به واژه‌نامه اصلی خود اضافه خواهد کرد.

پاسخنامه مسئله 5.6: توزیع فرکانس کلمات بر اساس قانون زیپف

بخش 1) مفهوم قانون زیپف

$$f_i = \frac{f_1}{i}$$

طبق قانون زیپف، فرکانس تکرار واژه‌ها با رتبه آن‌ها رابطه‌ای معکوس دارد. در معادله فوق، f_i فرکانس واژه با رتبه i و f_1 فرکانس پرتکرارترین واژه (رتبه 1) است. این رابطه نشان می‌دهد هرچه رتبه یک واژه بزرگ‌تر شود، فرکانس تکرار آن به صورت $i/1$ کاهش می‌یابد. بنابراین، واژه دوم نیمی از فرکانس واژه اول، واژه سوم یک‌سوم آن و به همین ترتیب فرکانس سایر واژه‌ها محاسبه می‌شود.

بخش 2) داده‌های مسئله

$$f_1 = 1,000,000$$

طبق داده‌های صورت سؤال، فرکانس پرتکرارترین واژه در این مجموعه متنی دقیقاً **1,000,000** بار است. هدف اصلی این مسئله، یافتن مجموع فرکانس واژه‌هایی است که در رتبه‌های 2 تا 5 قرار دارند.

بخش 3) محاسبه فرکانس هر رتبه

$$f_2 = \frac{1,000,000}{2} = 500,000$$

$$f_3 = \frac{1,000,000}{3} \approx 333,333$$

$$f_4 = \frac{1,000,000}{4} = 250,000$$

$$f_5 = \frac{1,000,000}{5} = 200,000$$

با قرار دادن مقدار f_1 در رابطه زیپف، فرکانس تک‌تک رتبه‌های مورد نظر استخراج می‌شود. بر این اساس، کلمه دارای رتبه دوم دقیقاً 500,000 بار، کلمه سوم (با تقریب) 333,333 بار، کلمه چهارم 250,000 بار و کلمه پنجم 200,000 بار در مجموعه اسناد تکرار شده‌اند.

بخش 4) محاسبه مجموع فرکانس‌ها

$$\sum_{i=2}^5 f_i = 500,000 + 333,333 + 250,000 + 200,000 = 1,283,333$$

در این گام، برای رسیدن به خواسته مسئله، تمامی مقادیر به‌دست‌آمده برای رتبه‌های 2 الی 5 را با یکدیگر جمع می‌کنیم. حاصل جمع فرکانس این چهار کلمه برابر با **1,283,333** خواهد بود.

بخش 5) پاسخ نهایی

$$f_1 = 1,000,000$$

$$\sum_{i=2}^5 f_i \approx 1,283,333$$

به عنوان جمع بندی؛ با داشتن فرکانس پرتکرارترین کلمه (1,000,000 بار)، مجموع دفعات تکرار کلماتی که در رتبه های 2 تا 5 قرار دارند طبق توزیع مدل زیپف محاسبه گردید. این مقدار تقریباً برابر با **1,283,333** مرتبه است که معادل است با حدود **1.28 million occurrences**.

پاسخنامه مسئله 5.7: تحلیل ناهنجاری در رشد واژه نامه با استفاده از قانون هیپ

بخش 1) صورت مسئله

در این مسئله، لاگ های یک موتور جست و جو بررسی می شود. در گزارش ماه اول، تعداد توکن های پردازش شده برابر با **100,000,000** است و اندازه واژه نامه برابر با **400,000** کلمه گزارش شده است. در ماه دوم، با اضافه شدن اسناد جدید، تعداد کل توکن ها به **120,000,000** می رسد؛ یعنی تعداد توکن ها افزایش پیدا کرده است. اما اندازه واژه نامه سیستم به جای افزایش، عدد **380,000** کلمه را نشان می دهد. هدف این است که با استفاده از ساختار قانون هیپ (**Heaps' Law**)، این پدیده را تحلیل کنیم و مشخص کنیم آیا این وضعیت الزاماً به معنی خطای محاسباتی است یا می تواند توجیه منطقی در حوزه پردازش متن داشته باشد.

بخش 2) یادآوری قانون هیپ

$$M = k \times T^b$$

$$0 < b < 1$$

قانون هیپ برای تخمین رشد اندازه واژه نامه در یک مجموعه متنی استفاده می شود. در این رابطه، **M** اندازه واژه نامه (کلمات یکتا)، **T** تعداد کل توکن های پردازش شده، **k** ثابت مقیاس، و **b** توان رشد واژه نامه است. نامساوی **0 < b < 1** نشان می دهد که با افزایش تعداد توکن ها، اندازه واژه نامه رشد می کند، اما سرعت این رشد زیرخطی است. بنابراین در حالت طبیعی، با پردازش اسناد بیشتر، انتظار داریم واژه نامه بزرگ تر شود یا دست کم ثابت بماند؛ نه اینکه کاهش یابد.

بخش 3) تحلیل داده های ماه اول

$$400,000 = k \times (100,000,000)^b$$

در ماه اول، سیستم با پردازش **100,000,000** توکن موفق به استخراج **400,000** کلمه یکتا شده است. با قرار دادن این مقادیر در رابطه قانون هیپ، معادله فوق حاصل می شود. تا این مرحله، داده های ماه اول به تنهایی رفتار غیرعادی خاصی را نشان نمی دهند، زیرا ما فقط یک نقطه از نمودار رشد سیستم را در اختیار داریم و برای تحلیل رفتار، نیازمند بررسی روند تغییرات هستیم.

بخش 4) تحلیل داده های ماه دوم

$$\frac{T_2}{T_1} = \frac{120,000,000}{100,000,000} = 1.2$$

$$\frac{M_2}{M_1} = \frac{380,000}{400,000} = 0.95$$

با بررسی گزارش ماه دوم متوجه یک تناقض می‌شویم. محاسبات نشان می‌دهند که تعداد کل توکن‌های سیستم **20%** افزایش یافته است (ضریب 1.2)؛ اما در کمال تعجب، اندازه واژه‌نامه به جای رشد، **5%** کاهش یافته است (ضریب 0.95). این وضعیت از منظر قانون هیپ کاملاً غیرعادی است، زیرا در یک مجموعه تجمعی و پایدار، افزایش ورودی (توکن‌ها) نباید به کاهش خروجی (یکتا) (واژه‌نامه) منجر شود.

بخش 5) تحلیل ریاضی با نسبت‌های قانون هیپ

$$\frac{M_2}{M_1} = \left(\frac{T_2}{T_1} \right)^b$$

$$0.95 = (1.2)^b$$

با فرض اینکه پارامترهای **k** و **b** ثابت مانده باشند، نسبت اندازه واژه‌نامه‌ها باید با نسبت توکن‌ها به توان **b** برابر باشد. با قرار دادن ضرایب استخراج‌شده در این فرمول، به معادله **$1.2^b = 0.95$** می‌رسیم. این رابطه از نظر ریاضی دچار یک تعارض است؛ زیرا عدد 1.2 از 1 بزرگ‌تر است و هر توان مثبتی از آن، قطعاً نتیجه‌ای بزرگ‌تر از 1 خواهد داشت. خروجی 0.95 ایجاب می‌کند که توان **b** عددی منفی باشد، که این موضوع با منطق پردازش متن ناسازگار است.

بخش 6) محاسبه مقدار ضمنی پارامتر رشد

$$b = \frac{\log(0.95)}{\log(1.2)}$$

$$b \approx -0.28$$

برای اثبات دقیق این ناهنجاری، با استفاده از لگاریتم، مقدار ضمنی پارامتر **b** را استخراج می‌کنیم. محاسبه نشان می‌دهد که پارامتر رشد مقدار **-0.28** را به خود می‌گیرد. در پردازش متون طبیعی، توان **b** همواره عددی مثبت و در بازه حدودی 0.4 تا 0.6 قرار دارد. این نتیجه منفی اثبات می‌کند که داده‌های گزارش‌شده، پیرو الگوی طبیعی رشد تجمعی در قانون هیپ نیستند.

بخش 7) نتیجه ریاضی اولیه

از نظر ریاضی، اگر پیکره ماه دوم ادامه همان پیکره ماه اول باشد، اسناد قبلی حذف نشده باشند، تعریف کلمه یکتا تغییر نکرده باشد، روش توکن‌سازی و نرمال‌سازی ثابت مانده باشد و واژه‌نامه به صورت تجمعی محاسبه شده باشد، اندازه واژه‌نامه نباید از **400,000** به **380,000** کاهش پیدا کند. در چنین شرایط پایداری، کاهش اندازه واژه‌نامه نشانه یک ناهنجاری است و می‌تواند به خطای محاسباتی، خطای گزارش‌گیری، خطای لاگ، یا تغییر پنهان در فرایند پردازش اشاره کند.

بخش 8) آیا این وضعیت حتماً خطای محاسباتی است؟

پاسخ دقیق این است که نه، این وضعیت الزاماً به معنی خطای محاسباتی نیست. اما اگر گزارش ادعا کند که ماه دوم دقیقاً ادامه ماه اول است و همه تنظیمات ثابت مانده‌اند، آنگاه این کاهش از نظر قانون هیپ غیرطبیعی است. با این حال، در پروژه‌های واقعی پردازش متن، دلایل فنی متعددی وجود دارد که می‌تواند باعث شود حجم واژه‌نامه با وجود افزایش توکن‌ها، کوچک‌تر گزارش شود.

بخش 9) توجیه‌های منطقی در پردازش متن

چندین سناریوی فنی می‌تواند این کاهش را توجیه کند:

- تغییر نرمال‌سازی (Normalization):** اگر در ماه دوم سیستم شکل‌های مختلف نوشتاری (مثل یکسان‌سازی حروف بزرگ/کوچک یا ی/ی در فارسی) را یکپارچه کرده باشد، تعداد کلمات یکتا کاهش می‌یابد.
- ریشه‌یابی (Stemming/Lemmatization):** اعمال ریشه‌یابی در ماه دوم باعث می‌شود کلمات هم‌خانواده (مثل runs و running) به یک بن‌واژه واحد تبدیل شوند.
- پاک‌سازی داده‌ها:** حذف نویزها، خطاهای تایپی، اسپم‌ها و آدرس‌های اینترنتی که در ماه اول به‌عنوان واژه شمرده می‌شدند، حجم واژه‌نامه را به‌شدت کم می‌کند.
- حذف اسناد بی‌کیفیت:** اضافه شدن توکن‌های جدید ممکن است هم‌زمان با فیلتر شدن انبوهی از اسناد تکراری یا خارج از زبان هدف در پایگاه داده رخ داده باشد.
- اعمال آستانه فراوانی:** سیستم ممکن است کلمات بسیار نادری را که فراوانی آن‌ها کمتر از یک حد مشخص (مثلاً فقط 1 تکرار) بوده است، از نمایه نهایی حذف کرده باشد.
- تغییر روش توکن‌سازی:** به‌روزرسانی قوانین جداسازی کلمات (مثلاً نحوه برخورد با خط تیره یا نیم‌فاصله) مستقیماً روی استخراج کلمات یکتا تأثیر می‌گذارد.

بخش 10) تفسیر نهایی بر اساس قانون هیپ

$$\frac{M_2}{M_1} = \sqrt{1.2} \approx 1.095$$
$$M_2 \approx 400,000 \times 1.095 = 438,000$$

برای درک بهتر اختلاف، اگر فرض کنیم روند پردازش ثابت بوده و پارامتر رشد را مقدار رایج $b = 0.5$ در نظر بگیریم، با 20% افزایش توکن‌ها، اندازه واژه‌نامه باید حدود 9.5% رشد می‌کرد. در این صورت انتظار می‌رفت حجم واژه‌نامه در ماه دوم به حدود 438,000 کلمه برسد. شکاف عمیق بین این پیش‌بینی و عدد گزارش‌شده (380,000) نشانگر آن است که قواعد استخراج واژگان در سیستم دچار تغییرات ساختاری شده است.

بخش 11) پاسخ نهایی مسئله

از نظر ریاضی و بر اساس الگوی طبیعی قانون هیپ، کاهش اندازه واژه‌نامه از 400,000 به 380,000 هم‌زمان با رشد توکن‌ها به 120,000,000، یک ناهنجاری محسوب می‌شود. در یک روند تجمعی پایدار، سیستم دچار خطای محاسباتی یا گزارش‌گیری شده است. اما در محیط عملیاتی، اعمال تکنیک‌های بهینه‌سازی متن (مثل ریشه‌یابی، نرمال‌سازی، و حذف نویز و کلمات نادر) توجیه‌های کاملاً منطقی و استانداردی برای این کاهش به شمار می‌آیند.

نتیجه‌گیری: بر اساس قانون هیپ با پارامترهای ثابت، این گزارش غیرطبیعی است؛ اما در دنیای واقعی، به‌روزرسانی خط‌لوله پردازش متن می‌تواند دلیل اصلی این کاهش باشد و الزاماً نمایانگر خطا نیست.

پاسخنامه مسئله 5.8 : شناسایی و فیلتر کلمات توقف بر پایه قانون زیپف

بخش 1 (صورت مسئله

قانون زیپف بیانگر این واقعیت است که در متون طبیعی، بخش عمده‌ای از کل توکن‌ها توسط تعداد انگشت‌شماری از واژه‌ها با فرکانس بسیار بالا تولید می‌شوند. این کلمات پرتکرار، که غالباً نقش دستوری داشته و فاقد بار معنایی غنی هستند، در حوزه بازایی اطلاعات تحت عنوان «کلمات توقف» یا **Stop Words** شناخته می‌شوند. هدف این مسئله، طراحی یک سازوکار الگوریتمی است که با استفاده از دیکشنری فرکانس کلمات، آن دسته از واژگانی را شناسایی کند که مجموع تکرار آن‌ها، درصدی مشخص (مثلاً **10%**) از کل توکن‌های مجموعه را پوشش می‌دهد.

بخش 2 (مفهوم پایه و ارتباط با قانون زیپف

توزیع زیپفی نشان می‌دهد که فراوانی کلمات در زبان دارای عدم‌توازن شدیدی است. تعداد بسیار کمی از واژه‌ها دارای فرکانس بسیار بالا بوده و بخش بزرگی از حجم متن را اشغال می‌کنند. برای شناسایی کلمات توقف، کافی است کلمات را بر اساس فرکانس به‌صورت نزولی مرتب کرده و با جمع تدریجی فراوانی‌ها، به نقطه‌ای برسیم که درصد آستانه هدف محقق شود. کلماتی که در این بازه ابتدایی (پرتکرارترین‌ها) قرار می‌گیرند، همان کلمات توقف هستند.

بخش 3 (ورودی‌های الگوریتم

طراحی الگوریتم به سه ورودی اصلی نیاز دارد:

- **دیکشنری فرکانس کلمات:** نگاشت کلمات به تعداد تکرار آن‌ها
- **تعداد کل توکن‌ها (T):** مجموع تمامی تکرارهای موجود در مجموعه
- **درصد آستانه (P):** نسبتی از کل توکن‌ها (به‌عنوان مثال **0.10**) که باید توسط کلمات توقف پوشش داده شود

بخش 4 (گام‌های اجرایی الگوریتم

فرایند محاسباتی الگوریتم در گام‌های زیر خلاصه می‌شود

گام 1: محاسبه کل توکن‌ها

ابتدا مجموع فراوانی تمام کلمات موجود در دیکشنری محاسبه می‌شود

$$\text{Total Tokens} = \sum(f_i)$$

گام 2: محاسبه مقدار آستانه

میزان پوشش مورد نظر با استفاده از رابطه زیر تعیین می‌شود

$$\text{Threshold} = P \times \text{Total Tokens}$$

گام 3: مرتب‌سازی نزولی

دیکشنری کلمات بر اساس فرکانس (f) به‌صورت نزولی مرتب می‌شود تا واژه‌های پرتکرار در ابتدای لیست قرار گیرند.

گام 4: جمع تجمعی و شناسایی

فراوانی کلمات از ابتدای لیست مرتب‌شده به صورت تجمعی جمع می‌شود تا زمانی که

$$\text{Cumulative Sum} \geq \text{Threshold}$$

کلماتی که تا رسیدن به این شرط در مجموعه قرار گرفته‌اند، به‌عنوان «کلمات توقف» علامت‌گذاری می‌شوند.

بخش 5 (مزایای فیلترسازی و تفسیر نهایی

حذف کلمات توقف شناسایی شده منجر به بهینه‌سازی‌های زیر در سیستم بازیابی اطلاعات می‌شود

- کاهش حجم حافظه مصرفی توسط نمایه معکوس (Inverted Index)
- افزایش سرعت پردازش و پاسخ‌دهی پرس‌وجوها
- کاهش نویز معنایی؛ چرا که این کلمات نقش تمایزدهنده در بازیابی اسناد ندارند

نتیجه‌گیری: این رویکرد الگوریتمی، نسبت به لیست‌های ایستای کلمات توقف، برتری دارد؛ زیرا به‌جای تکیه بر دانش پیشینی زبان، خود را با ویژگی‌های آماری منحصربه‌فرد هر پیکره متنی (بر اساس توزیع زیپفی) تطبیق می‌دهد.

پاسخنامه مسئله 5.9: محاسبه حافظه در ساختار آرایه با طول ثابت

بخش 1) صورت مسئله

در این مسئله، یک واژه‌نامه شامل **200,000** کلمه در نظر گرفته شده است. میانگین طول واقعی هر کلمه برابر با **6** بایت است؛ اما واژه‌نامه قرار است با استفاده از ساختار **Fixed-Length Array** یا آرایه با طول ثابت ذخیره شود. در این روش، برای هر کلمه یک بلوک حافظه با اندازه ثابت اختصاص داده می‌شود که طبق صورت مسئله، برابر با **15** بایت است. هدف مسئله این است که مشخص کنیم برای ذخیره کل واژه‌نامه در این ساختار، در مجموع چند بایت و چند مگابایت حافظه مورد نیاز خواهد بود.

بخش 2) مفهوم ساختار آرایه با طول ثابت

$$\text{Block Size} = 15 \text{ bytes}$$

در ساختار آرایه با طول ثابت، برای هر عنصر آرایه مقدار مشخص و ثابتی از حافظه رزرو می‌شود؛ به این معنی که حتی اگر طول واقعی یک داده کمتر از اندازه بلوک باشد، باز هم کل فضای آن بلوک برای آن داده اشغال می‌گردد. این روش دسترسی مستقیم و ساده‌تری به عناصر فراهم می‌کند، اما ممکن است باعث اتلاف حافظه شود. در این مسئله، با وجود اینکه میانگین طول کلمات تنها **6** بایت است، سیستم برای هر کلمه دقیقاً **15** بایت حافظه ثابت رزرو می‌کند. بنابراین، مبنای محاسبه حافظه کل، همین اندازه بلوک ثابت است.

بخش 3) محاسبه حافظه مورد نیاز برای کل واژه‌نامه

$$\text{Total Memory} = \text{Number of Words} \times \text{Block Size}$$

$$\text{Total Memory} = 200,000 \times 15$$

$$\text{Total Memory} = 3,000,000 \text{ bytes}$$

تعداد کل کلمات واژه‌نامه **200,000** کلمه است و برای ذخیره هر یک از آن‌ها **15** بایت فضا رزرو می‌شود. بنابراین حافظه کل مورد نیاز، از ضرب مستقیم این دو مقدار به دست می‌آید که برابر با **3,000,000** بایت است. توجه داشته باشید که عدد **6** بایت (میانگین طول واقعی کلمات) در محاسبه فضای اشغالی نهایی هیچ نقش مستقیمی ندارد، زیرا ساختار طول ثابت این فضای اضافی را پرشده در نظر می‌گیرد.

بخش 4) تبدیل بایت به مگابایت

$$1 \text{ MB} = 1,048,576 \text{ bytes}$$

$$\text{Total Memory (MB)} = \frac{3,000,000}{1,048,576} \approx 2.86 \text{ MB}$$

برای معادل‌سازی مقدار حافظه از واحد بایت به مگابایت، از ثابت استاندارد که در آن هر مگابایت برابر با **1,048,576** بایت است استفاده می‌کنیم. با تقسیم کل حجم محاسباتی، یعنی **3,000,000** بایت بر این عدد، نتیجه نهایی نشان می‌دهد که فضای واقعی رزرو شده در این ساختار تقریباً معادل **2.86** مگابایت خواهد بود.

بخش 5) تحلیل اتلاف حافظه در ساختار طول ثابت

$$\text{Waste per Word} = 15 - 6 = 9 \text{ bytes}$$

$$\text{Total Waste} = 200,000 \times 9 = 1,800,000 \text{ bytes}$$

اگرچه پاسخ اصلی مسئله صرفاً بر اساس اندازه بلوک ثابت محاسبه شد، اما بررسی میزان هدررفت حافظه، هزینه این رویکرد را روشن‌تر می‌سازد. از آنجا که به طور میانگین کلمات **6** بایت طول دارند اما درون کادرهای **15** بایتی گنجانده می‌شوند، در ازای هر کلمه **9** بایت فضا خالی می‌ماند. با تسری این اتلاف به کل مجموعه، دریافتیم که **1,800,000** بایت از فضای ذخیره‌سازی، کاملاً بلااستفاده باقی می‌ماند. این آمار اثبات می‌کند که روش **Fixed-Length Array** علیرغم سرعت بالا در دسترسی، برای متونی با طول متغیر به شدت پرهزینه است.

بخش 6) پاسخ نهایی مسئله

$$\text{Total Memory} = 200,000 \times 15 = 3,000,000 \text{ bytes}$$

$$\text{Total Memory (MB)} = \frac{3,000,000}{1,048,576} \approx 2.86 \text{ MB}$$

به عنوان یک جمع‌بندی؛ برای استقرار واژه‌نامه‌ای متشکل از **200,000** کلمه در داخل یک ساختار آرایه با طول ثابت، به شرط اختصاص دادن بلوک‌های **15** بایتی برای هر ورودی، سیستم دقیقاً به **3,000,000** بایت حافظه خام نیاز خواهد داشت.

بنابراین حافظه مورد نیاز برای ذخیره کل واژه‌نامه در ساختار Fixed Length Array تقریباً برابر با 2.86 MB است.

پاسخنامه مسئله 5.10: فشرده‌سازی واژه‌نامه به روش Dictionary as a String

بخش 1) یادآوری داده‌های مسئله قبل

$$\text{Total Memory (Fixed-Length)} = 200,000 \times 15 = 3,000,000 \text{ bytes} \approx 2.86 \text{ MB}$$

همان‌طور که در محاسبه فوق مشخص است، در مسئله پیشین واژه‌نامه‌ای شامل **200,000** کلمه با میانگین طول واقعی **6** بایت در اختیار داشتیم. در آن سناریو با استفاده از ساختار **Fixed-Length Array** برای هر کلمه دقیقاً **15** بایت حافظه رزرو می‌شد که در نهایت حافظه کل مصرفی سیستم را به حدود **2.86** مگابایت می‌رساند.

در روش بهینه‌تر **Dictionary as a String**، بلوک‌های با اندازه ثابت حذف می‌شوند و تمام کلمات به صورت پشت‌سرهم در قالب یک رشته بزرگ و پیوسته ذخیره می‌گردند. برای تفکیک کلمات از یکدیگر، یک ساختار داده کمکی ایجاد می‌شود که برای هر کلمه یک «اشاره‌گر» (Pointer) نگهداری می‌کند تا محل شروع آن را در رشته اصلی مشخص کند. در این مسئله، اندازه هر اشاره‌گر برابر با **3** بایت در نظر گرفته شده است. بنابراین در این معماری، حافظه کل شامل دو بخش مجزا خواهد بود: حافظه کاراکترهای پیوسته و حافظه اشاره‌گرها.

بخش 3) محاسبه حافظه کاراکترها

$$\text{Characters Memory} = 200,000 \times 6 = 1,200,000 \text{ bytes}$$

برای محاسبه فضای اشغالی توسط خود کلمات در این رشته طولانی، از میانگین طول واقعی آن‌ها استفاده می‌کنیم. با توجه به اینکه میانگین طول هر یک از **200,000** کلمه برابر با **6** بایت است، تنها برای ذخیره کاراکترهای خالص به **1,200,000** بایت حافظه نیاز خواهیم داشت.

بخش 4) محاسبه حافظه اشاره‌گرها

$$\text{Pointers Memory} = 200,000 \times 3 = 600,000 \text{ bytes}$$

علاوه بر کاراکترها، سیستم باید ساختار کمکی را نیز در حافظه نگه دارد. از آنجا که برای هر یک از **200,000** کلمه، یک اشاره‌گر مجزای **3** بایتی تخصیص می‌یابد، کل فضای مورد نیاز برای ذخیره‌سازی اشاره‌گرها برابر با **600,000** بایت خواهد بود.

بخش 5) محاسبه حافظه کل در روش جدید

$$\text{Total Memory} = 1,200,000 + 600,000 = 1,800,000 \text{ bytes}$$

حافظه نهایی مورد نیاز در معماری Dictionary as a String از مجموع فضای کاراکترها و اشاره‌گرها به دست می‌آید. با جمع این دو بخش، کل حافظه خام محاسبه‌شده دقیقاً برابر با **1,800,000** بایت می‌شود.

بخش 6) تبدیل به مگابایت

$$1 \text{ MB} = 1,048,576 \text{ bytes}$$
$$\text{Total Memory (MB)} = \frac{1,800,000}{1,048,576} \approx 1.72 \text{ MB}$$

برای معادل‌سازی این حجم به واحد مگابایت، مقدار به دست آمده را بر عدد استاندارد مبنای دو (1,048,576) تقسیم می‌کنیم. نتیجه محاسبات نشان می‌دهد که در این ساختار جدید، حافظه کل اشغال‌شده توسط واژه‌نامه تقریباً معادل **1.72** مگابایت خواهد بود.

بخش 7) میزان صرفه‌جویی نسبت به روش قبلی

$$\text{Memory Saved} = 2.86 - 1.72 = 1.14 \text{ MB}$$

با یک مقایسه ساده میان دو سناریو، تفاوت کارایی آن‌ها مشخص می‌شود. در حالی که حافظه مصرفی در ساختار آرایه با طول ثابت حدود **2.86** مگابایت بود، روش جدید این مقدار را به **1.72** مگابایت کاهش داد.

نتیجه‌گیری: استفاده از ساختار Dictionary as a String باعث جلوگیری از اتلاف فضای درون‌بلوکی شده و در نتیجه صرفه‌جویی چشمگیری به میزان تقریبی **1.14 MB** در مصرف حافظه به همراه دارد.

پاسخنامه مسئله 5.11: معماری حافظه در ذخیره‌سازی بلوکی (Blocked Storage)

بخش 1) صورت مسئله و داده‌ها

$$\begin{aligned} \text{Words} &= \{\text{system, systematic, systems, systemic}\} \\ \text{Block Size } (k) &= 4 \end{aligned}$$

در این مسئله چهار کلمه مشخص جهت مدیریت در حافظه در اختیار ما قرار دارد. روش ذخیره‌سازی مورد استفاده، Blocked Storage است که در آن طبق مفروضات سیستم، ظرفیت و اندازه هر بلوک (**k**) برابر با **4** کلمه تعیین شده است.

بخش 2) توضیح مفهومی ذخیره‌سازی بلوکی

در روش‌های ساده پردازش متن، به ازای هر کلمه یک اشاره‌گر جداگانه ذخیره می‌شود که باعث افزایش شدید سربار حافظه می‌گردد. در روش ذخیره‌سازی بلوکی، چندین کلمه در یک بلوک مشترک قرار می‌گیرند و تنها یک اشاره‌گر به ابتدای آن بلوک نگهداری می‌شود. از آنجا که اندازه بلوک برابر با **4** است، هر چهار کلمه این مسئله در یک بلوک واحد ذخیره شده و فقط یک اشاره‌گر برای فراخوانی کل این مجموعه کافی خواهد بود.

بخش 3) ضرورت ذخیره‌سازی طول کلمات

$$\text{Structure} = [\text{Length}][\text{Word}]$$

به‌منظور شناسایی دقیق مرز بین کلمات در رشته‌ای یکپارچه، از پارامتری به نام پیشوند طول (Length Prefix) استفاده می‌شود. بدین ترتیب، فرمت ذخیره‌سازی هر کلمه در حافظه طبق ساختار نمایش داده‌شده تنظیم می‌گردد. این رویکرد به سیستم اجازه می‌دهد هنگام خواندن داده‌ها، ابتدا طول کلمه را پردازش کرده و سپس دقیقاً همان تعداد کاراکتر بعدی را استخراج نماید.

بخش 4) محاسبه طول کلمات

$$|\text{system}| = 6$$

$$|\text{systematic}| = 10$$

$$|\text{systems}| = 7$$

$$|\text{systemic}| = 8$$

برای پیاده‌سازی این الگو، طول کاراکتری کلمات باید به دقت استخراج شود. همان‌طور که در روابط ریاضی بالا استخراج شده است، طول کلمه نخست 6 کاراکتر، کلمه دوم 10 کاراکتر، کلمه سوم 7 کاراکتر و نهایتاً کلمه چهارم 8 کاراکتر محاسبه گردیده است که به‌عنوان پیشوند عددی آن‌ها لحاظ خواهد شد.

بخش 5) رشته یکپارچه درون بلوک

$$\text{Block String} = 6 \text{ system } 10 \text{ systematic } 7 \text{ systems } 8 \text{ systemic}$$

با ترکیب پیشوندهای عددی (طول کلمات) و خود کلمات به صورت متوالی، رشته پیوسته نهایی جهت ذخیره‌سازی در حافظه ساخته می‌شود. این رشته در حافظه اصلی هیچ‌گونه فضای خالی اضافه‌ای برای جداسازی ندارد و مرز بین مقادیر صرفاً با اعداد پیشوند کنترل می‌شود.

بخش 6) بازیابی و نتیجه نهایی

فرایند بازیابی اطلاعات با خواندن مقادیر طول آغاز می‌شود؛ برای نمونه، سیستم با خواندن عدد 6 متوجه می‌شود که باید 6 بایت بعدی را برداشت کند تا کلمه اول (system) استخراج شود. این چرخه خوانش پیشوند و برداشت کاراکتر تا پایان بلوک ادامه می‌یابد.

نتیجه‌گیری: با ذخیره‌سازی این چهار کلمه در یک بلوک با اندازه 4 و با بهره‌گیری از یک اشاره‌گر واحد به‌جای چهار اشاره‌گر مستقل، مصرف حافظه در نمایه سیستم به‌شدت بهینه‌سازی می‌شود.

پاسخنامه مسئله 5.12: بررسی کارایی ذخیره‌سازی بلوکی برای واژه‌های بسیار کوتاه

بخش 1) فرض مسئله

در این مسئله واژه‌های واژه‌نامه از توالی‌های ژنتیکی بسیار کوتاه تشکیل شده‌اند که طول آن‌ها تنها 2 یا 3 کاراکتر است. همچنین فرض شده است که هر کاراکتر معادل 1 بایت، اندازه هر اشاره‌گر 4 بایت و ظرفیت بلوک در روش Blocked Storage برابر با k کلمه است. در معماری ذخیره‌سازی بلوکی، کلمات یک بلوک به صورت یک رشته پیوسته ذخیره می‌شوند و برای تفکیک آن‌ها از یکدیگر، پیش از هر کلمه طول آن نیز در قالب متادیتا ثبت می‌گردد.

بخش 2) حالت اول: ذخیره‌سازی معمولی (بدون بلوک‌بندی)

$$\text{Word Space} \approx 2.5 \text{ bytes}$$

$$\text{Pointer Space} = 4 \text{ bytes}$$

$$\text{Total Space per Word} = 2.5 + 4 = 6.5 \text{ bytes}$$

در رویکرد متعارف Dictionary as a String، تمامی کلمات به صورت متوالی در یک رشته ذخیره شده و برای هر یک، یک اشاره گر اختصاصی **4** بیتی در ساختار کمکی نگهداری می شود. با در نظر گرفتن میانگین طول **2.5** کاراکتر (یا همان 2.5 بایت) برای این توالی های ژنتیکی، فضای کل مورد نیاز برای ذخیره هر واژه از جمع فضای کاراکترها و اشاره گر اختصاصی آن به دست می آید که مجموعاً برابر با **6.5** بایت خواهد بود.

بخش 3) حالت دوم: ذخیره سازی بلوکی با $k = 8$

$$\text{Pointer Share} = \frac{4}{8} = 0.5 \text{ bytes}$$

$$\text{Length Metadata} = 1 \text{ byte}$$

$$\text{Actual Word} \approx 2.5 \text{ bytes}$$

$$\text{Total Space per Word} = 0.5 + 1 + 2.5 = 4 \text{ bytes}$$

در ساختار Blocked Storage، هزینه اشاره گر بین کلمات یک بلوک سرشکن می شود. در این سناریو با داشتن بلوک های **8** تایی، سهم هر کلمه از اشاره گر تنها **0.5** بایت خواهد بود. اما در این روش، برای اسکن و خواندن کلمات به صورت متوالی، سیستم ملزم است پیش از هر کلمه طول آن را نیز ثبت کند که حداقل به **1** بایت متادیتا نیاز دارد. با اضافه کردن این دو مقدار به میانگین طول خود واژه (**2.5** بایت)، کل فضای مصرفی برای هر کلمه در این معماری برابر با **4** بایت برآورد می گردد.

بخش 4) تحلیل اتلاف فضا در این دامنه خاص

$$\text{Actual Word Space} = 2 \text{ bytes}$$

$$\text{Length Metadata} = 1 \text{ byte}$$

$$\text{Metadata Overhead} \approx 50\%$$

در پردازش متون استاندارد که طول کلمات معمولاً **8** تا **10** بایت است، اختصاص یک بایت متادیتا برای پیشوند طول، سربار ناچیزی محسوب می شود. اما در دامنه توالی های ژنتیکی با طول استثنائاً کوتاه (مانند کلمات **2** بیتی)، افزودن **1** بایت پیشوند عددی بدان معناست که نیمی از فضای ذخیره سازی داده (یعنی حدود **50%** سربار) صرفاً به نگهداری اطلاعات متادیتا اختصاص می یابد، که این امر کارایی روش بلوکی را به شدت افت می دهد.

بخش 5) پاسخ نهایی

در دامنه ای مانند توالی های ژنتیکی که واژه ها تنها **2** تا **3** کاراکتر طول دارند، استفاده از روش Blocked Storage با اندازه بلوک **k = 8** کارایی مطلوبی ندارد. اگرچه این روش با سرشکن کردن اشاره گرها باعث کاهش فضای کل مصرفی (از 6.5 بایت به 4 بایت در هر کلمه) می شود، اما الزام ذخیره طول هر کلمه (حداقل **1** بایت) سربار نسبی بسیار بالایی ایجاد می کند.

نتیجه گیری: به دلیل کوچکی بیش از حد داده های واقعی، متادیتای افزوده در ذخیره سازی بلوکی بخش بزرگی از حجم حافظه را می بلعد. لذا در این واژه نامه خاص، بلوک بندی منجر به اتلاف نسبی فضا شده و مزیت فشرده سازی آن نسبت به دامنه های متنی استاندارد بسیار کم رنگ تر خواهد بود.

پاسخنامه مسئله 5.13: جستجوی دودویی در واژه نامه به روش رشته یکپارچه

در روش ذخیره‌سازی «دیکشنری به عنوان یک رشته» که با عنوان **Dictionary as a String** شناخته می‌شود، تمامی کلمات واژه‌نامه به صورت متوالی و پشت سر هم در یک رشته طولانی ذخیره می‌شوند. چالش اصلی این است که در این رشته، مرزهای مشخصی (مانند فاصله یا کاراکترهای جداکننده) بین کلمات وجود ندارد. پرسش این است که چگونه می‌توان عملیات جستجوی دودویی (**Binary Search**) را که نیاز به دسترسی مستقیم به عناصر میانی دارد، روی این ساختار یکپارچه پیاده‌سازی کرد.

ساختار داده‌های مورد نیاز (Offset Array)

برای اینکه بتوانیم روی ساختار «رشته یکپارچه» عملیات جستجوی دودویی انجام دهیم، وجود یک ساختار کمکی به نام «آرایه اشاره‌گرها» (**Offset Array**) الزامی است. اگر فقط خود رشته کاراکترها را داشته باشیم، برای پیدا کردن کلمه وسط باید تمام رشته را از ابتدا پیمایش کنیم که این کار زمان جستجو را از حالت بهینه (لگاریتمی) به حالت (خطی) تبدیل می‌کند. بنابراین، سیستم واژه‌نامه همیشه در کنار رشته اصلی، یک آرایه از اعداد صحیح نگه می‌دارد. هر عضو در این آرایه، «آدرس» یا همان «موقعیت شروع» کلمه مربوطه در رشته اصلی را ذخیره می‌کند. برای مثال، خانه اول آرایه (اندیس 0) موقعیت شروع کلمه اول را نشان می‌دهد و خانه دوم آرایه (اندیس 1) موقعیت شروع کلمه دوم را مشخص می‌کند. با استفاده از این آرایه، سیستم دقیقاً می‌داند هر کلمه در چه نقطه‌ای از رشته اصلی آغاز و در چه نقطه‌ای تمام می‌شود.

سازوکار دسترسی به کلمات در طول جستجو

$$\text{Length} = \text{Offset}[i + 1] - \text{Offset}[i]$$

برای تعیین اندازه و خواندن یک کلمه در طول جستجو، سیستم از اندیس‌های آرایه اشاره‌گرها استفاده می‌کند. همان‌طور که در رابطه ریاضی فوق مشخص است، تفاوت بین موقعیت شروع کلمه بعدی و موقعیت شروع کلمه جاری، دقیقاً برابر با «طول کلمه جاری» خواهد بود. سیستم با داشتن «موقعیت شروع» و «طول کلمه»، بخش مربوطه را از رشته اصلی جدا کرده و آن را برای مقایسه با کلمه مورد جستجو استخراج می‌کند.

مراحل اجرای الگوریتم Binary Search

$$\text{Mid} = \left\lfloor \frac{\text{Low} + \text{High}}{2} \right\rfloor$$

$$\text{High} = \text{Mid} - 1 \quad (\text{If Target} < \text{Mid Word})$$

$$\text{Low} = \text{Mid} + 1 \quad (\text{If Target} > \text{Mid Word})$$

Stop Condition : $\text{Low} > \text{High}$

با فرض وجود آرایه اشاره‌گرها، سیستم عملیات جستجو را با تنظیم دو متغیر **Low** و **High** در ابتدا و انتهای دامنه آغاز کرده و با فرمول نخست، اندیس میانی (**Mid**) را محاسبه می‌کند. پس از خواندن کلمه میانی، در صورتی که کلمه هدف از نظر الفبایی کوچکتر باشد، با استفاده از فرمول دوم دامنه جستجو به نیمه اول محدود می‌گردد. بالعکس، در صورت بزرگتر بودن کلمه هدف، با فرمول سوم دامنه به نیمه دوم شیف‌ت پیدا می‌کند.

این رویه تا یافتن کلمه مورد نظر و یا تحقق شرط توقف (فرمول چهارم) که نشان‌دهنده نبود کلمه در واژه‌نامه است، تکرار می‌گردد.

نتیجه‌گیری: با این ساختار، سیستم بدون نیاز به پیمایش کامل رشته یکپارچه، تنها با مرتبه زمانی $O(\log N)$ به کلمه هدف دسترسی پیدا می‌کند و پیچیدگی زمانی بهینه الگوریتم کاملاً حفظ می‌شود.

پاسخنامه مسئله 5.14: رمزگذاری فواصل با استفاده از کدهای بایت متغیر

بخش 1) صورت مسئله

در این مسئله، یک لیست پستینگ مربوط به یک کلمه خاص داده شده است. این لیست شامل شناسه‌های اسناد 120، 135، 263 و 10,503 است. هدف مسئله دو مرحله دارد. ابتدا باید لیست فواصل یا همان **Gap List** را از روی شناسه‌های اسناد استخراج کنیم. سپس باید این فواصل را با استفاده از روش **Variable-Byte Codes** به نمایش باینری تبدیل کنیم. در این روش، بیت پرارزش هر بایت به عنوان بیت ادامه یا **Continuation Bit** در نظر گرفته می‌شود. اگر بیت ادامه برابر با 1 باشد، یعنی عدد هنوز تمام نشده و بایت بعدی نیز متعلق به همین عدد است، و اگر بیت ادامه برابر با 0 باشد، یعنی این بایت آخرین بایت مربوط به عدد فعلی است.

بخش 2) مفهوم لیست فواصل

در سیستم‌های بازیابی اطلاعات، لیست پستینگ معمولاً شامل شناسه اسنادی است که یک کلمه خاص در آن‌ها ظاهر شده است. ذخیره مستقیم شناسه‌ها همیشه بهینه نیست، زیرا ممکن است بعضی از شناسه‌ها اعداد بزرگی باشند. برای کاهش حجم ذخیره‌سازی، به جای ذخیره خود شناسه‌ها، اختلاف بین شناسه‌های پشت سر هم ذخیره می‌شود. به این اختلاف‌ها **Gap** گفته می‌شود. در این روش، شناسه اول معمولاً بدون تغییر ذخیره می‌شود و سپس برای هر شناسه بعدی، اختلاف آن با شناسه قبلی محاسبه می‌گردد.

بخش 3) استخراج لیست فواصل

$$\begin{aligned} \text{Gap}_1 &= 120 \\ \text{Gap}_2 &= 135 - 120 = 15 \\ \text{Gap}_3 &= 263 - 135 = 128 \\ \text{Gap}_4 &= 10,503 - 263 = 10,240 \end{aligned}$$

برای ساختن لیست فواصل، شناسه سند اول یعنی 120 را بدون تغییر ثبت می‌کنیم. سپس همان‌طور که در معادلات فوق محاسبه شده است، هر شناسه بعدی را از شناسه قبلی آن کسر می‌کنیم تا فاصله میان آن‌ها به دست آید. بدین ترتیب، فواصل 15، 128 و 10,240 استخراج می‌شوند. در نهایت، لیست فواصل نهایی سیستم به ترتیب برابر با 120، 15، 128 و 10,240 خواهد بود.

بخش 4) مفهوم کدگذاری Variable-Byte Code

در روش **Variable-Byte Code**، هر عدد با استفاده از یک یا چند بایت ذخیره می‌شود. هر بایت شامل 8 بیت است که در آن، بیت پرارزش (MSB) برای مشخص کردن ادامه داشتن عدد استفاده می‌شود. در نتیجه هر بایت شامل 1 بیت ادامه و 7 بیت داده

است. اگر عددی در ظرفیت **7** بیت جا شود، تنها با یک بایت نمایش داده می‌شود و بیت ادامه آن برابر با **0** خواهد بود. اما اگر عدد بزرگ‌تر باشد، به گروه‌های **7** بیتی تقسیم می‌شود و برای همه بایت‌ها به جز بایت آخر، بیت ادامه برابر با **1** قرار می‌گیرد.

بخش 5) کدگذاری اعداد

$120 \Rightarrow 0\ 1111000$
 $15 \Rightarrow 0\ 0001111$
 $128 \Rightarrow 1\ 0000001\quad 0\ 0000000$
 $10,240 \Rightarrow 1\ 1010000\quad 0\ 0000000$

همان‌طور که در کادر بالا مشاهده می‌شود، تک‌تک اعداد موجود در لیست فواصل به باینری تبدیل شده و سپس بر اساس روش بایت متغیر بلوک‌بندی شده‌اند. اعداد **120** و **15** در قالب **7** بیت داده به راحتی جا می‌گیرند، بنابراین تنها با یک بایت نشان داده شده و بیت پرارزش (ادامه) آن‌ها برابر با **0** تنظیم می‌گردد. اما اعداد **128** و **10,240** فراتر از ظرفیت **7** بیت هستند؛ در نتیجه به دو بایت شکسته می‌شوند. در این حالت، برای بایت اول بیت ادامه برابر با **1** و برای بایت آخر بیت ادامه برابر با **0** قرار داده می‌شود تا نشان‌دهنده پایان عدد باشد.

بخش 6) جمع‌بندی نهایی

در این مسئله ابتدا لیست شناسه‌های اسناد به لیست فواصل شامل **120**، **15**، **128** و **10,240** تبدیل شد. سپس هر فاصله با استفاده از روش **Variable-Byte Code** و بر اساس تقسیم به گروه‌های **7** بیتی کدگذاری گردید. در اعداد کوچک از یک بایت و در اعداد بزرگ‌تر از دو بایت استفاده شد. بدین ترتیب، هم مرحله استخراج فاصله‌ها و هم مرحله فشرده‌سازی باینری به درستی و با بهینه‌ترین حالت انجام پذیرفت.

پاسخنامه مسئله 5.15: مهندسی معکوس و خطایابی جریان بیتی کدهای گاما

بخش اول: ساختار Gamma Codes

کدگذاری گاما یک روش فشرده‌سازی برای اعداد صحیح مثبت است که از دو بخش تشکیل شده است. بخش اول، بخش طول (**Length**) نام دارد و شامل تعدادی عدد **1** است که با یک عدد **0** به پایان می‌رسد. تعداد این اعداد **1** برابر با بخش صحیح لگاریتم عدد در مبنای دو (**$\log x$**) است. بخش دوم، بخش افسست (**Offset**) است که شامل نمایش باینری عدد، بدون بیت پرارزش (**Most Significant Bit**) می‌باشد. این ساختار باعث می‌شود کدهای گاما دارای ویژگی پیشوندی یا همان **Prefix-free** باشند؛ بدین معنا که هیچ کدی پیشوندِ کد دیگر نیست و سیستم می‌تواند جریان بیتی را بدون ابهام و بدون نیاز به جداکننده، به کدهای مجزا تفکیک کند.

بخش دوم: تحلیل جریان بیتی ورودی

جریان بیتی ارائه شده به صورت **111111100101** است. برای رمزگشایی، ابتدا باید اولین عدد **0** را پیدا کنیم تا انتهای بخش طول مشخص شود. با خواندن بیت‌ها از سمت چپ، هفت عدد **1** و سپس یک عدد **0** مشاهده می‌شود:

11111110

این هشت بیت نشان‌دهنده بخش طول هستند و مقدار **7** را برای بخش افست تعیین می‌کنند. طبق تعریف کدهای گاما، اکنون سیستم انتظار دارد هفت بیت بعدی، داده‌های مربوط به بخش افست باشند.

بخش سوم: بن‌بست منطقی در رمزگشایی

پس از خواندن بخش طول، جریان بیتی باقی‌مانده به صورت زیر است:

0101

در اینجا با یک بن‌بست منطقی مواجه هستیم. طبق محاسبات، سیستم برای تکمیل بخش افست نیاز به **7** بیت داده دارد، اما تنها **4** بیت در جریان بیتی باقی مانده است. این اختلاف بین «تعداد مورد انتظار» و «تعداد موجود»، باعث می‌شود عملیات رمزگشایی متوقف شود و سیستم نتواند عدد اصلی را بازیابی کند.

بخش چهارم: نقص در جریان بیتی و ویژگی پیشوندی

ویژگی **Prefix-free** زمانی کارایی دارد که جریان داده کامل و بدون نقص باشد. در این مسئله، جریان بیتی ناقص است و «ساختار دستوری» کد گاما را نقض کرده است. وقتی سیستم در میانه خواندن بخش افست با پایان جریان بیتی مواجه می‌شود، نه تنها عدد فعلی ناقص باقی می‌ماند، بلکه مسیر برای تشخیص شروع کد بعدی نیز مسدود می‌گردد. نقص ساختاری در اینجا به معنای فروپاشی کامل فرآیند رمزگشایی است، زیرا سیستم به یک نقطه کور می‌رسد که در آن بازیابی اطلاعات غیرممکن است.

بخش پنجم: نتیجه‌گیری نهایی

با توجه به تحلیل‌های فوق، رمزگشایی شناسه‌های اسناد از این جریان بیتی امکان‌پذیر نیست. بن‌بست منطقی به دلیل کمبود سه بیت در بخش افست رخ داده است. ویژگی **Prefix-free** در مواجهه با داده‌های مخدوش، توانایی بازیابی ندارد و سیستم نمی‌تواند به بخش‌های بعدی جریان دسترسی یابد. این مسئله نشان‌دهنده خطای جدی در ذخیره‌سازی یا فساد داده در فایل پستینگ است.

پاسخنامه مسئله 5.16: نسبت فراوانی واژه‌ها بر اساس قانون زیپ (Zipf's Law)

بخش 1) یادآوری قانون زیپ

$$f_i = \frac{F}{i}$$

طبق قانون زیپ (**Zipf's Law**)، فراوانی واژه‌ها در یک مجموعه متنی با رتبه آن‌ها رابطه معکوس دارد. به عبارت دیگر، هرچه رتبه یک واژه در فهرست واژه‌های مرتب‌شده بر اساس فراوانی بزرگ‌تر باشد، تعداد وقوع آن واژه کمتر خواهد بود. در معادله ریاضی فوق، اگر فرکانس پرتکرارترین واژه (رتبه 1) را با متغیر **F** نشان دهیم، فرکانس واژه‌ای با رتبه **i** تقریباً معادل کسر **F / i** محاسبه می‌گردد. این رابطه به وضوح نشان می‌دهد که فراوانی واژه‌ها با افزایش رتبه به صورت معکوس و کسری کاهش می‌یابد.

بخش 2) محاسبه برای رتبه 10

$$f_{10} = \frac{F}{10}$$

با جای‌گذاری مقدار رتبه در رابطه قانون زیپ، می‌خواهیم فراوانی واژه‌ای را که در رتبه **10** قرار دارد به دست آوریم. همان‌طور که معادله نشان می‌دهد، فراوانی واژه با جایگاه دهم، تقریباً برابر با یک‌دهم فراوانی پرتکرارترین واژه در کل مجموعه متنی خواهد بود.

بخش 3) تعیین نسبت فراوانی

$$\text{Ratio} = \frac{f_{10}}{F}$$

$$\text{Ratio} = \frac{\frac{F}{10}}{F} = \frac{1}{10}$$

برای به دست آوردن نسبت فراوانی واژه رتبه **10** به واژه رتبه **1**، مقدار محاسبه‌شده در مرحله قبل را بر فرکانس واژه اول (**F**) تقسیم می‌کنیم. با ساده‌سازی این کسر، متغیر **F** از صورت و مخرج حذف شده و در نهایت نسبت دقیق **1/10** به دست می‌آید.

بخش 4) پاسخ نهایی

$$f_{10} = \frac{F}{10}$$

$$\frac{f_{10}}{F} = \frac{1}{10}$$

به‌عنوان جمع‌بندی؛ طبق قانون زیپ، فراوانی واژه‌ای که در رتبه **10** قرار دارد، معادل کسر **F / 10** محاسبه می‌شود. بنابراین، نسبت فراوانی آن به پرتکرارترین واژه، دقیقاً برابر با **10 / 1** است.

نتیجه‌گیری: واژه‌ای که در جایگاه دهم قرار گرفته است، تقریباً یک‌دهم فراوانی واژه نخست را به خود اختصاص می‌دهد.

پاسخنامه مسئله 5.17: تأثیر فشرده‌سازی واژه‌نامه در سیستم‌های بازیابی اطلاعات

بخش اول: مفهوم فشرده‌سازی واژه‌نامه

در سیستم‌های بازیابی اطلاعات، ساختار **Dictionary** شامل تمام واژه‌های یکتای موجود در مجموعه اسناد است. هر واژه معمولاً همراه با اطلاعاتی مانند اشاره‌گر به **Postings List** و سایر متاداده‌های مرتبط ذخیره می‌شود.

با افزایش اندازه مجموعه اسناد، تعداد واژه‌های متمایز نیز افزایش می‌یابد و در نتیجه اندازه واژه‌نامه بزرگ‌تر می‌شود. برای کاهش حجم حافظه مورد نیاز، از روش‌های **فشرده‌سازی واژه‌نامه** استفاده می‌شود. این روش‌ها معمولاً با حذف بخش‌های تکراری رشته‌ها (مانند پیشوندهای مشترک) یا استفاده از روش‌های کدگذاری فشرده، حجم داده‌های ذخیره‌شده را کاهش می‌دهند.

الف) تأثیر بر مصرف حافظه

مهم‌ترین اثر فشرده‌سازی واژه‌نامه، کاهش مصرف حافظه است. در بسیاری از زبان‌ها، واژه‌های متوالی در واژه‌نامه دارای پیشوندهای مشترک هستند. اگر هر واژه به طور کامل ذخیره شود، این پیشوندها بارها در ساختار داده تکرار خواهند شد.

فشرده‌سازی واژه‌نامه با حذف این تکرارها باعث می‌شود تعداد بایت‌های مورد نیاز برای ذخیره واژه‌ها کاهش یابد و در نتیجه واژه‌نامه فضای کمتری در حافظه اصلی (**RAM**) یا فضای ذخیره‌سازی دیسک اشغال کند.

این کاهش مصرف حافظه به‌ویژه در مجموعه‌های بسیار بزرگ مانند موتورهای جست‌وجوی وب اهمیت زیادی دارد.

بنابراین می‌توان نتیجه گرفت که فشرده‌سازی واژه‌نامه باعث **کاهش قابل توجه مصرف حافظه** در سیستم‌های بازیابی اطلاعات می‌شود.

ب) تأثیر بر سرعت دسترسی به واژه‌ها

اثر فشرده‌سازی بر سرعت دسترسی به واژه‌ها دارای دو جنبه متفاوت است. از یک طرف، هنگام جست‌وجو در واژه‌نامه ممکن است لازم باشد داده‌ها ابتدا از حالت فشرده خارج شوند یا رشته‌های واژه بازسازی شوند.

این عملیات نوعی پردازش اضافی ایجاد می‌کند و ممکن است دسترسی مستقیم به داده‌ها را اندکی کندتر کند.

از طرف دیگر، از آنجا که واژه‌نامه فشرده‌شده حجم کمتری دارد، احتمال بیشتری وجود دارد که کل آن در حافظه اصلی **RAM** نگهداری شود. دسترسی به داده‌ها در حافظه اصلی بسیار سریع‌تر از خواندن آن‌ها از دیسک است.

در بسیاری از سیستم‌های بازیابی اطلاعات، مزیت کوچک‌تر شدن واژه‌نامه و قرارگیری آن در حافظه اصلی معمولاً از هزینه پردازش اضافی مهم‌تر است. به همین دلیل، فشرده‌سازی در عمل می‌تواند باعث **بهبود سرعت کلی دسترسی** به واژه‌ها شود.

ج) تأثیر بر مقیاس‌پذیری سیستم

مقیاس‌پذیری (**Scalability**) به این معناست که یک سیستم بتواند با افزایش حجم داده‌ها همچنان عملکرد قابل قبولی داشته باشد.

فشرده‌سازی واژه‌نامه در این زمینه نقش مهمی ایفا می‌کند، زیرا باعث کاهش فضای ذخیره‌سازی مورد نیاز برای نگهداری داده‌ها می‌شود. در نتیجه، داده‌های بیشتری می‌توانند در حافظه اصلی قرار بگیرند و انتقال داده بین حافظه و دیسک نیز کاهش می‌یابد.

این ویژگی به سیستم اجازه می‌دهد مجموعه‌های اسنادی بسیار بزرگ‌تر را مدیریت کند. در موتورهای جست‌وجوی بزرگ که میلیاردها سند را پردازش می‌کنند، استفاده از روش‌های فشرده‌سازی برای حفظ کارایی سیستم ضروری است.

بنابراین فشرده‌سازی واژه‌نامه باعث **افزایش مقیاس‌پذیری سیستم‌های بازیابی اطلاعات** و امکان مدیریت مجموعه‌های داده بسیار بزرگ می‌شود.

پاسخ نهایی

الف) مصرف حافظه: فشرده‌سازی واژه‌نامه با حذف داده‌های تکراری و استفاده از روش‌های کدگذاری فشرده، باعث کاهش قابل توجه مصرف حافظه می‌شود.

ب) سرعت دسترسی: اگرچه ممکن است بازسازی داده‌ها مقداری پردازش اضافی ایجاد کند، اما کوچک‌تر شدن واژه‌نامه و امکان نگهداری آن در **RAM** معمولاً باعث بهبود سرعت کلی دسترسی می‌شود.

(ج) **مقیاس‌پذیری سیستم:** با کاهش حجم داده‌ها و استفاده بهینه از حافظه، فشرده‌سازی واکه‌نامه باعث افزایش مقیاس‌پذیری سیستم‌های بازیابی اطلاعات و توانایی مدیریت مجموعه‌های بسیار بزرگ اسناد می‌شود.

پاسخنامه مسئله 5.18: ارزیابی تکنیک‌های فشرده‌سازی در سطح سخت‌افزار

بخش 1) صورت مسئله

در این مسئله دو تکنیک رایج فشرده‌سازی در سیستم‌های بازیابی اطلاعات، یعنی **Variable-Byte Codes** و **Gamma Codes**، از دو دیدگاه متفاوت مقایسه می‌شوند. دیدگاه اول مربوط به میزان فشرده‌سازی داده‌ها در شرایطی است که فواصل بین شناسه‌های اسناد (**Gaps**) بسیار بزرگ باشند. دیدگاه دوم به سرعت پردازش و عملیات خارج‌سازی داده‌ها از حالت فشرده (**Decompression**) اختصاص دارد تا مشخص شود کدام روش روی معماری پردازنده‌های مدرن **32** بیتی یا **64** بیتی عملکرد بهینه‌تری دارد.

بخش 2) مروری بر مفهوم فاصله‌ها در لیست پستینگ

در سیستم‌های بازیابی اطلاعات، به جای ذخیره مستقیم شناسه‌های اسناد، معمولاً اختلاف بین شناسه‌های متوالی ذخیره می‌شود که به آن فاصله یا **Gap** گفته می‌شود. برای مثال، اگر شناسه‌های اسناد شامل **100**، **130**، **190** و **210** باشد، لیست فاصله‌ها به صورت **100**، **30**، **60** و **20** خواهد بود. در بسیاری از موارد فاصله‌ها کوچک هستند، اما در مجموعه‌داده‌های بزرگ یا اسناد پراکنده، ممکن است این فاصله‌ها به اعداد بسیار بزرگی تبدیل شوند که این امر رفتار و کارایی تکنیک‌های فشرده‌سازی را کاملاً تغییر می‌دهد.

بخش 3) رفتار Gamma Codes در فشرده‌سازی فاصله‌های بزرگ

$$\text{Gamma Length} \approx 2b - 1$$

در روش **Gamma Code** هر عدد به دو بخش «طول» و «آفست» تقسیم می‌شود. همان‌طور که در رابطه بالا مشهود است، اگر یک عدد برای نمایش در مبنای دو به **b** بیت نیاز داشته باشد، طول کل کد گاما تقریباً برابر با **2b - 1** خواهد بود (تقریباً دو برابر طول واقعی عدد). هنگامی که فاصله‌ها کوچک باشند، این روش بسیار کارآمد است؛ اما با بزرگ شدن فاصله‌ها، مقدار **b** نیز افزایش می‌یابد و سرِبارِ بخشِ طول (که به صورت متوالی از یک‌ها تشکیل شده) باعث می‌شود حجم ذخیره‌سازی با شیب تندی رشد کند.

بخش 4) رفتار Variable-Byte Codes در فشرده‌سازی فاصله‌های بزرگ

$$\text{VB Length (bits)} = 8 \times \left\lceil \frac{b}{7} \right\rceil$$

در روش **Variable-Byte Code**، هر بایت شامل **1** بیت برای کنترل ادامه عدد و **7** بیت برای داده واقعی است. بر اساس معادله فوق، اگر عددی به **b** بیت داده نیاز داشته باشد، تعداد بایت‌های مورد نیاز برابر با سقف کسر **b/7** است. با ضرب این مقدار در **8**

(تعداد بیت‌های هر بایت)، طول کل کد به دست می‌آید. رشد طول کد در این روش تقریباً خطی و متناسب با خود **b** است، نه دو برابر آن. بنابراین در مواجهه با فاصله‌های بزرگ، این معماری پایداری بسیار بهتری از خود نشان می‌دهد.

بخش 5) مقایسه عملی میزان فشرده‌سازی

$$\text{Gamma Length} = (2 \times 21) - 1 = 41 \text{ bits}$$

$$\text{VB Length} = \left\lceil \frac{21}{7} \right\rceil \times 8 = 3 \times 8 = 24 \text{ bits}$$

برای درک بهتر، فرض کنید فاصله‌ای داریم که برای نمایش دودویی دقیقاً به **21** بیت داده نیاز دارد. همان‌طور که در محاسبات یکپارچه بالا مشاهده می‌کنید، در روش **Gamma Code** طول این کد با فرمول دو برابری به **41** بیت می‌رسد. اما در روش **Variable-Byte**، با تقسیم **21** بیت بر ظرفیت **7** بیتی هر بایت، دقیقاً به **3** بایت نیاز خواهیم داشت که طول نهایی آن معادل **24** بیت می‌شود. این مقایسه ریاضی به روشنی اثبات می‌کند که در فاصله‌های بزرگ، روش **Variable-Byte** فضای بسیار کمتری را هدر می‌دهد.

بخش 6) مقایسه سرعت Decompression

روش **Variable-Byte** کاملاً بایت‌محور است و با معماری پردازنده‌های مدرن (که ذاتاً بر اساس بایت، کلمه **32** بیتی یا **64** بیتی کار می‌کنند) سازگاری کامل دارد. در این روش، بایت‌ها مستقیماً خوانده شده و با عملیات ساده شیفت بیتی (Shift) و ترکیب (OR) به سرعت بازسازی می‌شوند. در مقابل، **Gamma Code** یک روش بیت‌محور است؛ یعنی مرز کدها الزاماً با مرز بایت‌های حافظه هم‌تراز نیست. این موضوع باعث می‌شود شمارش تعداد یک‌ها و استخراج بیت‌ها پیچیده‌تر شده و نیاز به عملیات ماسک‌گذاری (Masking) متعددی داشته باشد که وابستگی ترتیبی ایجاد کرده و بهینه‌سازی سخت‌افزاری پردازنده را دشوار می‌کند.

بخش 7) نتیجه نهایی

با بررسی جنبه‌های مختلف می‌توان نتیجه گرفت که در شرایطی که فاصله‌های اسناد (Gaps) بسیار بزرگ باشند، تکنیک **Variable-Byte Codes** برتری همه‌جانبه و محسوسی دارد. از نظر فشرده‌سازی، رشد خطی این روش در برابر رشد دوبرابری **Gamma Codes**، فضای بهینه‌تری را اشغال می‌کند.

نتیجه‌گیری: از نظر سرعت پردازش، سازگاری ساختار بایت‌محور Variable-Byte با معماری سخت‌افزار مدرن (32/64 بیتی)، موجب کارایی بالاتر و سرعت به‌مراتب بیشتر در عملیات Decompression نسبت به روش سنتی و بیت‌محور گاما می‌گردد.

پاسخنامه مسئله 5.19: تحلیل بدترین حالت حافظه در ذخیره‌سازی نمایه‌ها

بخش 1) صورت مسئله و ادعای طراح

$$\text{Max Gap Memory} = 2 \log_2(N) + 1$$

$$\text{Designer Claim} = M \times (2 \log_2(N) + 1)$$

در این مسئله، یک لیست پستینگ برای کلمه‌ای خاص شامل **M** شناسه سند از میان **N** سند کل مجموعه بررسی می‌شود. شناسه‌ها به صورت فواصل (**Gap**) ذخیره و با استفاده از ساختار **Gamma Code** فشرده می‌شوند. ادعای طراح سیستم این است که در بدترین حالت، هر فاصله می‌تواند به تنهایی به بزرگی کل مجموعه (**N**) باشد و در نتیجه، وی مدعی است برای کل لیست پستینگ، حافظه مورد نیاز بر اساس ضرب تعداد اسناد در طول ماکزیمم کدهای گاما (همانند معادله دوم) محاسبه خواهد شد.

بخش 2) تعریف ساختار فاصله‌ها و محدودیت مجموع آن‌ها

$$\begin{aligned} g_1 &= d_1 \\ g_2 &= d_2 - d_1 \\ &\vdots \\ g_M &= d_M - d_{M-1} \\ \sum_{i=1}^M g_i &= d_M \leq N \end{aligned}$$

فرض کنید شناسه‌های اسناد مرتب‌شده به صورت **d₁, d₂, ..., d_M** باشند. همان‌طور که در معادلات بالا مشخص است، فواصل (گپ‌ها) به صورت تفاضل هر شناسه با شناسه قبلی‌اش تعریف می‌شوند. یک ویژگی بنیادی و بسیار مهم در این ساختار این است که اگر تمام فاصله‌ها را با هم جمع کنیم، به دلیل حذف متقابل متغیرهای میانی، مجموع کل آن‌ها دقیقاً برابر با بزرگ‌ترین شناسه سند (یعنی **d_M**) خواهد شد. از آنجا که شناسه اسناد نمی‌تواند از کل حجم مجموعه فراتر رود، قطعاً مجموع کل فاصله‌ها همواره کوچکتر یا مساوی **N** خواهد بود.

بخش 3) نقد ادعای طراح

$$\text{False Assumption Sum} = M \times N \not\leq N$$

اگر فرض طراح صحیح می‌بود و هر فاصله می‌توانست مستقل از بقیه به اندازه ماکزیمم **N** رشد کند، آنگاه مجموع فاصله‌ها برابر با حاصل‌ضرب **M × N** می‌شد. اما در بخش قبل به صورت ریاضی اثبات کردیم که مجموع فاصله‌ها از لحاظ ساختاری هرگز نمی‌تواند از **N** فراتر رود. بنابراین، فرض طراح از اساس نادرست و غیرمنطقی است و فواصل متوالی در لیست پستینگ کاملاً به یکدیگر وابسته‌اند.

بخش 4) تحلیل حافظه با استفاده از ویژگی‌های لگاریتمی

$$\text{Total Memory} = \sum_{i=1}^M (2 \log_2(g_i) + 1)$$

$$\text{Total Memory} = 2 \sum_{i=1}^M \log_2(g_i) + M$$

$$\text{Worst Case } g_i \approx \frac{N}{M}$$

$$\sum_{i=1}^M \log_2(g_i) \approx M \log_2\left(\frac{N}{M}\right)$$

برای یافتن مصرف واقعی حافظه در بدترین حالت، طول کد گامای تک تک فاصله‌ها را با یکدیگر جمع می‌کنیم که معادله اول را تشکیل می‌دهد. بر اساس اصول بهینه‌سازی و نامساوی ینسن (Jensen's Inequality)، برای بیشینه شدن مجموع لگاریتم‌ها تحت این شرط که مجموع متغیرها ثابت است، فاصله‌ها باید به صورت یکنواخت توزیع شوند. بنابراین در بدترین حالت، هر فاصله تقریباً برابر با کسر **N / M** خواهد بود که با قرار دادن آن در معادله اصلی، مجموع لگاریتم‌ها بهینه‌سازی می‌شود.

بخش 5) کران بالای واقعی حافظه

$$\text{Actual Upper Bound} = M \times \left(2 \log_2\left(\frac{N}{M}\right) + 1 \right)$$

با جای‌گذاری نتیجه توزیع یکنواخت گپ‌ها در معادله اصلی حافظه، کران بالای دقیق و ریاضی حافظه برای ذخیره کل لیست پستینگ استخراج می‌شود. این فرمول نمایانگر سقف واقعی مصرف حافظه است.

بخش 6) نتیجه نهایی

تحلیل دقیق ریاضی اثبات می‌کند که فرض اولیه طراح سیستم، که حافظه کل را بدون در نظر گرفتن محدودیت مجموع فاصله‌ها محاسبه کرده بود، نادرست و به شدت بدبینانه است.

نتیجه‌گیری: به دلیل محدودیت فیزیکی مجموع فاصله‌ها، کران بالای واقعی و منطقی حافظه برای لیست پستینگ با استفاده از کدهای گاما دقیقاً برابر با $M \times (2 \log_2(N/M) + 1)$ خواهد بود، که از لحاظ حجم به مراتب کمتر و بهینه‌تر از تصور اولیه طراح است.

پاسخنامه مسئله 5.20: مقایسه پارامترهای رشد در مدل هیپ

بخش 1) صورت مسئله و نقش قانون هیپ

$$M = k \times T^\beta$$

قانون هپ برای پیش‌بینی رشد اندازه واژه‌نامه (دیکشنری) در یک پیکره متنی استفاده می‌شود. در رابطه ریاضی فوق، M نشان‌دهنده تعداد واژه‌های متمایز (اندازه دیکشنری) و T نشان‌دهنده تعداد کل توکن‌های مشاهده‌شده در پیکره است. در این فرمول، k یک ضریب ثابت است که روی مقدار اولیه و سطح کلی اندازه دیکشنری اثر می‌گذارد، در حالی که β (توان رشد) تعیین می‌کند دیکشنری با چه سرعتی نسبت به افزایش حجم متن رشد کند. در این مسئله، دو پیکره متنی داریم: در پیکره اول، مقدار k بزرگ‌تر است و در پیکره دوم، مقدار β بزرگ‌تر است. هدف، بررسی این موضوع است که در نهایت (با افزایش حجم اسناد) کدام پیکره دیکشنری بزرگ‌تری خواهد داشت.

بخش 2) اثر بزرگ‌تر بودن ضریب k در پیکره اول

$$\begin{aligned}\text{Corpus 1 : } M_1 &= k_1 \times T^{\beta_1} \\ \text{Corpus 2 : } M_2 &= k_2 \times T^{\beta_2} \\ \text{Condition : } k_1 &> k_2\end{aligned}$$

ضریب k مانند یک مقیاس اولیه عمل می‌کند. همان‌طور که در معادلات بالا مشخص شده است، به دلیل آنکه $k_1 > k_2$ می‌باشد، اگر دو پیکره در ابتدا حجم مشابه (و کوچکی) داشته باشند، پیکره‌ای که k بزرگ‌تری دارد، در آغاز تعداد واژه‌های متمایز بیشتری از خود نشان می‌دهد. بنابراین در حجم‌های پایین یا متوسط، همین ضریب بزرگ‌تر می‌تواند باعث شود اندازه دیکشنری اول (M_1) موقتاً از دیکشنری دوم (M_2) بزرگ‌تر باشد.

بخش 3) اثر بزرگ‌تر بودن توان β در پیکره دوم

$$\begin{aligned}\text{Condition : } \beta_2 &> \beta_1 \\ \lim_{T \rightarrow \infty} : T^{\beta_2} &\gg T^{\beta_1}\end{aligned}$$

پارامتر β نقش بسیار کلیدی‌تری در بلندمدت ایفا می‌کند، زیرا به عنوان «توان» در معادله حضور دارد. وقتی تعداد توکن‌ها (T) بسیار بزرگ می‌شود، تفاوت در توان رشد بسیار تأثیرگذارتر از تفاوت در ضریب ثابت k خواهد بود. به زبان ساده، k روی مقدار اولیه اثر دارد، اما β شتاب یا نرخ رشد بلندمدت را تعیین می‌کند. از آنجا که $\beta_2 > \beta_1$ است، با افزایش چشمگیر T ، جمله T^{β_2} بسیار سریع‌تر از T^{β_1} رشد کرده و در نهایت از آن پیشی می‌گیرد.

بخش 4) مقایسه ریاضی دقیق

$$\begin{aligned}\frac{M_2}{M_1} &= \frac{k_2 \times T^{\beta_2}}{k_1 \times T^{\beta_1}} \\ \frac{M_2}{M_1} &= \left(\frac{k_2}{k_1} \right) \times T^{\beta_2 - \beta_1} \\ \text{Since } (\beta_2 - \beta_1) &> 0 \implies \lim_{T \rightarrow \infty} T^{\beta_2 - \beta_1} = \infty \\ \therefore M_2 &> M_1\end{aligned}$$

برای اثبات این موضوع، نسبت اندازه دیکشنری پیکره دوم به پیکره اول را محاسبه می‌کنیم. اگرچه مقدار کسر k_2 / k_1 عددی کوچک‌تر از یک است، اما چون توان $\beta_1 - \beta_2$ یک مقدار کاملاً مثبت است، عبارت $T^{\beta_2 - \beta_1}$ با بزرگ شدن T به سمت بی‌نهایت میل می‌کند. در نتیجه، رشد نمایی این بخش در نهایت کسر کوچک ضرایب را خنثی کرده و با قطعیت باعث می‌شود $M_2 > M_1$ گردد.

بخش 5) تفسیر مفهومی در دنیای واقعی

از منظر مفهومی، پیکره اول (با k بزرگ‌تر) ممکن است در همان ابتدا واژگان بسیار متنوعی داشته باشد؛ به عنوان مثال شامل مجموعه‌ای از اسناد با سبک‌های زبانی گوناگون باشد. اما پیکره دوم (با β بزرگ‌تر) ماهیتی دارد که با ورود اسناد جدید، واژه‌های کاملاً تازه‌ای با سرعت بالا به آن تزریق می‌شوند (مثل شبکه اجتماعی که مدام موضوعات و هشتک‌های جدید در آن ترند می‌شود). بنابراین در کوتاه‌مدت، پیکره اول غنی‌تر به نظر می‌رسد، اما در بلندمدت پیکره دوم به دلیل نرخ بالای کشف واژگان، دیکشنری بسیار گسترده‌تری خواهد ساخت.

بخش 6) پاسخ نهایی

در حجم‌های کوچک، ممکن است پیکره اول به دلیل برتری در ضریب اولیه، دیکشنری بزرگ‌تری داشته باشد؛ اما در حجم‌های بزرگ متنی، توان ریاضی تعیین‌کننده اصلی خواهد بود.

نتیجه‌گیری: از آنجا که تأثیر پارامتر β در بلندمدت و با رشد چشمگیر T بسیار قوی‌تر از ضریب ثابت k است، در نهایت پیکره دوم (با توان β بزرگ‌تر) قطعاً واژه‌نامه (دیکشنری) بزرگ‌تری خواهد داشت.

پاسخنامه مسئله 5.21: محاسبات تخمینی ابعاد دیکشنری با قانون هیپس

بخش 1) داده‌های مسئله

$$\begin{aligned}T &= 100,000,000 = 10^8 \\k &= 50 \\\beta &= 0.5 \\M &= k \times T^\beta\end{aligned}$$

طبق صورت مسئله، یک سیستم بازیابی اطلاعات مجموعه‌ای متنی شامل **100,000,000** توکن را پردازش کرده است. مقادیر ثابت قانون هیپس برای این مجموعه خاص به صورت $k = 50$ و $\beta = 0.5$ داده شده‌اند. در فرمول قانون هیپس، M نشان‌دهنده تعداد تقریبی کلمات یکتا (اندازه دیکشنری) است که از طریق رابطه توانی فوق با استفاده از حجم توکن‌ها محاسبه می‌گردد.

بخش 2) محاسبه تعداد تقریبی کلمات یکتا

$$M = 50 \times (100,000,000)^{0.5}$$

$$M = 50 \times \sqrt{100,000,000}$$

$$M = 50 \times 10,000$$

$$M = 500,000 \text{ unique words}$$

برای یافتن اندازه دیکشنری، مقادیر داده شده را در فرمول قانون هیپس جای گذاری می کنیم. در ریاضیات، توان **0.5** معادل گرفتن ریشه دوم (جذر) عدد است. از آنجا که مجذور **10,000** برابر با **100,000,000** می شود، جذر تعداد کل توکن ها دقیقاً برابر با **10,000** خواهد بود. با ضرب این عدد استخراج شده در ضریب ثابت مقیاس (**50**)، تعداد تقریبی کلمات متمایز و یکتای این مجموعه برابر با **500,000** کلمه محاسبه می شود.

بخش 3) محاسبه فضای لازم برای شناسه های عددی واژگان

$$\text{Unique Words} = 500,000$$

$$\text{ID Size} = 4 \text{ bytes}$$

$$\text{Total Space} = 500,000 \times 4 = 2,000,000 \text{ bytes}$$

طبق فرضیات سیستم، برای ذخیره سازی و مدیریت هر واژه یکتا در واژه نامه، یک شناسه عددی با طول ثابت **4** بایت اختصاص می یابد. با توجه به محاسبه بخش قبل که تعداد کل کلمات یکتا را **500,000** عدد نشان داد، برای یافتن کل فضای مورد نیاز، این دو مقدار را در هم ضرب می کنیم. نتیجه به دست آمده نشان می دهد که برای نگهداری تمامی شناسه های عددی در این دیکشنری، حداقل به **2,000,000** بایت فضای خام نیاز داریم.

بخش 4) تبدیل بایت به مگابایت

$$\text{Decimal Base } (10^6 \text{ bytes}) \implies \text{Space} \approx \frac{2,000,000}{1,000,000} = 2 \text{ MB}$$

$$\text{Binary Base } (2^{20} \text{ bytes}) \implies \text{Space} = \frac{2,000,000}{1,048,576} \approx 1.91 \text{ MB}$$

برای درک ملموس تر از حجم اشغال شده در سیستم، مقدار خام به دست آمده را از واحد بایت به مگابایت تبدیل می کنیم. اگر طبق محاسبات دهدهی (رایج در بازار تجهیزات ذخیره سازی) هر مگابایت را معادل **1,000,000** بایت در نظر بگیریم، فضای مورد نیاز دقیقاً **2** مگابایت خواهد بود. اما اگر از مبنای دقیق تر و دودویی معماری رایانه استفاده کنیم که در آن هر مگابایت معادل **1,048,576** بایت است، حجم اشغالی تقریباً **1.91** مگابایت محاسبه می شود. در هر دو صورت، سیستم برای این بخش نیازمند فضایی در بازه **1.91** تا **2** مگابایت است.

بخش 5) پاسخ نهایی

$$M = 50 \times \sqrt{100,000,000} = 500,000 \text{ words}$$

$$\text{Total Space} = 500,000 \times 4 = 2,000,000 \text{ bytes}$$

$$\text{Total Space (MB)} \approx 1.91 \text{ MB to } 2 \text{ MB}$$

به عنوان جمع بندی؛ با پردازش **100 میلیون** توکن متنی و اعمال پارامترهای قانون هیپس، اندازه دیکشنری سیستم تقریباً بالغ بر **500,000** کلمه یکتا برآورد می شود.

نتیجه گیری: با تخصیص 4 بایت به هر شناسه کلمه، مجموع حافظه خام مورد نیاز سیستم معادل **2,000,000** بایت است که در مقیاس مگابایت، فضایی معادل **1.91** الی **2** مگابایت از حافظه را اشغال خواهد کرد.

پاسخنامه مسئله 5.22: روابط معکوس در قانون زیپ

بخش 1) مفهوم قانون زیپ

$$\text{Rank} \times \text{Frequency} \approx \text{Constant}$$

طبق قانون زیپ، بین **رتبه یک واژه** و **فرکانس تکرار آن واژه** رابطه ای معکوس وجود دارد. یعنی هرچه رتبه واژه بزرگ تر شود، فرکانس آن کمتر می شود. به بیان ساده، واژه های با رتبه پایین مانند **1**، **2** و **3** بسیار پرتکرار هستند، در حالی که واژه های با رتبه بالا مانند **50**، **100** یا **1,000** کمتر تکرار می شوند. در شکل ساده قانون زیپ، همان طور که در رابطه ریاضی فوق مشاهده می شود، حاصل ضرب رتبه و فرکانس تقریباً مقدار ثابتی است.

بخش 2) محاسبه مقدار ثابت زیپ

$$\begin{aligned}\text{Constant} &= 10 \times 5,000 \\ \text{Constant} &= 50,000 \\ \Rightarrow \text{Rank} \times \text{Frequency} &\approx 50,000\end{aligned}$$

در صورت مسئله گفته شده است که واژه ای با رتبه **10** دارای فرکانس **5,000** است. بنابراین مقدار ثابت زیپ از ضرب مستقیم این دو مقدار به دست می آید که برابر با **50,000** خواهد بود. در نتیجه، برای کل این مجموعه متنی، حاصل ضرب رتبه در فرکانس هر واژه باید تقریباً معادل این مقدار ثابت باشد.

بخش 3) الف - محاسبه فرکانس واژه ای با رتبه 50

$$\begin{aligned}50 \times \text{Frequency} &= 50,000 \\ \text{Frequency} &= \frac{50,000}{50} = 1,000\end{aligned}$$

برای یافتن فرکانس مورد انتظار واژه ای که در جایگاه و رتبه **50** قرار دارد، از مقدار ثابت به دست آمده استفاده می کنیم. با قرار دادن رتبه در معادله و تقسیم مقدار ثابت (**50,000**) بر عدد **50**، فرکانس مورد انتظار برای این واژه برابر با **1,000** تکرار استخراج می گردد.

بخش 4) ب - محاسبه رتبه واژه ای با فرکانس 50

$$\text{Rank} \times 50 = 50,000$$

$$\text{Rank} = \frac{50,000}{50} = 1,000$$

اکنون حالت معکوس را در نظر می‌گیریم؛ فرض کنید واژه‌ای در کل مجموعه دارای فرکانس دقیقاً **50** است و می‌خواهیم جایگاه آن را در فهرست واژگان بیابیم. مجدداً با استفاده از معادله زیپ و تقسیم مقدار ثابت بر فرکانس داده‌شده، مشخص می‌شود که این واژه تقریباً در رتبه **1,000** فهرست واژگان قرار خواهد گرفت.

بخش 5) پاسخ نهایی

$$\text{Zipf's Constant} = 10 \times 5,000 = 50,000$$

$$\text{(A) Frequency for Rank 50} = \frac{50,000}{50} = 1,000$$

$$\text{(B) Rank for Frequency 50} = \frac{50,000}{50} = 1,000$$

به‌عنوان جمع‌بندی؛ با استفاده از داده‌های اولیه، مقدار ثابت قانون زیپ برای این پیکره برابر با **50,000** استخراج شد. بر این اساس در بخش الف، فرکانس واژه رتبه **50** معادل **1,000** بار تکرار به دست آمد.

نتیجه‌گیری: در بخش ب نیز رتبه واژه‌ای با فرکانس **50**، دقیقاً جایگاه **1,000** محاسبه گردید که نشان‌دهنده تقارن در این رابطه معکوس است.

پاسخنامه مسئله 5.23: تعیین ثابت تناسب c در مدل زیپ

بخش 1) مدل زیپ و تعریف ثابت c

$$f(r) = \frac{c}{r}$$

$$T = \sum_{r=1}^n \frac{c}{r} = c \sum_{r=1}^n \frac{1}{r}$$

$$c = \frac{T}{\sum_{r=1}^n \frac{1}{r}}$$

در فرم استاندارد قانون زیپ، فرکانس واژه‌ای با رتبه **r** (که از **1** تا **n** تغییر می‌کند) به صورت کسر **c / r** مدل می‌شود. در این رابطه **n** نمایانگر تعداد واژه‌های یکتا و **c** ثابت تناسب است. اگر تمامی فرکانس‌های به دست آمده را با یکدیگر جمع کنیم، حاصل آن باید دقیقاً معادل با کل توکن‌های موجود در پیکره (**T**) باشد. با فاکتورگیری ثابت **c** از عبارت سیگما، فرمول نهایی برای استخراج این ثابت تناسب به دست می‌آید که معادل تقسیم کل توکن‌ها بر مجموع معکوس رتبه‌ها است.

بخش 2) داده‌های مسئله و تقریب سری هارمونیک

$$n = 1,000$$

$$T = 100,000$$

$$\sum_{i=1}^n \frac{1}{i} \approx \ln(n) + 0.577$$

طبق مفروضات مسئله، سیستم دارای **1,000** واژه یکتا و مجموعاً **100,000** توکن است. برای محاسبه مخرج کسر (مجموع معکوس رتبه‌ها)، به دلیل طولانی بودن محاسبه مستقیم، از تقریب ریاضی «سری هارمونیک» استفاده می‌شود. در این تقریب، حاصل‌جمع با استفاده از لگاریتم طبیعی تعداد واژه‌های یکتا ($\ln(n)$) به علاوه عدد **0.577** (که معادل تقریب ثابت اوایلر-ماشرونی است) تخمین زده می‌شود.

بخش 3) محاسبه مجموع سری هارمونیک

$$\ln(1,000) \approx 6.907$$

$$\sum_{i=1}^{1,000} \frac{1}{i} \approx 6.907 + 0.577 = 7.484$$

در این مرحله، مقادیر عددی را در فرمول تقریب هارمونیک اعمال می‌کنیم. ابتدا لگاریتم طبیعی عدد **1,000** محاسبه می‌شود که تقریباً برابر با **6.907** است. با افزودن ثابت اوایلر-ماشرونی به این مقدار، مجموع معکوس رتبه‌ها برای این مجموعه هزار کلمه‌ای، برابر با عدد **7.484** استخراج می‌گردد.

بخش 4) محاسبه ثابت تناسب c

$$c = \frac{T}{\sum_{r=1}^n \frac{1}{r}}$$

$$c \approx \frac{100,000}{7.484} \approx 13,365$$

اکنون با در اختیار داشتن تعداد کل توکن‌ها (**100,000**) و مقدار مخرج کسر (**7.484**)، می‌توانیم معادله اولیه را تکمیل کنیم. با تقسیم این دو مقدار بر یکدیگر، ثابت تناسب برای این مجموعه با دقت مناسبی در حدود عدد **13,365** محاسبه می‌شود.

بخش 5) پاسخ نهایی و تفسیر مفهومی

$$T = c \sum_{r=1}^{1,000} \frac{1}{r}$$

$$c \approx \frac{100,000}{7.484} \approx 13,365$$

به‌عنوان جمع‌بندی؛ با استفاده از مدل پایه زیپف و اعمال تقریب سری هارمونیک، مقدار ثابت تناسب (**c**) در این پیکره متنی تقریباً **13,365** به دست آمد. از نظر مفهومی، این ثابت معادل فراوانی واژه‌ای است که در رتبه نخست قرار دارد.

نتیجه‌گیری: واژه با رتبه 1 حدود 13,365 بار در متن تکرار می‌شود. پیرو قانون زیپف، واژه دوم نیمی از این مقدار و واژه دهم حدود یک‌دهم آن تکرار خواهند شد؛ که این رفتار کاهشی کاملاً با توزیع استاندارد متون طبیعی همخوانی دارد.

پاسخنامه مسئله 5.24: ارزیابی ساختارهای ذخیره‌سازی آرایه با طول ثابت در برابر رشته متصل

بخش اول: معرفی مدل آرایه با طول ثابت (Fixed-Length Array)

در این مدل، برای ذخیره‌سازی هر کلمه در واژه‌نامه، یک جایگاه با اندازه ثابت و از پیش تعیین‌شده در نظر گرفته می‌شود. به عنوان مثال، اگر حداکثر طول کلمات در یک مجموعه 20 کاراکتر باشد، برای تمام کلمات، اعم از آن‌هایی که 2 کاراکتر دارند یا آن‌هایی که 20 کاراکتر دارند، دقیقاً 20 بایت فضا رزرو می‌شود. در این ساختار، دسترسی به هر کلمه بسیار ساده است. اگر اندیس کلمه را داشته باشیم، با ضرب کردن شماره اندیس در اندازه ثابت هر بلاک، می‌توانیم موقعیت شروع کلمه را در حافظه مستقیماً محاسبه کنیم. این روش از نظر سادگی پیاده‌سازی و سرعت دسترسی، عملکرد بسیار مطلوبی دارد.

بخش دوم: پدیده هدررفت حافظه (Internal Fragmentation) در مدل آرایه‌ای

هدررفت حافظه داخلی یا همان **Internal Fragmentation**، به فضاهایی گفته می‌شود که درون یک واحد حافظه رزرو شده باقی می‌مانند و بدون استفاده می‌مانند. در مدل آرایه با طول ثابت، این پدیده به شدت مشاهده می‌شود. وقتی برای کلمه‌ای با طول کم، مثلاً 3 کاراکتر، فضایی به اندازه بزرگترین کلمه ممکن، مثلاً 20 کاراکتر، اختصاص می‌دهیم، عملاً مقدار زیر از فضای اختصاص‌یافته بلااستفاده می‌ماند.

$$20 - 3 = 17 \text{ bytes}$$

چون این فضای خالی نه به کلمه قبلی تعلق دارد و نه به کلمه بعدی، و نه می‌تواند برای ذخیره‌سازی داده دیگری استفاده شود، به عنوان اتلاف حافظه داخلی شناخته می‌شود. هرچه تفاوت بین میانگین طول کلمات و حداکثر طول کلمات بیشتر باشد، این میزان هدررفت در مدل آرایه‌ای بیشتر خواهد بود.

بخش سوم: معرفی مدل رشته متصل (Dictionary as a String)

در مدل رشته متصل که گاهی به آن روش دیکشنری یکپارچه نیز می‌گویند، کلمات بدون هیچ‌گونه فاصله یا فضای رزرو شده اضافی، پشت سر هم در یک رشته طولانی قرار می‌گیرند. در این ساختار، هر کلمه دقیقاً به اندازه تعداد کاراکترهای خودش فضا اشغال می‌کند.

برای مدیریت این ساختار و امکان دسترسی به کلمات، از یک آرایه کمکی به نام آرایه اشاره‌گرها (**Offset Array**) استفاده می‌شود. این آرایه محل شروع هر کلمه را در رشته اصلی ذخیره می‌کند. در واقع، سیستم به جای رزرو کردن فضای اضافه برای هر کلمه، هزینه‌ای را برای ذخیره اشاره‌گرها پرداخت می‌کند تا بتواند به کلمات دسترسی پیدا کند.

بخش چهارم: مقایسه پدیده هدررفت حافظه در دو مدل

در مدل رشته متصل، هدررفت حافظه داخلی عملاً نزدیک به صفر است. چرا که هیچ‌گونه فضای خالی برای «تطبیق با بزرگترین کلمه» رزرو نمی‌شود. هر بایت از حافظه که در این روش اشغال شده است، حاوی بخشی از یک کلمه واقعی است.

در مقابل، مدل آرایه با طول ثابت، علیرغم مزیت سرعت، دچار مشکل اساسی در کارایی حافظه است. اگر توزیع طول کلمات در واژه‌نامه پراکنده باشد، یعنی ترکیبی از کلمات بسیار کوتاه و بسیار بلند داشته باشیم، مدل آرایه‌ای بخش بزرگی از حافظه اصلی سیستم را با «فضاهای خالی بی‌فایده» پر می‌کند.

بنابراین، در مدل آرایه با طول ثابت، هزینه پردازشی کم است، زیرا دسترسی مستقیم انجام می‌شود، اما هزینه حافظه به دلیل **Internal Fragmentation** بالا است. در مدل رشته متصل، هزینه حافظه بهینه است، زیرا **Internal Fragmentation** حذف می‌شود، اما هزینه پردازشی کمی افزایش می‌یابد، زیرا برای یافتن محل کلمه نیاز به خواندن آرایه اشاره‌گرها وجود دارد.

انتخاب میان این دو مدل، یک بده‌بستان (**Trade-off**) کلاسیک بین سرعت دسترسی و بهینگی مصرف حافظه در سیستم‌های بازیابی اطلاعات است.

پاسخنامه مسئله 5.25: ذخیره‌سازی بلوکی واژه‌های واژه‌نامه

بخش اول: روش ذخیره‌سازی بلوکی چیست؟

در روش ذخیره‌سازی عادی، هر واژه در واژه‌نامه **Dictionary** به صورت کامل ذخیره می‌شود. در مقابل، در روش **Blocked Storage** واژه‌ها در بلوک‌هایی با اندازه ثابت دسته‌بندی می‌شوند. در این ساختار، واژه‌ها به صورت گروه‌هایی شامل **k** واژه مرتب می‌شوند.

در هر بلوک، تنها واژه اول به طور کامل ذخیره می‌شود و برای واژه‌های بعدی، به جای ذخیره کل رشته، تنها **طول پیشوند مشترک** و همچنین بخش متمایزکننده واژه ذخیره می‌شود. این کار باعث حذف تکرارهای غیرضروری در رشته‌ها شده و فضای ذخیره‌سازی مورد نیاز را کاهش می‌دهد.

به عنوان نمونه، اگر چهار واژه زیر در یک بلوک قرار گیرند:

automation automaton automatic automata

در این حالت، پیشوند مشترک **auto** در تمام واژه‌ها تکرار می‌شود. در روش ذخیره‌سازی بلوکی، به جای ذخیره چندباره این پیشوند، تنها واژه اول به طور کامل نگهداری می‌شود و واژه‌های بعدی با استفاده از اطلاعات اختلافی نسبت به واژه قبلی بازسازی می‌شوند.

بخش دوم: مزیت اصلی (Memory Reduction)

مهم‌ترین مزیت این روش، کاهش قابل توجه فضای مورد نیاز برای ذخیره‌سازی واژه‌نامه است. در یک واژه‌نامه بزرگ که شامل میلیون‌ها واژه می‌باشد، بسیاری از واژه‌ها دارای پیشوندهای مشترک هستند. ذخیره‌سازی کامل هر واژه باعث می‌شود این پیشوندها بارها در ساختار داده تکرار شوند.

در روش **Blocked Dictionary Storage** به جای ذخیره رشته کامل، تنها **طول پیشوند مشترک** و **بخش متفاوت واژه** ذخیره می‌شود. این کار باعث نوعی فشرده‌سازی طبیعی در ساختار واژه‌نامه می‌شود و در نتیجه حجم کل واژه‌نامه به طور قابل توجهی کاهش می‌یابد.

کاهش اندازه واژه‌نامه این امکان را فراهم می‌کند که کل ساختار واژه‌نامه در حافظه اصلی (**RAM**) قرار گیرد. قرارگیری کامل واژه‌نامه در حافظه اصلی معمولاً باعث کاهش زمان دسترسی و افزایش سرعت کلی سیستم بازیابی اطلاعات می‌شود.

بخش سوم: تأثیر بر فرآیند جست‌وجو

استفاده از روش ذخیره‌سازی بلوکی باعث ایجاد یک فرآیند جست‌وجوی دو مرحله‌ای در واژه‌نامه می‌شود.

مرحله اول: جست‌وجوی شاخص (Index Search)

در این مرحله، سیستم تنها روی واژه‌های اول هر بلوک جست‌وجو انجام می‌دهد. از آنجا که این واژه‌ها به صورت مرتب ذخیره شده‌اند، می‌توان از الگوریتم **Binary Search** برای یافتن بلوک مناسب استفاده کرد.

مرحله دوم: جست‌وجوی درون بلوک (Block Traversal)

پس از مشخص شدن بلوک هدف، سیستم باید واژه‌های داخل آن بلوک را به صورت ترتیبی بررسی کند. در این مرحله، با استفاده از طول پیشوند مشترک و بخش متمایزکننده ذخیره شده، واژه‌ها بازسازی می‌شوند تا واژه مورد نظر کاربر شناسایی شود.

جمع‌بندی نهایی

در حالتی که اندازه بلوک برابر با $k = 4$ در نظر گرفته شود، روش ذخیره‌سازی بلوکی یک موازنه میان مصرف حافظه و پیچیدگی جست‌وجو ایجاد می‌کند.

1. کاهش فضای حافظه به دلیل حذف پیشوندهای تکراری در واژه‌ها.

2. افزایش جزئی پیچیدگی جست‌وجو به دلیل نیاز به پیمایش خطی در داخل هر بلوک.

با این حال، در بسیاری از سیستم‌های بازیابی اطلاعات، کاهش اندازه واژه‌نامه باعث می‌شود کل ساختار در حافظه **RAM** قرار گیرد و در نتیجه عملکرد کلی سیستم از نظر سرعت بازیابی بهبود قابل توجهی پیدا کند.

پاسخنامه مسئله 5.26: ایده اصلی و مراحل کدگذاری کدهای گاما (Gamma Codes)

بخش اول: ایده اصلی کدهای گاما

ایده اصلی کدهای گاما (**Gamma Codes**) برای فشردن اعداد صحیح، جداسازی یک عدد به دو بخش مجزا است تا بتوان آن را به شکلی ذخیره کرد که در هنگام خواندن، مرز بین اعداد به‌طور دقیق مشخص باشد. این ویژگی باعث می‌شود این روش دارای خاصیت **Prefix-free** باشد.

در کدهای طول‌متغیر (**Variable-Length Codes**) هدف این است که اعداد کوچک‌تر با تعداد بیت کمتری نمایش داده شوند. در این روش هر عدد به دو بخش تقسیم می‌شود:

1. بخش یکانی (**Unary**) که طول نمایش دودویی عدد را مشخص می‌کند.

2. بخش دودویی (**Binary**) که مقدار واقعی عدد را نشان می‌دهد.

در این ساختار، به جای ذخیره طول عدد با تعداد بیت ثابت، طول آن با چند بیت «یک» و یک «صفر» مشخص می‌شود. این کار باعث می‌شود رمزگشا بتواند مرز بین اعداد را بدون ابهام تشخیص دهد.

بخش دوم: مراحل کدگذاری یک عدد x

برای کدگذاری یک عدد صحیح مثبت **x** در روش کدهای گاما، مراحل زیر انجام می‌شود.

مرحله 1: محاسبه طول (بخش یکانی)

ابتدا عدد **x** را در مبنای دو (**Binary**) می‌نویسیم. اگر نمایش دودویی این عدد شامل **n** بیت باشد، آنگاه بخش یکانی باید مقدار **n** را نشان دهد.

این کار با نوشتن **(n - 1)** عدد «یک» و سپس یک «صفر» انجام می‌شود.

$$\text{Unary} = 111...10$$

برای مثال اگر **n = 3** باشد، مقدار **(3 - 1)** برابر **2** است. بنابراین دو بیت «یک» و سپس یک «صفر» نوشته می‌شود:

110

مرحله 2: نوشتن مقدار عددی (بخش دودویی)

در این مرحله مقدار عدد **x** را در مبنای دو می‌نویسیم، اما بیت اول آن حذف می‌شود. دلیل این کار این است که در نمایش دودویی هر عدد مثبت، بیت سمت چپ همیشه **1** است. بنابراین برای صرفه‌جویی در فضای ذخیره‌سازی می‌توان آن را حذف کرد. به این تکنیک، **حذف بیت پیشرو** یا **Leading Bit Removal** گفته می‌شود.

بخش سوم: مثال عملی برای عدد 5

برای درک بهتر، عدد **5** را با استفاده از کد گاما رمزگذاری می‌کنیم.

مرحله 1: نمایش دودویی عدد **5** به صورت زیر است:

101

طول این عدد برابر با **3** بیت است، بنابراین **n = 3**. بخش یکانی برابر خواهد بود با:

110

مرحله 2: در مرحله بعد بیت اول از **101** حذف می‌شود و مقدار زیر باقی می‌ماند:

01

مرحله 3: در نهایت دو بخش به یکدیگر متصل می‌شوند:

11001

جمع‌بندی

کدهای گاما (**Gamma Codes**) چند ویژگی مهم دارند که باعث می‌شود در سیستم‌های بازیابی اطلاعات کاربرد زیادی داشته باشند.

1. اعداد کوچک با تعداد بیت بسیار کمی نمایش داده می‌شوند.

2. ساختار کدها دارای خاصیت **Prefix-free** است، بنابراین هنگام رمزگشایی هیچ ابهامی ایجاد نمی‌شود.

3. در بسیاری از سیستم‌های بازیابی اطلاعات، فاصله بین شناسه اسناد (**Document Gaps**) اعداد کوچکی هستند و استفاده از کد گاما باعث فشرده‌سازی مؤثر این مقادیر می‌شود.

پاسخنامه مسئله 5.27: محاسبه ظرفیت حافظه در مدل رشته‌ای

(Dictionary as a String)

بخش 1) صورت مسئله

$$\begin{aligned}\text{Total Words } (N) &= 50,000 \\ \text{Average Length} &= 8 \text{ chars/bytes} \\ \text{Pointer Size} &= 4 \text{ bytes} \\ \text{Frequency Size} &= 4 \text{ bytes}\end{aligned}$$

در این مسئله، یک واژه‌نامه شامل 50,000 واژه است و میانگین طول هر واژه برابر با 8 کاراکتر (معادل 8 بایت) در نظر گرفته شده است. در مدل **Dictionary as a String** کلمات به صورت پشت سر هم در یک رشته بزرگ ذخیره می‌شوند. علاوه بر رشته اصلی، برای هر واژه دو فیلد اضافی نیز ذخیره می‌شود: یک اشاره‌گر 4 بایتی که محل شروع واژه را در رشته اصلی مشخص می‌کند و یک فیلد فرکانس 4 بایتی که تعداد وقوع واژه در مجموعه اسناد را نگه می‌دارد. هدف محاسبه کل فضای حافظه مصرفی برای ذخیره این واژه‌نامه است.

بخش 2) محاسبه حافظه مورد نیاز برای رشته اصلی واژه‌ها

$$\text{String Memory} = 50,000 \times 8 = 400,000 \text{ bytes}$$

در این مدل، تمام واژه‌ها پشت سر هم در یک رشته ذخیره می‌شوند. بنابراین فضای مورد نیاز برای رشته اصلی برابر است با تعداد واژه‌ها ضربدر میانگین طول هر واژه. با فرض اینکه هر کاراکتر 1 بایت فضا اشغال کند، حافظه رشته اصلی از ضرب 50,000 واژه در میانگین طول 8 کاراکتر به دست می‌آید که معادل 400,000 بایت خواهد بود.

بخش 3) محاسبه حافظه مورد نیاز برای اشاره‌گرها

$$\text{Pointer Memory} = 50,000 \times 4 = 200,000 \text{ bytes}$$

برای هر واژه یک اشاره‌گر مجزا در نظر گرفته شده است که محل شروع آن واژه در رشته اصلی را مشخص می‌کند. اندازه هر اشاره‌گر برابر با 4 بایت است و تعداد اشاره‌گرها نیز دقیقاً با تعداد کل واژه‌ها یعنی 50,000 برابر است. با ضرب این دو مقدار، حافظه مورد نیاز برای کل ساختار اشاره‌گرها، 200,000 بایت استخراج می‌گردد.

بخش 4) محاسبه حافظه مورد نیاز برای فیلد فرکانس

$$\text{Frequency Memory} = 50,000 \times 4 = 200,000 \text{ bytes}$$

علاوه بر اشاره‌گر، برای هر واژه یک فیلد فرکانس نیز به‌منظور نگهداری تعداد وقوع آن در مجموعه اسناد ذخیره می‌شود. اندازه هر فیلد فرکانس نیز برابر با **4** بایت است. مشابه محاسبه مرحله قبل، با ضرب تعداد **50,000** واژه در حجم این فیلد، فضای لازم برای ذخیره فرکانس‌ها نیز معادل **200,000** بایت محاسبه می‌شود.

بخش 5) محاسبه حافظه کل و پاسخ نهایی

$$\begin{aligned}\text{Total Memory} &= 400,000 + 200,000 + 200,000 = 800,000 \text{ bytes} \\ \text{Total Memory (KB)} &\approx \frac{800,000}{1,024} \approx 781 \text{ KB} \\ \text{Total Memory (MB)} &\approx \frac{800,000}{1,048,576} \approx 0.76 \text{ MB}\end{aligned}$$

اکنون سه بخش حافظه استخراج‌شده با یکدیگر جمع می‌شوند: حافظه رشته اصلی واژه‌ها (**400,000** بایت)، حافظه اشاره‌گرها (**200,000** بایت) و حافظه فیلدهای فرکانس (**200,000** بایت). بنابراین مجموع حافظه کل معادل **800,000** بایت خواهد بود. با تبدیل این مقدار بر مبنای توان‌های 2 (دودویی)، حجم اشغالی تقریباً معادل **781** کیلوبایت یا **0.76** مگابایت می‌شود.

نتیجه‌گیری: کل حافظه مورد نیاز برای ذخیره این دیکشنری در مدل Dictionary as a String برابر با 800,000 بایت (حدود 0.76 MB) است.

پاسخنامه مسئله 5.28: بهینه‌سازی حافظه با استفاده از تکنیک بلوک بندی (Blocking)

بخش 1) صورت مسئله و مفروضات

در این مسئله، یک دیکشنری شامل **50,000** واژه با میانگین طول **8** کاراکتر در نظر گرفته شده است. در ساختار اولیه، هر واژه دارای یک اشاره‌گر **4** بایتی و یک فیلد فرکانس **4** بایتی است. هدف، اعمال تکنیک بلوک‌بندی با گام **4** است؛ به این معنا که برای هر **4** واژه، تنها یک اشاره‌گر در حافظه ذخیره می‌شود تا میزان فضای صرفه‌جویی‌شده و درصد آن نسبت به حافظه کل اولیه محاسبه گردد.

بخش 2) محاسبه حافظه مصرفی در ساختار اولیه

$$\text{Initial Memory} = 400,000 + 200,000 + 200,000 = 800,000 \text{ bytes}$$

در ساختار اولیه پیش از اعمال هرگونه بهینه‌سازی، مجموع حافظه مصرفی از سه بخش تشکیل شده است: حافظه رشته اصلی واژه‌ها (**400,000** بایت)، حافظه اشاره‌گرها برای هر واژه (**200,000** بایت) و حافظه فیلدهای فرکانس (**200,000** بایت). با جمع کردن این مقادیر، مصرف کل حافظه در حالت استاندارد **800,000** بایت خواهد بود.

بخش 3) تعیین تعداد بلوک‌ها

$$\text{Blocks Count} = \frac{50,000}{4} = 12,500 \text{ blocks}$$

با اعمال تکنیک بلوک‌بندی با گام 4، سیستم به جای اختصاص یک اشاره‌گر برای هر واژه، به ازای هر دسته 4 تایی از واژه‌ها تنها یک اشاره‌گر در ساختار خود ذخیره می‌کند. با انجام این تقسیم، مشخص می‌شود که در معماری جدید به جای 50,000 اشاره‌گر، تنها به 12,500 اشاره‌گر (بلوک) نیاز داریم.

بخش 4) محاسبه حافظه جدید اشاره‌گرها و حافظه کل

$$\text{New Pointers Memory} = 12,500 \times 4 = 50,000 \text{ bytes}$$

$$\text{New Total Memory} = 400,000 + 50,000 + 200,000 = 650,000 \text{ bytes}$$

با کاهش تعداد اشاره‌گرها به 12,500 عدد (که هر کدام 4 بایت فضا اشغال می‌کنند)، حافظه جدید برای بخش اشاره‌گرها به شدت افت کرده و به 50,000 بایت می‌رسد. اکنون با جمع کردن این مقدار با حجم رشته اصلی و فیلدهای فرکانس (که در این تکنیک بدون تغییر باقی می‌مانند)، حافظه کل جدید سیستم برابر با 650,000 بایت محاسبه می‌گردد.

بخش 5) محاسبه میزان صرفه‌جویی و درصد آن

$$\text{Memory Saved} = 800,000 - 650,000 = 150,000 \text{ bytes}$$

$$\text{Percentage Saved} = \left(\frac{150,000}{800,000} \right) \times 100 = 18.75\%$$

میزان صرفه‌جویی مطلق در حافظه از تفاضل حافظه اولیه و حافظه جدید به دست می‌آید که این رقم برابر با 150,000 بایت است. با تقسیم این مقدار بر کل حافظه اولیه و ضرب آن در عدد 100، استخراج می‌شود که این استراتژی فشرده‌سازی باعث کاهش 18.75 درصدی در هدررفت حافظه شده است.

بخش 6) پاسخ نهایی

به‌عنوان جمع‌بندی؛ پیاده‌سازی تکنیک بلوک‌بندی موجب می‌شود تا هزینه سربرار اشاره‌گرها به شکل چشمگیری کاهش یابد.

نتیجه‌گیری: با اعمال تکنیک بلوک‌بندی با گام 4، حافظه مورد نیاز دیکشنری از 800,000 بایت به 650,000 بایت کاهش می‌یابد که این امر منجر به صرفه‌جویی 150,000 بایتی (معادل 18.75%) در کل حافظه می‌گردد.

پاسخنامه مسئله 5.29: کدگذاری پیشوند (Front Coding)

بخش 1) مفهوم کدگذاری پیشوند

کدگذاری پیشوند یا **Front Coding** یکی از روش‌های متداول برای فشرده‌سازی ساختار واژه‌نامه در سیستم‌های بازیابی اطلاعات (**Information Retrieval**) است. ایده اصلی این روش بر این مشاهده استوار است که در یک دیکشنری مرتب‌شده بر اساس حروف الفبا، کلمات مجاور معمولاً دارای پیشوندهای مشترک طولانی هستند. در این تکنیک، اولین کلمه هر بلوک به صورت کامل ذخیره می‌شود. برای کلمات بعدی، به جای ذخیره کامل واژه، تنها دو بخش ذخیره می‌گردد: نخست، طول پیشوند مشترک با کلمه قبلی و سپس بخش باقی‌مانده کلمه که به آن پسوند (**Suffix**) گفته می‌شود. از آنجا که به جای ذخیره کامل هر کلمه، تنها تفاوت آن با کلمه قبلی ثبت می‌شود، بخش بزرگی از کاراکترهای تکراری حذف شده و این روش می‌تواند به شکل قابل‌توجهی مصرف حافظه دیکشنری را کاهش دهد.

بخش 2) تحلیل گام‌به‌گام واژگان

Words : automata → automate → automatic → automation
Word 1 (automata) ⇒ Prefix Length = 0 (No prior word)
Word 2 (automate) ⇒ Shared = "automat" ⇒ Prefix = 7, Suffix = "e"
Word 3 (automatic) ⇒ Shared = "automat" ⇒ Prefix = 7, Suffix = "ic"
Word 4 (automation) ⇒ Shared = "automati" ⇒ Prefix = 8, Suffix = "on"

بر اساس لیست کلمات داده‌شده، هر واژه با واژه قبل از خود مقایسه می‌شود تا طول پیشوند مشترک آن‌ها تعیین گردد. از آنجا که **automata** اولین عضو لیست است، واژه قبلی برای مقایسه وجود ندارد؛ بنابراین پیشوند آن 0 بوده و کل کلمه ذخیره می‌شود. واژه دوم (**automate**) در مقایسه با کلمه قبل، در عبارت **automat** مشترک است (طول 7) و پسوند باقی‌مانده حرف **e** خواهد بود. واژه سوم (**automatic**) نیز با واژه قبل از خود دقیقاً 7 کاراکتر مشترک دارد و پسوند آن **ic** است. در نهایت، واژه چهارم (**automation**) با واژه قبلی خود در عبارت **automati** اشتراک دارد، بنابراین پیشوند مشترک به طول 8 افزایش یافته و تنها پسوند **on** ذخیره می‌گردد.

بخش 3) نمایش ساختار فشرده نهایی

0	automata
7	e
7	ic
8	on

بر اساس محاسبات انجام‌شده، نمایش فشرده واژگان به صورت ساختار (طول پیشوند مشترک – باقی‌مانده واژه) در کادر بالا خلاصه شده است.

نتیجه‌گیری: در این ساختار، واژه اول به طور کامل ذخیره شده و واژه‌های بعدی تنها با استفاده از طول پیشوند مشترک و بخش باقی‌مانده بازسازی می‌شوند. این روش موجب حذف بخش‌های تکراری در واژگان هم‌ریشه و در نتیجه کاهش محسوس مصرف حافظه در ذخیره‌سازی دیکشنری می‌شود.

پاسخنامه مسئله 5.30: تحلیل موازنه کارایی میان اندازه بلوک، نرخ فشردگی و زمان پاسخ‌گویی

بخش 1) صورت مسئله و مفاهیم پایه

در طراحی واژه‌نامه‌ها (دیکشنری‌ها)، پارامتر k که نشان‌دهنده اندازه بلوک است، نقش کلیدی در معماری سیستم بازی می‌کند. اندازه بلوک مشخص می‌کند که به ازای هر اشاره‌گر، چند واژه در یک واحد ذخیره‌سازی قرار می‌گیرند. افزایش این مقدار به معنای ذخیره تعداد واژگان بیشتر در هر بلوک و کاهش تعداد اشاره‌گرها است. مسئله اصلی در اینجا، بررسی بده‌بستان (**Trade-off**) میان دو شاخص حیاتی است: **نرخ فشردگی و زمان پاسخ‌گویی** یا همان **Latency**.

بخش 2) اثر افزایش اندازه بلوک بر نرخ فشردگی

$$\text{Number of Pointers} \approx \frac{\text{Total Words}}{k}$$

سربار اصلی در ساختار دیکشنری، مربوط به **Metadata** یا همان اشاره‌گرهایی است که محل شروع واژگان را تعیین می‌کنند. همان‌طور که در رابطه ریاضی فوق نشان داده شده است، در ساختار بلوکی، با افزایش مقدار k ، تعداد اشاره‌گرهای کل سیستم به صورت معکوس کاهش می‌یابد. هرچه مقدار k بزرگ‌تر شود، بخش بزرگ‌تری از حافظه به جای ذخیره اشاره‌گرها (که نوعی سربار هستند)، صرف ذخیره خود داده‌ها می‌شود. بنابراین، افزایش اندازه بلوک به‌طور مستقیم منجر به بهبود نرخ فشردگی و کاهش فضای اشغال‌شده توسط ساختارهای کمکی می‌گردد.

بخش 3) اثر افزایش اندازه بلوک بر زمان پاسخ‌گویی (Latency)

در مقابل فشردگی، بزرگ شدن بلوک اثر منفی بر سرعت بازیابی دارد. فرآیند جستجو در این ساختار معمولاً دو مرحله‌ای است: ابتدا یافتن بلوک مربوطه و سپس پیمایش ترتیبی درون آن بلوک برای یافتن واژه هدف. وقتی k بزرگ باشد، طول هر بلوک نیز به نسبت افزایش می‌یابد. در نتیجه، پس از رسیدن به آدرس بلوک، سیستم باید تعداد واژگان بیشتری را بررسی کند تا به واژه دقیق دست یابد. این افزایش تعداد عملیات پیمایش (**Sequential Scan**)، مستقیماً باعث افزایش زمان پاسخ‌گویی یا همان **Latency** می‌شود.

بخش 4) تبیین موازنه (Trade-off)

طراح سیستم با یک تقابل کلاسیک روبروست:

- اگر k کوچک باشد: تعداد بلوک‌ها و اشاره‌گرها زیاد شده و حافظه بیشتری مصرف می‌شود، اما جستجوی درون‌بلوکی سریع‌تر انجام می‌شود.
- اگر k بزرگ باشد: تعداد اشاره‌گرها کم شده و حافظه بهینه می‌شود، اما زمان لازم برای پیمایش درون بلوک افزایش می‌یابد.

بخش 5) شرایط انتخاب بلوک کوچک

انتخاب مقدار کوچک برای k معمولاً در شرایط زیر بهینه است:

۱. اهمیت بالای سرعت پاسخ‌گویی: در موتورهای جستجوی با ترافیک بالا یا سامانه‌های **Real-time**، کاهش زمان جستجو اولویت اول است.

۲. محدود نبودن حافظه: زمانی که سیستم با مشکل کمبود حافظه روبرو نیست و طراح می‌تواند فضای اضافه برای اشاره‌گرها را بپذیرد.

۳. تعدد بالای پرس‌وجوها: در سیستم‌هایی که نرخ درخواست‌ها بسیار زیاد است، بهینه‌سازی زمان هر پرس‌وجو تأثیر تجمعی بسیار مثبتی دارد.

۴. هزینه بالای پیمایش: اگر الگوریتم مقایسه واژگان داخل بلوک (مانند بازسازی کلمات در کدگذاری پیشوند) از نظر محاسباتی سنگین باشد، کوچک کردن بلوک ضرورت می‌یابد.

بخش 6) پاسخ نهایی

موازنه میان **k**، فشردگی و **Latency** یک تصمیم‌گیری استراتژیک است. افزایش **k** نرخ فشردگی را بهبود می‌دهد اما با افزایش زمان جستجو، سیستم را کندتر می‌کند.

نتیجه‌گیری: انتخاب مقدار کوچک برای **k** زمانی ترجیح داده می‌شود که کاهش زمان پاسخ‌گویی (**Latency**) و پشتیبانی از حجم بالای جستجوها اهمیت بیشتری از صرفه‌جویی در مصرف حافظه داشته باشد.

پاسخنامه مسئله 5.31: مفهوم ذخیره‌سازی تفاضلی در لیست‌های ارجاع

بخش 1) صورت مسئله

در سیستم‌های بازیابی اطلاعات (**Information Retrieval**) برای هر واژه مجموعه‌ای از شناسه‌های اسناد ذخیره می‌شود که نشان می‌دهد آن واژه در کدام اسناد ظاهر شده است. این ساختار با نام **Posting List** شناخته می‌شود و هر عنصر آن یک شناسه سند یا **DocID** است. یک روش ساده ذخیره‌سازی آن است که شناسه کامل هر سند مستقیماً در لیست ذخیره شود. با این حال در سیستم‌های واقعی معمولاً از تکنیکی به نام **Gap Encoding** یا «ذخیره‌سازی تفاضلی» استفاده می‌شود. در این روش به جای ذخیره مستقیم شناسه اسناد، اختلاف عددی میان شناسه‌های متوالی ذخیره می‌گردد. هدف اصلی این روش کاهش اندازه اعداد ذخیره‌شده و فراهم کردن امکان فشردگی مؤثرتر است.

بخش 2) مرتب بودن شناسه اسناد در Posting List

$$\begin{aligned}\text{DocIDs} &= [10, 23, 27, 40, 55] \\ \text{Gap}_1 &= 10 \\ \text{Gap}_2 &= 23 - 10 = 13 \\ \text{Gap}_3 &= 27 - 23 = 4 \\ \text{Gap}_4 &= 40 - 27 = 13 \\ \text{Gap}_5 &= 55 - 40 = 15 \\ \text{Gaps List} &= [10, 13, 4, 13, 15]\end{aligned}$$

در یک **Posting List**، شناسه اسناد همواره به صورت صعودی مرتب و ذخیره می‌شوند. همان‌طور که در معادلات بالا مشاهده می‌کنید، فرض کرده‌ایم یک واژه در اسناد **10** تا **55** ظاهر شده باشد. اگر این مقادیر به صورت مستقیم ذخیره شوند، هر شناسه باید به طور کامل در حافظه بنشیند. اما در روش ذخیره‌سازی تفاضلی، با محاسبه اختلاف میان هر دو شناسه متوالی، اعداد بسیار کوچک‌تری (مانند **13** و **4**) جایگزین شناسه‌های اصلی می‌شوند. این ویژگی که مقادیر تفاضلی نسبت به شناسه‌های اصلی به مراتب کوچک‌ترند، پایه اصلی استفاده از **Gap Encoding** را تشکیل می‌دهد.

بخش 3) دلیل استفاده از Gap به جای DocID

در مجموعه‌های بزرگ اسناد، مقادیر **DocID** ممکن است بسیار بزرگ باشند. برای مثال اگر تعداد اسناد یک میلیون باشد، شناسه‌ها ممکن است به مقادیری در حدود **1,000,000** برسند. تخصیص حافظه برای نمایش مستقیم چنین اعدادی نیازمند تعداد بیت نسبتاً زیادی (حداقل 20 بیت) است. اما اختلاف میان شناسه‌های متوالی در یک لیست مرتب، معمولاً بسیار کوچک‌تر است. به عنوان نمونه ممکن است یک واژه در اسناد نزدیک به هم ظاهر شود و فاصله میان شناسه‌ها تنها چند واحد باشد. هنگامی که این اختلاف‌ها ذخیره شوند، بزرگی اعداد به طور چشمگیری افت کرده و در نتیجه تعداد بیت‌های مورد نیاز برای ذخیره آن‌ها کاهش می‌یابد. در مقیاس سیستم‌های واقعی که شامل میلیون‌ها واژه و میلیارد‌ها ارجاع هستند، همین کاهش در اندازه اعداد به صرفه‌جویی عظیمی در مصرف دیسک و رم منجر می‌شود.

بخش 4) ارتباط Gap Encoding با کدهای طول متغیر

کدهای فشرده‌سازی با طول متغیر (مانند **Variable-Byte Coding** و **Gamma Coding**) برای نمایش اعداد کوچک به شدت کارآمد هستند؛ به طوری که در این روش‌ها هرچه مقدار عدد کوچک‌تر باشد، تعداد بیت‌های مصرفی نیز به شدت کاهش می‌یابد. اگر شناسه‌های بزرگ اسناد مستقیماً به این روش‌ها داده شوند، فشرده‌سازی مؤثری رخ نمی‌دهد. اما هنگامی که شناسه‌ها ابتدا به **Gap** تبدیل می‌شوند و به مقادیر کوچکی تنزل پیدا می‌کنند، این کدهای طول متغیر می‌توانند با حداقل بیتِ ممکن آن‌ها را فشرده سازند.

بخش 5) پاسخ نهایی

در لیست‌های ارجاع، شناسه اسناد به دلیل مرتب‌سازی صعودی، اختلاف‌های متوالی بسیار کوچک‌تری نسبت به مقدار واقعی خود تولید می‌کنند.

نتیجه‌گیری: ترکیب تکنیک Gap Encoding (که اعداد را کوچک می‌کند) با Variable-Length Codes (که اعداد کوچک را با کمترین بیت کد می‌کند)، کارآمدترین و مهم‌ترین استراتژی فشرده‌سازی در ساختار نمایه‌های معکوس (Inverted Index) به شمار می‌رود.

پاسخنامه مسئله 5.32: کدگذاری بایت متغیر (Variable Byte Encoding)

بخش 1) تعریف روش Variable Byte

در روش **Variable Byte Encoding** هر عدد صحیح به یک یا چند بایت هشت‌بیتی تبدیل می‌شود. در این ساختار، هر بایت شامل دو بخش است: بیت پرارزش که به آن **Continuation Bit** گفته می‌شود و **7** بیت دیگر که برای ذخیره داده استفاده می‌شوند. قاعده تعیین این بیت کنترل‌کننده به این صورت است که اگر بایت مورد نظر آخرین بایت نمایش عدد باشد، این بیت

برابر **1** قرار می‌گیرد؛ در غیر این صورت مقدار آن **0** خواهد بود. بنابراین هر بایت حداکثر **7** بیت از مقدار عدد را نگهداری می‌کند و اعداد بزرگ‌تر با استفاده از چند بایت متوالی نمایش داده می‌شوند. برای کدگذاری یک عدد، ابتدا نمایش دودویی آن نوشته می‌شود، سپس این نمایش از سمت راست به بلوک‌های **7** بیتی تقسیم می‌گردد و در نهایت برای هر بلوک یک بیت ادامه در ابتدای آن قرار می‌گیرد.

بخش 2) کدگذاری عدد 82

$$82_{10} = 1010010_2$$
$$VB(82) = 11010010$$

برای کدگذاری عدد **82**، ابتدا آن را به مبنای دو تبدیل می‌کنیم. همان‌طور که در معادلات بالا مشخص است، این مقدار دقیقاً **7** بیت دارد، بنابراین به صورت کامل در یک بلوک داده قرار می‌گیرد. از آنجا که این تنها بایت برای نمایش عدد است، مقدار **Continuation Bit** در ابتدای آن برابر **1** خواهد بود. در نتیجه، عدد **82** تنها با یک بایت نمایش داده می‌شود و بیت اول آن پایان عدد را تأیید می‌کند.

بخش 3) کدگذاری عدد 215

$$215_{10} = 11010111_2$$
$$\text{Blocks} = 0000001 \quad 1010111$$
$$VB(215) = 00000001 \quad 11010111$$

برای عدد **215**، ابتدا نمایش دودویی آن استخراج می‌شود. از آنجا که این مقدار بیش از **7** بیت دارد، باید آن را از سمت راست به بلوک‌های هفت‌بیتی تقسیم کنیم. با این گروه‌بندی، یک بلوک **7** بیتی کامل در سمت راست و یک بلوک تک‌بیتی (که با صفرها پر می‌شود تا به ۷ بیت برسد) در سمت چپ به دست می‌آید. اکنون برای هر بلوک یک بیت ادامه اضافه می‌کنیم. چون بلوک اول بایت نهایی نیست، مقدار آن **0** خواهد بود و برای بلوک دوم که آخرین بایت است مقدار **1** قرار می‌گیرد. بنابراین نمایش **Variable Byte** برای عدد **215** شامل دو بایت است.

بخش 4) کدگذاری عدد 12,450

$$12,450_{10} = 11000010100010_2$$
$$\text{Blocks} = 1100001 \quad 0100010$$
$$VB(12,450) = 01100001 \quad 10100010$$

ابتدا نمایش دودویی عدد **12,450** را می‌نویسیم. این مقدار دقیقاً شامل **14** بیت است و در نتیجه به‌طور کامل به دو بلوک هفت‌بیتی تقسیم می‌شود. در مرحله بعد، بیت ادامه به ابتدای هر بلوک اضافه می‌شود. برای بلوک اول (که هنوز عدد ادامه دارد) مقدار **0** و برای بلوک دوم (که پایان عدد را اعلام می‌کند) مقدار **1** قرار می‌گیرد. بدین ترتیب عدد **12,450** نیز با دو بایت در قالب این روش کدگذاری نمایش داده می‌شود.

$$\begin{aligned} \text{VB}(82) &= 11010010 \\ \text{VB}(215) &= 00000001 \quad 11010111 \\ \text{VB}(12,450) &= 01100001 \quad 10100010 \end{aligned}$$

به عنوان جمع‌بندی، نمایش کدهای بایت متغیر برای اعداد داده‌شده در کادر بالا خلاصه شده است. در تمام این موارد، بیت پرارزش هر بایت نقش تعیین‌کننده ادامه یا پایان عدد را بر عهده دارد.

نتیجه‌گیری: مقدار 0 نشان‌دهنده ادامه عدد در بایت بعدی و مقدار 1 نشان‌دهنده پایان نمایش عدد است. این معماری به سیستم اجازه می‌دهد تا اعداد کوچک را با بایت‌های کمتر و اعداد بزرگ را با بایت‌های بیشتر، به صورت کاملاً پویا و فشرده ذخیره کند.

پاسخنامه مسئله 5.33: کدگذاری گاما (Gamma Code)

بخش 1) مبانی کدگذاری گاما

$$\begin{aligned} k &= \lfloor \log_2(x) \rfloor \\ \text{Offset} &= x - 2^k \end{aligned}$$

کدگذاری گاما (**Gamma Coding**) یکی از روش‌های فشرده‌سازی برای نمایش اعداد صحیح مثبت با طول متغیر است. در این روش، هر عدد با استفاده از دو بخش اصلی نمایش داده می‌شود: بخش طول (**Length**) و بخش آفست (**Offset**). ابتدا برای یک عدد مثبت **x**، مقدار **k** به صورت کف لگاریتم در مبنای دو آن عدد محاسبه می‌شود. بخش اول (طول) مقدار مرتبه بزرگی عدد را نشان می‌دهد و به صورت کد یونیری (**Unary**) نوشته می‌شود؛ یعنی **k** بار رقم **1** نوشته شده و در انتها یک **0** قرار می‌گیرد. بخش دوم (آفست)، مقدار دقیق باقی‌مانده عدد را مشخص می‌کند که طبق معادله دوم، معادل تفاضل عدد اصلی و بزرگترین توان دوی کوچکتر از آن است. این مقدار باقی‌مانده، در مبنای دو و با طول ثابت **k** بیت نوشته می‌شود.

بخش 2) کدگذاری عدد 53 (محاسبه k)

$$\begin{aligned} k &= \lfloor \log_2(53) \rfloor \\ 2^5 &= 32 \\ 2^6 &= 64 \\ \implies k &= 5 \quad (\text{Since } 32 \leq 53 < 64) \end{aligned}$$

برای کدگذاری عدد **53** ابتدا باید مقدار **k** را با استفاده از لگاریتم محاسبه کنیم. از آنجا که توان‌های متوالی عدد دو به صورت **32** و **64** هستند و عدد **53** دقیقاً در این بازه قرار دارد، کف لگاریتم آن در مبنای دو برابر با **5** خواهد بود.

$$\text{Length Code} = 111110$$

برای تولید بخش طول، بر اساس مقدار به دست آمده برای k ، دقیقاً 5 بار رقم 1 نوشته شده و در انتها یک صفر قرار می‌گیرد. این رشته بیتی (111110) نشان‌دهنده طول نمایش دودویی عدد در کد گاما است.

بخش 4) محاسبه بخش آفست (Offset)

$$\begin{aligned}\text{Offset} &= 53 - 2^5 \\ \text{Offset} &= 53 - 32 = 21 \\ 21_{10} &= 10101_2 \quad (5 \text{ bits})\end{aligned}$$

ابتدا مقدار آفست را با کسر کردن بزرگ‌ترین توان دوی کوچکتر از عدد، از خود عدد محاسبه می‌کنیم. با کسر 32 از 53، مقدار آفست برابر با 21 به دست می‌آید. اکنون این عدد را در مبنای دو و با طول ثابت 5 بیت می‌نویسیم که معادل 10101 خواهد بود.

بخش 5) ساخت کد نهایی گاما و نتیجه‌گیری

$$\begin{aligned}\text{Gamma Code} &= \text{Length} + \text{Offset} \\ \text{Gamma}(53) &= 111110 + 10101 \\ \text{Gamma}(53) &= 11111010101\end{aligned}$$

کد نهایی گاما به سادگی با اتصال مستقیم بخش طول و بخش آفست ساخته می‌شود. در کدگذاری گاما، ابتدا طول نمایش دودویی عدد با استفاده از کد یونیری مشخص می‌شود و سپس بخش باقی‌مانده عدد به صورت باینری در ادامه آن نوشته می‌شود.

نتیجه‌گیری: برای عدد 53، با ترکیب بخش طول (111110) و بخش آفست (10101)، کد نهایی به صورت یکپارچه و فشرده برابر با 11111010101 تولید می‌گردد.

پاسخنامه مسئله 5.34:

مقایسه تطبیقی Variable Byte Encoding و Gamma Encoding

بخش 1) تفاوت در تراکم فشرده‌سازی (Compression Density)

$$\text{Gamma Length} = 2 \lfloor \log_2(x) \rfloor + 1$$

روش **Gamma Encoding** یک استراتژی فشرده‌سازی مبتنی بر بیت است که در آن طول کد دقیقاً متناسب با اندازه عدد تعیین می‌شود. همان‌طور که در معادله بالا مشخص است، تعداد بیت‌های مورد نیاز برای نمایش یک عدد مثبت **x** از طریق یک تابع لگاریتمی محاسبه می‌گردد. به دلیل این ساختار، کدگذاری گاما برای اعداد کوچک بسیار فشرده عمل می‌کند. این موضوع در سیستم‌های بازیابی اطلاعات اهمیت زیادی دارد، زیرا در **Posting List**ها معمولاً از روش **Gap Encoding** استفاده می‌شود و مقادیر شکاف (فواصل) اغلب کوچک هستند (برای نمونه، عدد **1** در کدگذاری گاما تنها با یک بیت نمایش داده می‌شود).

در مقابل، روش **Variable Byte Encoding (VB)** مبتنی بر بایت است. در این روش هر عدد حداقل به یک بایت کامل نیاز دارد و بنابراین کوچک‌ترین واحد ذخیره‌سازی برابر با **8** بیت است. این ویژگی باعث ایجاد نوعی اتلاف حافظه مشابه **Internal Fragmentation** می‌شود، زیرا فضای اختصاص‌یافته همیشه مضربی از **8** بیت است (به عنوان مثال، تمام اعداد کوچک‌تر از **128** در VB دقیقاً با یک بایت نمایش داده می‌شوند، در حالی که همان اعداد در گاما با چند بیت معدود ذخیره می‌گردند). بنابراین از نظر **Compression Density**، کدگذاری گاما در مجموعه داده‌هایی با مقادیر کوچک فراوان، عملکردی به مراتب بهتر دارد.

بخش 2) تفاوت در سرعت کدگشایی (Decoding Speed)

معماری پردازنده‌های مدرن برای پردازش داده‌ها در واحدهای بایت یا کلمه طراحی شده است. عملیات خواندن داده از حافظه معمولاً در بسته‌های **8**، **16**، **32** یا **64** بیتی انجام می‌شود. کدگذاری **Variable Byte** به دلیل ساختار بایت‌محور خود، با این معماری کاملاً سازگار است. در فرآیند کدگشایی VB، پردازنده تنها کافی است بایت‌های متوالی را بخواند و با دستورهای سریع سخت‌افزاری مانند **Bit Masking** و **Bit Shifting**، وضعیت بیت‌های کنترل‌کننده را بررسی کند؛ از این رو سرعت اجرای بسیار بالایی دارد.

در مقابل، کدگذاری **Gamma** مرز مشخصی با بایت‌ها ندارد و ممکن است یک کد در وسط یک بایت شروع یا پایان یابد. استخراج مقادیر در این روش نیازمند جداسازی دقیق بیت‌ها و عملیات ماسک‌گذاری‌های متعدد و پیچیده است که باعث می‌شود فرآیند **Decoding** در گاما به مراتب پیچیده‌تر و زمان‌برتر از VB باشد.

بخش 3) ارزیابی نهایی تریدآف

انتخاب بین این دو روش در عمل یک موازنه میان کاهش مصرف حافظه و سرعت بازیابی است. اگر هدف اصلی کاهش حداکثری اندازه نمایه روی دیسک باشد، **Gamma Encoding** گزینه ایده‌آلی است. در مقابل، در سیستم‌های عملیاتی مانند موتورهای جستجوی وب که سرعت پاسخ‌گویی حیاتی است، روش **Variable Byte** ترجیح داده می‌شود. (البته در سیستم‌های پیشرفته، برای تلفیق فشرده‌سازی و سرعت بالا، از معماری‌های پردازشی موازی مانند **SIMD** استفاده می‌گردد).

نتیجه‌گیری: در یک مقایسه مستقیم، **Gamma Encoding** از نظر تراکم فشرده‌سازی عملکرد بهتری دارد، در حالی که **Variable Byte Encoding** در سرعت کدگشایی و اجرای عملی برتری قاطعی دارد. انتخاب نهایی تابع اولویت‌های سیستم میان مصرف حافظه دیسک و سرعت پردازش CPU خواهد بود.

$$\begin{aligned}\text{DocIDs} &= [100, 102, 110, 310, 315] \\ \text{Gap}_1 &= 100 \\ \text{Gap}_2 &= 102 - 100 = 2 \\ \text{Gap}_3 &= 110 - 102 = 8 \\ \text{Gap}_4 &= 310 - 110 = 200 \\ \text{Gap}_5 &= 315 - 310 = 5 \\ \text{Gap List} &= [100, 2, 8, 200, 5]\end{aligned}$$

در سیستم‌های بازیابی اطلاعات، برای کاهش اندازه **Posting List** معمولاً از روش **Gap Encoding** استفاده می‌شود. در این روش به جای ذخیره مستقیم شناسه اسناد، اختلاف میان شناسه‌های متوالی محاسبه می‌گردد. همان‌طور که در محاسبات یکپارچه بالا مشاهده می‌شود، برای لیستی از شناسه‌های مفروض، شناسه اول بدون تغییر ذخیره شده و هر مقدار بعدی از مقدار قبلی خود کسر می‌گردد. از آنجا که شناسه اسناد به صورت صعودی مرتب هستند، این اختلاف‌ها (فواصل) به اعداد بسیار کوچک‌تری تبدیل می‌شوند که امکان فشرده‌سازی مؤثرتری را در مراحل بعدی فراهم می‌کنند.

بخش 2) بررسی کارایی کدگذاری Variable Byte

$$\begin{aligned}\text{VB Sizes} &= [1, 1, 1, 2, 1] \text{ bytes} \\ \text{Total VB Space} &= 1 + 1 + 1 + 2 + 1 = 6 \text{ bytes} \\ \text{Total VB Space (bits)} &= 6 \times 8 = 48 \text{ bits}\end{aligned}$$

در روش **Variable Byte Encoding (VB)** هر عدد در واحدهای بایت کامل ذخیره می‌شود. اگر مقدار عدد کوچک‌تر از **128** باشد، تنها به یک بایت نیاز دارد؛ در غیر این صورت از دو یا چند بایت استفاده می‌شود. با توجه به لیست شکاف‌های استخراج‌شده، اعداد **100**، **2**، **8** و **5** همگی کمتر از **128** بوده و تنها یک بایت اشغال می‌کنند. اما عدد **200** بزرگ‌تر است و به **2** بایت نیاز دارد. با جمع این مقادیر، مجموع فضای مورد نیاز برای این لیست در روش بایت متغیر برابر با **6** بایت یا معادل **48** بیت محاسبه می‌گردد.

بخش 3) بررسی کارایی کدگذاری Gamma

$$\begin{aligned}\text{Gamma Length} &= 2\lfloor \log_2(x) \rfloor + 1 \\ \text{Len}(100) &= 13 \text{ bits}, \quad \text{Len}(2) = 3 \text{ bits} \\ \text{Len}(8) &= 7 \text{ bits}, \quad \text{Len}(200) = 15 \text{ bits} \\ \text{Len}(5) &= 5 \text{ bits} \\ \text{Total Gamma Space} &= 13 + 3 + 7 + 15 + 5 = 43 \text{ bits}\end{aligned}$$

در روش **Gamma Encoding** طول کد برای یک عدد مثبت **x** از رابطه لگاریتمی فوق به دست می‌آید. با اعمال این فرمول ریاضی روی تک‌تک عناصر لیست شکاف‌ها، طول کد گاما برای مقادیر به دست آمده به ترتیب برابر با **13**، **3**، **7**، **15** و **5** بیت استخراج می‌شود. با جمع‌بندی این مقادیر، مجموع فضای مورد نیاز برای کدگذاری گاما در این مثال دقیقاً **43** بیت خواهد بود.

$$\begin{aligned} \text{Total VB Space} &= 48 \text{ bits} \\ \text{Total Gamma Space} &= 43 \text{ bits} \\ \implies \text{Gamma} &< \text{VB} \quad (43 < 48) \end{aligned}$$

با مقایسه فضای مصرفی دو روش، مشاهده می‌شود که در این مثال خاص، روش **Gamma Encoding** به دلیل عدم وابستگی به مرزهای بایت و توانایی فشرده‌سازی بسیار دقیق اعداد کوچک، فضای کمتری اشغال می‌کند (صرفه‌جویی 5 بیتی). اگرچه در مقیاس‌های بزرگ، اختلاف چند بیت در هر لیست ممکن است ناچیز به نظر برسد، اما در سطح نظری کدگذاری گاما در این مثال فشرده‌سازی بهتری ارائه می‌دهد.

نتیجه‌گیری: اگر اولویت اصلی سیستم حداکثر فشرده‌سازی دیسک باشد، کدگذاری گاما انتخاب بهتری است. اما اگر هدف اصلی سرعت پردازش و بازیابی سریع اطلاعات باشد، روش Variable Byte به دلیل سادگی در رمزگشایی و سازگاری بهتر با معماری پردازنده‌ها، گزینه عملی‌تری محسوب می‌شود.

پاسخنامه مسئله 5.36: تحلیل الگوریتم جست‌وجو در Blocked Dictionary

بخش 1) توضیح مسئله و ضرورت بلوک‌بندی

در دیکشنری‌های بزرگ (**Large Dictionaries**)، اگر هر واژه به صورت مستقل همراه با اشاره‌گر یا اطلاعات جانبی ذخیره شود، فضای زیادی از حافظه مصرف می‌شود. برای کاهش این سربار، از روش بلوک‌بندی (**Blocking**) استفاده می‌شود. در این روش، واژه‌ها به گروه‌هایی با اندازه ثابت تقسیم می‌شوند. اگر $k = 8$ باشد، یعنی هر بلوک شامل 8 واژه متوالی از دیکشنری است. برای مثال در این حالت، واژه‌های اول تا هشتم در بلوک اول، واژه‌های نهم تا شانزدهم در بلوک دوم، و به همین ترتیب ذخیره می‌شوند. هدف اصلی این ساختار، کاهش تعداد اشاره‌گرها و بهینه‌سازی فضای ذخیره‌سازی دیکشنری است؛ اما این کاهش فضا باعث می‌شود که دیگر نتوان برای تمام واژه‌ها به صورت مستقیم از جست‌وجوی دودویی (**Binary Search**) استفاده کرد.

بخش 2) چرا جست‌وجوی دودویی مستقیم برای همه واژه‌ها ممکن نیست؟

جست‌وجوی دودویی زمانی به صورت مستقیم قابل استفاده است که بتوان در زمان ثابت به عنصر میانی دسترسی داشت (**Random Access**)؛ یعنی ساختار داده باید شبیه یک آرایه از واژه‌های جداگانه باشد که هر خانه آن دقیقاً شروع یک واژه را مشخص کند. در دیکشنری بدون بلوک‌بندی، برای هر واژه یک اشاره‌گر جداگانه نگهداری می‌شود و می‌توان مستقیم به ابتدای رشته مربوطه رفت. اما در دیکشنری بلوک‌بندی شده، برای کاهش حافظه، تنها برای ابتدای هر بلوک یک اشاره‌گر در ایندکس نگهداری می‌شود. با فرض $k = 8$ ، اگر بخواهیم واژه پنجم داخل یک بلوک را مستقیماً بخوانیم، آدرس شروع آن را از قبل نداریم. به همین دلیل، شرط اصلی جست‌وجوی دودویی برای همه واژه‌ها نقض می‌گردد.

بخش 3) نقش ایندکس در دیکشنری بلوک‌بندی شده

در ساختار بلوک‌بندی شده، یک لایه ایندکس جداگانه نگهداری می‌شود که شامل اطلاعات مربوط به ابتدای هر بلوک است. برای هر بلوک، یک واژه کلیدی یا نماینده (معمولاً اولین واژه بلوک) و یک اشاره‌گر به محل شروع آن در فایل ذخیره می‌شود. اگر $k = 8$ باشد، ایندکس برای هر 8 واژه فقط یک ورودی دارد. بنابراین اندازه ایندکس تقریباً یک‌هشتم حالتی است که برای تمام واژه‌ها

اشاره‌گر جداگانه ذخیره شود. این کاهش اندازه بسیار مفید است، اما باعث می‌شود جست‌وجو دو مرحله‌ای شود: ابتدا باید بلوک مناسب پیدا شود و سپس جست‌وجوی داخلی صورت گیرد.

بخش 4) جست‌وجوی دودویی در لایه ایندکس

$$a_i \leq \text{query} < a_{i+1}$$

در مرحله اول، جست‌وجوی دودویی روی لایه ایندکس انجام می‌شود. ورودی‌های ایندکس مرتب هستند و سیستم به دنبال بلوک (مثلاً بلوک i) می‌گردد که واژه مورد نظر (**Query**) در آن بازه قرار گیرد؛ یعنی طبق فرمول بالا، از نماینده بلوک فعلی بزرگ‌تر یا مساوی و از نماینده بلوک بعدی کوچک‌تر باشد. اگر تعداد کل واژه‌ها N و اندازه بلوک $k = 8$ باشد، این جست‌وجوی دودویی روی حدود $N / 8$ ورودی انجام می‌شود که بسیار سریع‌تر از جست‌وجو روی کل N واژه است.

بخش 5) پرش به بلوک و پیمایش خطی

پس از یافتن بلوک مناسب، سیستم با استفاده از اشاره‌گر به ابتدای آن بلوک می‌پرد. چون واژه‌ها داخل بلوک به صورت فشرده و بدون آدرس مستقل ذخیره شده‌اند، سیستم باید پیمایش خطی (**Linear Scan**) را آغاز کند. روند کار شامل خواندن واژه‌ها یکی‌یکی و مقایسه آن‌ها با واژه مورد جست‌وجو است. اگر واژه پیدا شد، عملیات موفقیت‌آمیز است؛ و اگر واژه خوانده‌شده بزرگ‌تر از واژه مورد نظر شد (به دلیل مرتب بودن دیکشنری)، می‌توان نتیجه گرفت که واژه در لیست وجود ندارد. به دلیل کوچک بودن مقدار $k = 8$ ، این پیمایش خطی سربار ناچیزی دارد.

بخش 6) تحلیل هزینه جست‌وجو

$$\text{Cost}_{k=8} \approx \log_2 \left(\frac{N}{8} \right) + 8$$
$$\text{General Cost} \approx \log_2 \left(\frac{N}{k} \right) + k$$

هزینه نهایی جست‌وجو از ترکیب دو بخش حاصل می‌شود: هزینه جست‌وجوی دودویی در ایندکس به علاوه هزینه پیمایش خطی در بلوک. همان‌طور که در روابط ریاضی یکپارچه بالا نشان داده شده است، برای اندازه بلوک $k = 8$ ، هزینه از جمع لگاریتمی $N/8$ با مقدار ماکزیمم 8 پیمایش خطی استخراج می‌گردد. معادله دوم نیز فرمول تعمیم‌یافته این هزینه برای هر بلوک دلخواه با سایز k را نشان می‌دهد.

بخش 7) پاسخ نهایی

در دیکشنری بلوک‌بندی شده با $k = 8$ ، امکان استفاده مستقیم از جست‌وجوی دودویی برای همه واژه‌ها وجود ندارد، زیرا آدرس تک‌تک واژه‌ها در سیستم ذخیره نشده است. فرایند جست‌وجو شامل دو مرحله مجزا است: ابتدا جست‌وجوی دودویی در لایه ایندکس (که شامل واژه‌های آغازین بلوک‌هاست) برای یافتن بلوک مناسب، و سپس پیمایش خطی (حداکثر تا 8 واژه) داخل آن بلوک.

نتیجه‌گیری: این ساختار یک تعادل استراتژیک و هوشمندانه میان صرفه‌جویی در حافظه (Memory Efficiency) و

سرعت عملیاتی در جست‌وجو (Search Speed) ایجاد می‌کند.

پاسخنامه مسئله 5.37: بازدهی روش بایت متغیر و صرفه‌جویی 50% نسبت به ذخیره‌سازی 32-bit

بخش 1) مفهوم ذخیره‌سازی 32-bit استاندارد

$$\text{Total Space (32-bit)} = 4 \times 32 = 128 \text{ bits}$$

در روش ذخیره‌سازی **32-bit** استاندارد، برای هر عدد صحیح، بدون توجه به بزرگی یا کوچکی آن، دقیقاً **32** بیت فضا اختصاص داده می‌شود. یعنی اگر یک لیست ارجاع شامل چندین شکاف یا **Gap** باشد، هر شکاف با یک عدد **32** بیتی ذخیره می‌شود. در این حالت، حتی اگر مقدار شکاف بسیار کوچک باشد (مثلاً **1**، **5** یا **20**)، باز هم همان **32** بیت کامل برای آن مصرف می‌شود. همان‌طور که در معادله فوق مشخص است، به عنوان مثال، اگر چهار شکاف داشته باشیم، فضای مصرفی در روش **32-bit** دقیقاً **128** بیت خواهد بود. نکته مهم این است که در این روش، اندازه واقعی عدد هیچ تأثیری روی فضای مصرفی ندارد. اعداد **3**، **100**، **10,000** و **1,000,000** همگی به طور ثابت **32** بیت فضا هدر می‌دهند.

جمع‌بندی این بخش: در ذخیره‌سازی **32-bit** استاندارد، هر **Gap** همیشه **32** بیت فضا می‌گیرد، حتی اگر مقدار آن بسیار کوچک باشد. بنابراین این روش برای اعداد کوچک هدررفت زیادی دارد.

بخش 2) مفهوم کدگذاری بایت متغیر یا VB

در کدگذاری **Variable Byte (VB)**، هر عدد به صورت یک یا چند بایت ذخیره می‌شود. هر بایت **8** بیت دارد، اما تمام این **8** بیت برای داده عددی استفاده نمی‌شود. در هر بایت **VB**، یک بیت برای مشخص کردن ادامه‌دار بودن یا پایان عدد تخصیص می‌یابد و **7** بیت باقی‌مانده صرف ذخیره مقدار عددی می‌شود. این بیت کنترلی معمولاً **Continuation Bit** نام دارد. اگر بایت فعلی آخرین بایت عدد باشد، این بیت برابر **1** قرار می‌گیرد. اگر بعد از این بایت هنوز بایت دیگری برای همین عدد وجود داشته باشد، این بیت برابر **0** قرار می‌گیرد. بنابراین در **VB**، هر بایت فقط **7** بیت داده مفید دارد.

جمع‌بندی این بخش: در کدگذاری **VB**، برخلاف ذخیره‌سازی **32-bit**، اندازه فضای مصرفی به بزرگی عدد بستگی دارد. اعداد کوچک با بایت‌های کمتر و اعداد بزرگ با بایت‌های بیشتر ذخیره می‌شوند.

بخش 3) شرط کلی برای صرفه‌جویی 50%

صرفه‌جویی **50%** یعنی فضای مصرفی روش **VB** دقیقاً نصف فضای مصرفی روش **32-bit** باشد. در روش **32-bit**، هر عدد **32** بیت فضا مصرف می‌کند که نصف آن برابر با **16** بیت است. از آنجا که هر بایت **8** بیت دارد، **16** بیت معادل با **2** بایت است. بنابراین شرط اصلی این است: میانگین تعداد بایت‌های مصرفی برای هر **Gap** در روش **VB** باید حداکثر **2** بایت باشد.

جمع‌بندی این بخش: **VB** زمانی حداقل **50%** صرفه‌جویی نسبت به ذخیره‌سازی **32-bit** ایجاد می‌کند که میانگین اندازه کدگذاری هر **Gap** حداکثر **2** بایت (یعنی حداکثر **16** بیت) باشد.

بخش 4) نقش اندازه Gap‌ها در تعداد بایت‌های VB

در **VB**، تعداد بایت‌های لازم برای ذخیره هر **Gap** مستقیماً به اندازه آن بستگی دارد. محدوده‌های مهم در **VB** به این صورت هستند: مقادیر از **1** تا **127** با یک بایت، از **128** تا **16,383** با دو بایت، و از **16,384** تا **2,097,151** با سه بایت ذخیره می‌شوند. اگر همه **Gap**‌ها حداکثر **16,383** باشند، هر کدام حداکثر با **2** بایت ذخیره شده و در نتیجه، **VB** نسبت به **32** بیت، قطعاً حداقل **50%** صرفه‌جویی خواهد داشت.

جمع‌بندی این بخش: اندازه **Gap**‌ها عامل اصلی در بازدهی **VB** است. هرچه **Gap**‌ها کوچک‌تر باشند، تعداد بایت‌های لازم کمتر می‌شود و احتمال رسیدن به صرفه‌جویی **50%** یا بیشتر افزایش می‌یابد.

بخش 5) حالت دقیق صرفه‌جویی 50%

برای رسیدن دقیق به مرز صرفه‌جویی 50٪، میانگین فضای مصرفی هر **Gap** در **VB** باید دقیقاً 2 بایت باشد. این حالت ممکن است به چند شکل رخ دهد؛ مثلاً اگر همه شکاف‌ها در محدوده 128 تا 16,383 باشند، هر کدام دقیقاً با 2 بایت ذخیره می‌شوند. یا ممکن است ترکیبی از مقادیر کوچک (یک‌بایتی) و بزرگ (سه‌بایتی) داشته باشیم به طوری که میانگین کلی به 2 بایت برسد. در عمل، در لیست‌های ارجاع سیستم‌های بازیابی اطلاعات، معمولاً تعداد بسیار زیادی از این فواصل کوچک (یک‌بایتی) هستند.

جمع‌بندی این بخش: صرفه‌جویی دقیق 50٪ زمانی رخ می‌دهد که میانگین مصرف **VB** برابر با 2 بایت برای هر **Gap** باشد.

بخش 6) تحلیل با مثال عددی

$$\text{Total Space (32-bit)} = 1,000 \times 32 = 32,000 \text{ bits}$$

برای تحلیل دقیق‌تر، فرض کنید یک لیست ارجاع شامل 1,000 عدد **Gap** باشد. همان‌طور که در محاسبه بالا مشخص است، در روش **32-bit** فضای کل اشغالی برابر با 32,000 بیت خواهد بود. برای اینکه روش **VB** صرفه‌جویی 50٪ داشته باشد، مجموع مصرف آن باید حداکثر 16,000 بیت یا 2,000 بایت باشد (میانگین 2 بایت برای هر شکاف). اگر تمام فواصل کوچکتر از 128 باشند، کل مصرف 1,000 بایت خواهد بود (یعنی 75٪ صرفه‌جویی). اگر همه مقادیر بین 128 تا 16,383 باشند، کل مصرف 2,000 بایت می‌شود که دقیقاً معادل 50٪ صرفه‌جویی است.

بخش 7) نقش توزیع Gap‌ها در بازدهی VB

آنچه در تحلیل بازدهی **VB** اهمیت دارد، توزیع اندازه **Gap**‌ها است. ممکن است یک لیست چند مقدار بسیار بزرگ داشته باشد، اما اگر بیشتر آن‌ها کوچک باشند، میانگین همچنان پایین باقی می‌ماند و **VB** می‌تواند صرفه‌جویی 50٪ یا بیشتر ایجاد کند. بنابراین در تحلیل واقعی یک سیستم، باید بررسی کرد که چند درصد مقادیر یک‌بایتی، چند درصد دو‌بایتی، چند درصد سه‌بایتی و چند درصد چهاربایتی هستند.

بخش 8) پاسخ نهایی

کدگذاری **VB** زمانی نسبت به ذخیره‌سازی **32-bit** استاندارد منجر به صرفه‌جویی 50٪ می‌شود که میانگین فضای مصرفی برای هر **Gap** حداکثر 2 بایت باشد. نقش اندازه فواصل در این تحلیل بسیار مهم است.

نتیجه‌گیری: مقادیر کوچک‌تر از 128 فقط یک بایت مصرف می‌کنند و باعث صرفه‌جویی بیشتر از 50٪ می‌شوند. مقادیر بین 128 و 16,383 دو بایت مصرف می‌کنند و دقیقاً با شرط صرفه‌جویی 50٪ سازگارند. اما مقادیر بزرگ‌تر از 16,383 به سه بایت یا بیشتر نیاز دارند و در صورت تعدد، باعث می‌شوند صرفه‌جویی کل به کمتر از 50٪ تنزل پیدا کند.

پاسخنامه مسئله 5.38: اثر حذف کلمات توقف بر نمودار Zipf در مختصات Log-Log

بخش 1) قانون Zipf و شکل نمودار آن

$$f \approx \frac{C}{r}$$
$$\log(f) \approx \log(C) - \log(r)$$

قانون زیپف (**Zipf's Law**) رابطه‌ای بنیادین میان رتبه (Rank) یک واژه و فراوانی (Frequency) وقوع آن در یک پیکره متنی بزرگ را توصیف می‌کند. بر اساس معادله نخست در کادر بالا، اگر رتبه یک واژه را با r و فراوانی آن را با f نشان دهیم، فراوانی واژه با معکوس رتبه آن تناسب دارد که در آن C یک مقدار ثابت است. با انتقال این رابطه به مختصات دو لگاریتمی (Log-Log Plot) و اعمال قوانین لگاریتم، معادله به یک خط مستقیم مطابق رابطه دوم تبدیل می‌شود. این دگرگونی هندسی اثبات می‌کند که در فضای دو لگاریتمی، قانون زیپف خطی با شیب منفی -1 ایجاد می‌کند.

بخش 2) نقش کلمات توقف در ابتدای توزیع

کلمات توقف (**Stop Words**)، مانند «و»، «از»، «در»، «به»، واژه‌هایی هستند که در متن بسیار پرتکرارند اما ارزش معنایی کمی برای فرآیندهای بازیابی اطلاعات (Information Retrieval) دارند. در نمودار زیپف، این واژه‌ها دقیقاً در «ابتدای توزیع» (Head of the distribution) قرار می‌گیرند؛ یعنی رتبه‌های بسیار کوچکی (نزدیک به یک) دارند اما فراوانی وقوع آن‌ها فوق‌العاده زیاد است. به‌طور معمول، چند ده واژه اول، بخش عظیمی از کل رخدادهای واژگان پیکره را به خود اختصاص می‌دهند. این بخش ابتدایی منحنی معمولاً به دلیل اشباع فرکانسی، بالاتر از خط ایده‌آل قرار گرفته و دچار انحنای شکستگی ساختاری می‌شود.

بخش 3) اثر حذف 100 کلمه پرتکرار بر محور رتبه

$$r_{\text{new}} = r_{\text{old}} - 100 \quad (\text{For } r_{\text{old}} > 100)$$

اگر 100 کلمه پرتکرار اول (که عموماً همان کلمات توقف هستند) از مجموعه داده‌ها حذف شوند، توزیع رتبه‌ای واژگان دچار جابه‌جایی افقی می‌شود. همان‌طور که در رابطه فوق مدل شده است، واژه‌ای که پیش از این رتبه 101 را به خود اختصاص داده بود، با حذف صد کلمه پیشین، به رتبه 1 تغییر جایگاه می‌دهد. در نتیجه، کل منحنی توزیع به سمت چپ نمودار منتقل شده و محور رتبه‌ها فشرده‌تر می‌گردد. در این فرآیند، ترتیب نسبی و توزیعی واژه‌های باقی‌مانده کاملاً ثابت باقی می‌ماند و تنها اندیس عددی رتبه آن‌ها کاهش می‌یابد.

بخش 4) اثر حذف کلمات توقف بر فراوانی‌ها

$$f_{\text{max (new)}} = f(r_{\text{old}} = 101) \ll f_{\text{max (old)}}$$

با حذف کلمات توقف، مقادیر فرکانسی بسیار بزرگی که به این واژه‌ها تعلق داشتند از پیکره متنی حذف می‌شوند. طبق رابطه ریاضی فوق، این رویداد باعث می‌شود که سقف ماکزیمم فراوانی موجود در داده‌ها به شدت افت کرده و به سطح فراوانی واژه صد و یکم محدود شود. این امر اختلاف فاحش میان فرکانس کلمات صدرنشین با سایر واژگان را متعادل می‌کند. در نمودار دو لگاریتمی، این تغییر ساختار قله مرتفع و انحنا دار ابتدای منحنی را کاملاً حذف کرده و شروع نمودار را از نقطه‌ای به مراتب پایین‌تر در محور عمودی رقم می‌زند، هرچند فرکانس مطلق خود واژه‌های باقی‌مانده دستخوش تغییر نخواهد شد.

بخش 5) اثر حذف بر شیب نمودار در مختصات Log-Log

$$\text{Slope}_{\text{Log-Log}} \approx -1$$

قانون زیپ انطباق دقیق خود را عمدتاً در بخش‌های میانی و انتهایی توزیع نشان می‌دهد؛ چرا که کلمات توقف در ابتدای توزیع به دلیل حجم وقوع غیرعادی، همواره باعث انحراف و شکستگی خط مستقیم می‌شوند. با حذف این کلمات، بخش باقی‌مانده به خط ایده‌آل زیپ قرابت بیشتری پیدا می‌کند. همان‌طور که در رابطه بالا ذکر شده است، مقدار شیب کلی خط توزیع تغییر بنیادی نخواهد داشت و همچنان در حوالی **-1** پایدار می‌ماند؛ زیرا بدنه و شیب اصلی نمودار توسط رفتار کلمات میانی پیکره تعیین می‌شود و حذف سر این توزیع تنها به خطی‌تر و منظم‌تر شدن بدنه کمک می‌کند.

بخش 6) پاسخ نهایی و دگرگونی شکل کلی نمودار

پس از حذف **100** واژه پرتکرار اول، نمودار در مختصات دو لگاریتمی دچار اصلاحات ساختاری می‌شود: ابتدای انحنادار نمودار حذف شده و از نقطه‌ای پایین‌تر آغاز می‌شود، رتبه واژه‌ها کاهش یافته و نمودار کلی به سمت چپ منتقل می‌شود، و بخش میانی بیشتر شبیه یک خط مستقیم بی‌نقص می‌گردد. در واقع، شکل کلی نمودار کوتاه‌تر می‌شود اما ساختار قانون توانی (Power Law) آن کاملاً پابرجا می‌ماند.

نتیجه‌گیری نهایی: حذف کلمات توقف فرکانس‌های اغراق‌شده ابتدای لیست را پاکسازی می‌کند. این کار اگرچه نمودار را در فضای Log-Log به سمت چپ و پایین شیف‌ت می‌دهد، اما با حذف نویزهای ابتدایی، رفتار توزیع واژگان باقی‌مانده را به خط ایده‌آل زیپ با شیب -1 نزدیک‌تر و فرآیند تحلیل را خطی‌تر و نظام‌مندتر می‌کند.

پاسخنامه مسئله 5.39: بررسی نقض فرض یکنواختی فاصله‌ها و تأثیر آن بر فشرده‌سازی گاما

بخش 1) مفهوم فرض یکنواختی در فاصله‌ها

$$\text{Expected Gap} \approx \frac{N}{df}$$

در بسیاری از مدل‌های کلاسیک تخمین اندازه نمایه، فرض بر این است که کلمات به‌طور یکنواخت در میان تمام اسناد موجود در مجموعه توزیع شده‌اند. اگر تعداد کل اسناد را **N** و تعداد اسناد حاوی یک واژه خاص را **df** (فرکانس سند) در نظر بگیریم، طبق این فرض، فاصله‌های بین شناسه‌های اسناد (**DocID Gaps**) تقریباً برابر با عدد حاصل از تقسیم **N** بر **df** است. این فرض به این معناست که اگر یک واژه در ده درصد اسناد مجموعه حضور دارد، انتظار می‌رود در هر **10** سند، یکبار تکرار شود. این توزیع یکنواخت، کار طراحان سیستم را ساده می‌کند، زیرا باعث می‌شود اندازه شکاف‌ها (**Gaps**) تقریباً ثابت باشد و رفتار فشرده‌سازی کاملاً قابل پیش‌بینی باقی بماند.

بخش 2) مثال‌هایی از واژه‌های با توزیع غیریکنواخت

بسیاری از واژه‌ها در دنیای واقعی از الگوی یکنواخت پیروی نمی‌کنند. این واژه‌ها معمولاً در خوشه‌های زمانی یا موضوعی خاص ظاهر می‌شوند. چند نمونه بارز عبارت‌اند از:

- **واژه‌های رویدادمحور:** واژه‌هایی مانند «المپیک»، «زلزله» یا «انتخابات» که تنها در بازه‌های زمانی خاصی دیده می‌شوند.
- **واژه‌های مربوط به ترندهای زودگذر:** واژه‌هایی که به تکنولوژی‌های جدید یا سوژه‌های خبری جنجالی اشاره دارند و پس از فروکش کردن خبر، ناپدید می‌شوند.

- **واژه‌های فصلی:** واژه‌هایی مانند «کریسمس» یا «نوروز» که هر سال تنها در ماه‌های خاصی به وفور تکرار می‌شوند.

بخش 3) چرا توزیع فاصله‌ها یکنواخت نیست؟ (Burstiness)

علت اصلی غیریکنواخت بودن توزیع فاصله‌ها، پدیده‌ای به نام «خوشه‌ای بودن» یا **Burstiness** است. اسناد معمولاً بر اساس زمان تولید یا موضوع مرتب شده‌اند. وقتی موضوعی در مرکز توجه قرار می‌گیرد، تعداد زیادی مطلب درباره آن تولید می‌شود. در نتیجه، در بخشی از لیست ارجاع (**Posting List**) آن واژه، تعداد زیادی سند پشت سر هم قرار می‌گیرند که باعث می‌شود فاصله یا همان شکاف بین شناسه‌های اسناد بسیار کوچک شود. برعکس، پس از پایان آن رویداد، فاصله بین آخرین سند دوره اول و اولین سند دوره بعدی بسیار بزرگ خواهد بود. در واقع، لیست ارجاع شامل ترکیبی ناموزون از شکاف‌های بسیار کوچک و بسیار بزرگ است که این تضاد، نقض صریح فرض یکنواختی است.

بخش 4) تأثیر بر فشردگی گاما (Gamma Encoding)

$$\text{Gamma Length} = 2 \lfloor \log_2(\text{Gap}) \rfloor + 1$$

روش **Gamma-encoding** برای اعداد کوچک بسیار بهینه است. همان‌طور که در معادله لگاریتمی فوق نشان داده شده است، طول این کدگذاری تابعی از لگاریتم مبنای دو از عدد مورد نظر است. وقتی فاصله‌ها یکنواخت باشند، اندازه اکثر شکاف‌ها کوچک است و فشردگی گاما با کمترین تعداد بیت انجام می‌شود. اما وقتی شکاف‌ها به دلیل خوشه‌ای شدن غیریکنواخت باشند، دو اثر متناقض دیده می‌شود: اولاً در بخش‌های خوشه (**Burst**)، شکاف‌ها فوق‌العاده کوچک هستند و فشردگی عالی عمل می‌کند. دوماً در بخش‌های سکوت و فاصله بین دو خوشه، شکاف‌ها بسیار بزرگ می‌شوند. کدگذاری گاما برای اعداد بزرگ، طول کد را به صورت دو برابر لگاریتم عدد افزایش می‌دهد؛ در نتیجه، غیریکنواخت بودن فاصله‌ها باعث می‌شود بازدهی کدگذاری گاما در فواصل بین خوشه‌ها به شدت کاهش یابد.

بخش 5) پاسخ نهایی

فرض یکنواختی فاصله‌ها در لیست‌های ارجاع با واقعیت در تضاد است، زیرا بسیاری از واژه‌ها دارای رفتار خوشه‌ای (**Burstiness**) هستند. این واژه‌ها در بازه‌های زمانی خاص به صورت فشرده در اسناد ظاهر می‌شوند که منجر به شکاف‌های بسیار کوچک در زمان‌های پرکاربرد و شکاف‌های بسیار بزرگ در زمان‌های کم‌کاربرد می‌شود.

نتیجه‌گیری: از آنجا که روش **Gamma-encoding** منحصراً برای اعداد کوچک بهینه شده است، این پراکندگی نامنظم باعث می‌شود کارایی فشردگی در مواجهه با شکاف‌های بزرگ بین خوشه‌ها به شدت کاهش یابد؛ برخلاف فرضی که تصور می‌کند همواره با توزیعی همگن و کدهای کوتاه روبه‌رو هستیم.

پاسخنامه مسئله 5.40: بهینگی کد یکانی (Unary Code) برای توزیع هندسی

بخش 1) ساختار کد یکانی (Unary Code)

کد یکانی (**Unary Code**) یکی از ساده‌ترین روش‌های نمایش اعداد صحیح مثبت است. در این روش برای نمایش عدد صحیح n ، از تعداد $n-1$ عدد «یک» و سپس یک عدد «صفر» استفاده می‌شود (یا برعکس). به عنوان مثال: عدد 1 به صورت 0 (طول 1 بیت)، عدد 2 به صورت 10 (طول 2 بیت)، عدد 3 به صورت 110 (طول 3 بیت) و عدد 4 به صورت 1110 (طول 4 بیت) نمایش داده می‌شوند. به طور کلی، طول کد یکانی برای عدد n دقیقاً برابر با n بیت است.

$$P(n) = \left(\frac{1}{2}\right)^n$$

$$L(n) = -\log_2(P(n))$$

$$L(n) = -\log_2\left(\left(\frac{1}{2}\right)^n\right) = -(-n) = n$$

در نظریه اطلاعات (**Information Theory**)، هر کد زمانی بهینه است که طول کد برای هر مقدار **n** برابر با «محتوای اطلاعاتی» یا همان منفی لگاریتم احتمال وقوع آن مقدار باشد. یعنی اگر احتمال وقوع عدد **n** را **P(n)** بنامیم، طول ایده‌آل کد **L(n)** باید برابر با **-log₂(P(n))** باشد. کد یکانی دقیقاً برای توزیع هندسی (**Geometric Distribution**) با پارامتر **0.5** بهینه است. همان‌طور که در معادلات یکپارچه بالا مشاهده می‌شود، با قرار دادن این احتمال در فرمول محتوای اطلاعاتی شانون، طول کد دقیقاً برابر با **n** به دست می‌آید. از آنجا که طول واقعی کد یکانی نیز دقیقاً **n** است، این کد برای توزیعی که در آن احتمال وقوع اعداد به صورت توان‌های منفی دو کاهش می‌یابد، کاملاً و مطلقاً بهینه است.

بخش 3) چرا این توزیع در دنیای واقعی خاص است؟

توزیع هندسی با پارامتر **0.5**، توزیعی است که در آن عدد **1** محتمل‌ترین مقدار است (با احتمال **50%**). سپس عدد **2** با احتمال **25%**، عدد **3** با احتمال **12.5%** و به همین ترتیب رخ می‌دهد. این کد برای داده‌هایی طراحی شده که مقادیر بسیار کوچک در آن‌ها به شدت غالب هستند و احتمال وقوع مقادیر بزرگ به سرعت و به صورت نمایی کاهش می‌یابد. در چنین سناریویی، کد یکانی کمترین فضای ممکن را اشغال کرده و به طور کامل با آنتروپی شانون (**Shannon Entropy**) مطابقت دارد.

بخش 4) محدودیت کد یکانی و عدم عمومیت

کد یکانی یک کد فراگیر (**Universal**) نیست. اگر توزیع واقعی داده‌ها به گونه‌ای باشد که اعداد بزرگ‌تر هم با احتمال قابل توجهی رخ دهند، کد یکانی به دلیل رشد خطی (**Linear**) ناکارآمد می‌شود. برای مثال، برای عدد **100**، ما نیاز به **100** بیت داریم. در حالی که در کدهای فراگیر مانند گاما (**Gamma**) یا دلتا (**Delta**)، طول کد به صورت لگاریتمی (**Logarithmic**) رشد می‌کند. بنابراین، کد یکانی تنها زمانی بهترین گزینه است که داده‌ها دقیقاً با توزیع هندسی **(1/2)ⁿ** مطابقت داشته باشند.

بخش 5) پاسخ نهایی

کد یکانی برای توزیع هندسی با پارامتر **0.5** کاملاً بهینه است. در این توزیع، احتمال وقوع عدد **n** برابر با **(1/2)ⁿ** است.

نتیجه‌گیری: در این حالت خاص، میزان محتوای اطلاعاتی هر عدد دقیقاً برابر با طول کد یکانی (**n** بیت) است، که باعث می‌شود کد یکانی با آنتروپی شانون برای این توزیع مشخص کاملاً مطابقت داشته باشد و بهینه‌ترین حالت ممکن برای نمایش این داده‌ها محسوب شود.

مسئله 5.41: تحلیل معماری نمایه‌سازی و فشرده‌سازی در موتور جست‌وجوی «آرکو»

در این مسئله با یک موتور جست‌وجو به نام «آرکو» روبه‌رو هستیم که قرار است روی مجموعه‌ای بسیار بزرگ از لاگ‌های تولیدشده توسط سامانه‌های هوشمند کار کند. نکته بسیار مهم این است که این لاگ‌ها برخلاف متون طبیعی، با یک زبان ماشینی استاندارد و محدود تولید می‌شوند. در نتیجه، تعداد کل واژه‌های ممکن در این زبان ثابت و از پیش مشخص است. اما در مقابل، تعداد کل توکن‌های تولیدشده در این مجموعه به سرعت در حال افزایش است و انتظار می‌رود در آینده چندین برابر شود.

بخش الف) تحلیل استراتژی فشرده‌سازی واژه‌نامه

$$M = k \times T^b$$
$$M = \min(k \times T^b, V) \implies \lim_{T \rightarrow \infty} M = V$$
$$\lim_{T \rightarrow \infty} \frac{|\text{Dictionary}|}{|\text{Postings File}|} = 0$$

در این بخش، تیم مهندسی آرکو پیشنهاد داده است که روش **Dictionary as a String** کنار گذاشته شود و به جای آن از **Blocked Storage** با بلوک‌های بزرگ استفاده شود. برای تحلیل این پیشنهاد، قانون هیپس را بررسی می‌کنیم. در معادله نخست، **M** نشان‌دهنده تعداد واژه‌های متمایز و **T** بیانگر تعداد کل توکن‌ها است. از آنجا که در این مسئله تعداد کل واژه‌های ممکن (**V**) کاملاً محدود و ثابت است، طبق معادله دوم، در بلندمدت تعداد واژه‌های متمایز از مقدار **V** فراتر نرفته و به یک حد اشباع می‌رسد. در نتیجه، اندازه واژه‌نامه در بلندمدت تقریباً ثابت می‌شود (**O(1)**)، در حالی که اندازه فایل پستینگ با افزایش داده‌ها رشد خطی دارد (**O(T)**). بنابراین، بر اساس معادله سوم، نسبت اندازه واژه‌نامه به فایل پستینگ به سمت صفر میل می‌کند.

پاسخ نهایی بخش الف: با توجه به اینکه سهم واژه‌نامه از حافظه کل در گذر زمان ناچیز می‌شود، تمرکز اصلی بر فشرده‌سازی واژه‌نامه (تغییر ساختار به بلوک‌های بزرگ) از دید معماری تصمیم‌گیری نیست. اولویت و بودجه پردازشی سیستم باید منحصراً بر فشرده‌سازی و بهینه‌سازی فایل پستینگ (**Postings File**) متمرکز گردد.

بخش ب) تحلیل توزیع فراوانی و راه‌حل ترکیبی برای Postings

$$f_i \propto \frac{1}{i^3}$$
$$\text{High } df \text{ (Small Gaps)} \implies \text{Gamma Coding}$$
$$\text{Low } df \text{ (Large Gaps)} \implies \text{Variable Byte Coding}$$

همان‌طور که در معادله بالا مشخص است، توزیع فراوانی در این مسئله بسیار تندتر از زیپف کلاسیک بوده و از رابطه معکوس توان سوم پیروی می‌کند. این توزیع شدیداً نامتقارن، نشان‌دهنده آن است که تعداد کمی واژه به صورت فوق‌العاده پرتکرار (با شکاف‌های بسیار کوچک در حد ۱ تا ۳) ظاهر می‌شوند و در مقابل، تعداد بسیار زیادی واژه نادر (با شکاف‌های بسیار بزرگ) وجود دارند. استفاده یکنواخت از روش بایت متغیر (**Variable Byte Coding**) برای کلمات صدرنشین منجر به هدررفت فضا می‌شود، زیرا هر شکافِ تک‌رقمی، حداقل یک بایت کامل اشغال می‌کند. متقابلاً، استفاده یکنواخت از کدگذاری گاما (**Gamma Coding**) نیز باعث می‌شود برای کلمات نادرِ انتهای لیست، طول کدهای به شدت طولانی و غیربهینه‌ای تولید شود.

پاسخ نهایی بخش ب: استفاده یکنواخت از هر یک از این روش‌ها برای کل پیکره نادرست است. بهترین استراتژی، پیاده‌سازی یک رویکرد ترکیبی (هیبریدی) بر اساس مقدار فرکانس سندی (df) یا میانگین فاصله‌ها است: برای واژه‌های پرتکرار که شکاف‌های کوچکی دارند، باید از Gamma Coding استفاده شود تا بالاترین میزان فشرده‌سازی دیسک حاصل شود؛ و برای واژه‌های نادر با شکاف‌های بزرگ، باید از Variable Byte استفاده شود تا هم از طولانی‌شدن غیرمنطقی کدها جلوگیری شود و هم سرعت دسترسی افزایش یابد.

پاسخنامه مسئله 5.42: تحلیل رشد واژگان، توزیع Zipf و فشرده‌سازی ایندکس

مقدمه

در این مسئله با یک موتور جست‌وجوی آفلاین روبه‌رو هستیم که روی تبلت و در محیطی با منابع محدود اجرا می‌شود. بنابراین دو هدف اصلی باید هم‌زمان تأمین شود: نخست آنکه اندازه‌ی **Index** کوچک باقی بماند و دوم آنکه سرعت پاسخ‌گویی به **Query** بر اثر فشرده‌سازی بیش از حد کاهش پیدا نکند. از سوی دیگر، پیکره‌ی این سامانه شامل دو نوع متفاوت از واژه‌هاست: واژه‌های عمومی و پرتکرار مانند **error**، **charge** و **battery**، و در مقابل، واژه‌های تخصصی، شناسه‌مانند و کم‌تکرار مانند **bat4x**، **xr17** و **qz11pro**. این ساختار ترکیبی ایجاب می‌کند که تحلیل نظری بازیابی اطلاعات با طراحی عملی فشرده‌سازی و نمایه‌سازی همراه شود.

بخش الف) تحلیل Heaps و Zipf (محاسبه پارامتر b)

$$M = kN^b \implies \frac{M_2}{M_1} = \left(\frac{N_2}{N_1} \right)^b$$
$$\frac{67,000}{41,000} = \left(\frac{4,800,000}{1,200,000} \right)^b$$
$$1.6341 = 4^b \implies b = \frac{\log(1.6341)}{\log(4)} \approx 0.35$$

قانون **Heaps** بیان می‌کند که رشد تعداد واژه‌های متمایز (**M**) نسبت به تعداد کل توکن‌ها (**N**) یک رابطه توانی با پارامتر رشد **b** است. با جایگذاری مقادیر داده‌شده برای دو بازه زمانی در معادلات بالا، نسبت واژه‌ها و توکن‌ها را محاسبه کرده و با استفاده از قوانین لگاریتم، پارامتر **b** را استخراج می‌کنیم. مقدار **0.35** نشان می‌دهد که رشد واژگان **زیرخطی** است؛ یعنی با افزایش تعداد توکن‌ها، تعداد واژه‌های متمایز نیز افزایش می‌یابد، اما آهنگ این رشد به‌مراتب کمتر از رشد خود توکن‌هاست. بنابراین رشد واژگان کنترل‌شده است، اما متوقف نشده و همچنان ادامه دارد.

بررسی سازگاری رتبه‌های ابتدایی با قانون Zipf

$$f_r \propto \frac{1}{r}$$

$$\frac{f_1}{f_2} = \frac{0.080}{0.040} = 2$$

$$\frac{f_1}{f_4} = \frac{0.080}{0.020} = 4$$

$$\frac{f_1}{f_8} = \frac{0.080}{0.010} = 8$$

در مدل ساده‌ی **Zipf**، فراوانی هر کلمه با معکوس رتبه آن متناسب است. با توجه به داده‌های مسئله، اگر نسبت فرکانس رتبه 1 را به رتبه‌های 2، 4 و 8 محاسبه کنیم، مقادیر استخراج‌شده در کادر بالا دقیقاً با فرم **r/1** سازگارند. بنابراین می‌توان نتیجه گرفت که بخش ابتدایی توزیع، یعنی **Head**، رفتاری کاملاً سازگار با زیپف ساده دارد. اما این نتیجه فقط درباره‌ی واژه‌های پرتکرار و عمومی صدق می‌کند و به‌تنهایی چیزی درباره‌ی پایداری کل واژه‌نامه بیان نمی‌کند.

چرا Zipf-like Head به معنای پایداری Tail نیست؟

در این مسئله، **Head** و **Tail** دو بخش کاملاً متفاوت از توزیع واژه‌ها هستند. بخش **Head**، شامل واژه‌های عمومی و پرتکراری است که رفتار پایداری دارند. اما در بخش **Tail**، واژه‌های نادر و شناسه‌مانندی قرار دارند که غالباً یک‌بار دیده می‌شوند. بر اساس داده‌های مسئله، **64%** از واژه‌های جدید تک‌رخداد بوده‌اند و **72%** از آن‌ها ماهیتی شبیه شناسه و کد خطا دارند. این داده‌ها نشان می‌دهد که **Tail** نه‌تنها بزرگ، بلکه به‌شدت ناپایدار و دائماً در حال تغذیه‌شدن با واژه‌های جدید است. از این رو، سازگاری صرفِ بخشِ اول با زیپف، رشد دائمی واژه‌نامه در بخش انتهایی را متوقف نخواهد کرد.

بخش ب) طراحی فشرده‌سازی برای Dictionary و Postings File

Bigger Block $\implies$ Lower Memory Overhead
 Bigger Block $\implies$ Higher Search Cost inside Block

برای ذخیره‌سازی واژه‌نامه، روش **Dictionary as a String** با ذخیره پیوسته کلمات، سربارهای ساختاری (متادیتا) را کاهش داده و برای یک دستگاه با حافظه‌ی محدود انتخابی منطقی است. ترکیب این روش با **Blocked Storage** نیز موجب می‌شود تا تعداد اشاره‌گرها افت کند. البته همان‌طور که در معادلات بالا مشخص است، این استراتژی یک **Trade-off** ایجاد می‌کند: بلوک‌های بزرگتر باعث کاهش شدید مصرف حافظه می‌شوند اما هم‌زمان زمان جست‌وجوی خطی درون بلوک را بالا می‌برند؛ لذا باید سائز بلوک در حالت میانی بهینه‌ای تنظیم شود.

استخراج gap-d ها و استراتژی انتخابی

$$\begin{aligned} \text{term "charge"} &\Rightarrow \text{Posting} = [10, 11, 12, 13, 14, 300, 301, 302, 9000] \\ &\Rightarrow \text{Gaps} = (10, 1, 1, 1, 1, 286, 1, 1, 8698) \\ \text{term "xr17"} &\Rightarrow \text{Posting} = [8, 620, 621, 20000] \\ &\Rightarrow \text{Gaps} = (8, 612, 1, 19379) \\ \text{Average Gap} &\approx \frac{N}{df} \end{aligned}$$

با بررسی لیست ارجاع برای دو کلمه متفاوت (یکی پرتکرار و دیگری نادر)، مقادیر تفاضلی آن‌ها در کادر بالا استخراج شد. برای واژه عمومی **charge**، فواصل عموماً مقادیر بسیار کوچکی (مثل ۱) هستند. در این حالت، **Gamma Code** به شدت فشرده‌تر از **Variable Byte** عمل می‌کند. اما برای واژه تخصصی **xr17**، فواصل به شدت بزرگ و پراکنده‌اند؛ لذا روش بایت متغیر برای آن بهینه‌تر خواهد بود. از آنجا که کوئری‌های واقعی (مثل **charge bat4x**) ترکیبی از این دو نوع کلمه هستند، استفاده از یک متد یکنواخت برای کل سیستم مردود است. بر اساس معادله آخر، سیستم باید به صورت **Adaptive**، برای کلمات با میانگین گپ کوچک (فرکانس سندی بالا) از گاما و برای میانگین گپ بزرگ (کلمات نادر) از بایت متغیر استفاده کند.

جمع‌بندی نهایی: Zipf-like بودن بخش پرتکرار توزیع به معنای بی‌اهمیت بودن واژگان نادر و بخش Tail نیست؛ زیرا این بخش همچنان ناپایدار و پر از ترم‌های جدید شناسه‌مانند است. در سطح پیاده‌سازی نیز، برای واژه‌نامه از ترکیب ذخیره‌سازی رشته‌ای و بلوکی استفاده می‌کنیم و برای فایل پستینگ، الگوریتم فشرده‌سازی باید کاملاً وابسته به ویژگی‌های آماری هر ترم انتخاب گردد.

پاسخنامه مسئله 5.43: تحلیل رشد واژگان، توزیع فراوانی و انتخاب طراحی ایندکس

بخش 1) تخمین تعداد نهایی Term‌های متمایز با Heaps' Law

$$\begin{aligned} M &= k \times T^\beta \\ \frac{M_2}{M_1} &= \left(\frac{T_2}{T_1} \right)^\beta \\ T_2 &= 40M + 40M + 10M = 90M \\ M_2 &= 600K \times \left(\frac{90M}{40M} \right)^{0.5} \\ M_2 &= 600K \times \sqrt{2.25} = 600K \times 1.5 = 900K \end{aligned}$$

قانون **Heaps** برای پیش‌بینی اندازه واژه‌نامه استفاده می‌شود، که در آن **M** تعداد واژه‌های متمایز، **T** تعداد کل توکن‌ها، **k** ثابت مجموعه و **β** ضریب رشد واژگان است. با فرض اینکه داده‌های جدید از نظر زبانی مشابه داده‌های فعلی باشند، از نسبت رشد توکن‌ها برای محاسبه واژگان جدید استفاده می‌کنیم. در وضعیت فعلی **T₁ = 40M** توکن و **M₁ = 600K** واژه متمایز داریم. با اضافه شدن دو منبع جدید (مجموعاً **50M** توکن)، کل توکن‌ها به **T₂ = 90M** می‌رسد. با جایگذاری این مقادیر و **β = 0.5** در معادلات بالا، تعداد ترم‌های متمایز جدید (**M₂**) حدود **900K** استخراج می‌گردد. یعنی واژه‌نامه از **600 هزار** به حدود **900 هزار** کلمه افزایش می‌یابد. البته این یک تخمین پایه است و به دلیل وجود کاتالوگ فنی با شناسه‌های خاص، تعداد واقعی می‌تواند از این سقف فراتر رود.

ادعای مدیر محصول: «چون رشد واژگان طبق Heaps' Law زیرخطی است و طبق Zipf بیشتر فراوانی روی تعداد کمی واژه متمرکز می‌شود، پس رشد واژه‌نامه مشکل اصلی نیست و کافی است فقط برای Postings واژه‌های پرتکرار بهینه‌سازی کنیم.»

این ادعا از چند جهت ناقص و کاملاً گمراه‌کننده است که در ادامه به تفصیل بررسی می‌شود:

الف) Heaps' Law برای داده‌ی همگن خوب است، نه لزوماً کاتالوگ فنی

قانون **Heaps** معمولاً برای متن‌های نسبتاً طبیعی و همگن مناسب است. برای منبع اول (داده‌های کاربران) استفاده از این قانون منطقی است. اما منبع دوم (کاتالوگ فنی تأمین‌کنندگان) شامل شناسه‌ها و کدهای خاص است (مانند **AX-19K-PRO** یا **MTR-9000-V2**). بسیاری از این ترم‌ها تکرار دارند و نرمال‌سازی تهاجمی آن‌ها نیز مجاز نیست. در چنین داده‌ای، هر شناسه‌ی جدید عملاً یک مدخل جدید به دیکشنری اضافه می‌کند و رشد آن می‌تواند از تخمین ساده‌ی **900K** به شدت فراتر رود.

ب) Zipf نمی‌گوید Tail بی‌اهمیت است

قانون **Zipf** می‌گوید تعداد کمی از ترم‌ها (بخش **Head** مانند phone، price) فراوانی بسیار بالا دارند و تعداد کثیری از ترم‌ها (بخش **Tail** مانند SN-78219) فراوانی بسیار پایین دارند. ترم‌های نادر ممکن است فایل پستینگ کوتاهی داشته باشند، اما به دلیل تعداد انبوهشان، فشار عظیمی بر روی حافظه دیکشنری وارد می‌کنند. پس **Head** از نظر تعداد رخدادها (پستینگ) مهم است و **Tail** از نظر رشد واژه‌نامه (دیکشنری). بنابراین نادیده گرفتن **Tail** خطای استراتژیک است.

ج) عوامل غالب در رشد دیکشنری و پستینگ متفاوت‌اند

رشد دیکشنری مستقیماً تحت تأثیر ورود کلمات جدید و نادر (مانند شناسه‌های فنی) است، در حالی که رشد فایل پستینگ تحت تأثیر تکرار مداوم کلمات عمومی (مانند laptop) است. جدول زیر این تفاوت را نشان می‌دهد:

بخش سیستم	عامل غالب رشد	مثال بارز
Dictionary	ورود ترم‌های جدید، نادر و کدهای فنی	AX-19K-PRO
Postings File	تکرار انبوه ترم‌های عمومی و روزمره	phone, laptop

در نتیجه طراحی ایندکس باید لزوماً برای هر دو بخش بهینه‌سازی شود، نه فقط برای کلمات پرتکرار.

بخش 3) انتخاب طراحی نهایی و دفاع از آن

از میان ترکیبات موجود، بهترین انتخاب معماری ترکیبی **D2 + P1** است؛ یعنی استفاده از **Dictionary as a String + Blocked Storage** برای دیکشنری، و استفاده از **Variable-Byte Coding** برای فایل پستینگ.

۱) سازگاری D2 + P1 با الگوی کوئری‌ها

مهم‌ترین بخش بار کاری سیستم، **60%** کوئری‌هایی است که از یک ترم پرتکرار و یک ترم نادر تشکیل شده‌اند (مانند **laptop AX-19K-PRO**). **D2** با حذف سربارهای اختصاصی هر کلمه و مدیریت در سطح بلوک، مشکل رشد نمایی کلمات نادر در دیکشنری را کنترل می‌کند. از سوی دیگر، **P1** (بایت متغیر) به جای گاما کد استفاده می‌شود، زیرا کدگشایی (**Decoding**) بایت متغیر بسیار سریع‌تر بوده و بار **CPU** را برای پیمایش لیست‌های بلند کلمات پرتکرار کاهش می‌دهد و در نتیجه زمان پاسخ‌گویی (**Latency**) آنلاین را تضمین می‌کند.

۲) چرا «فشرده‌ترین» طراحی لزوماً بهترین نیست؟

ممکن است تصور شود **D2 + P2** (استفاده از گاما کد) به دلیل فشرده‌سازی بیشتر دیسک بهتر است. اما در موتور جست‌وجوی آنلاین، **Latency** و هزینه پردازشی **CPU** حیاتی است. گاما کد در سطح بیت کار می‌کند و رمزگشایی آن کند است، در حالی که بایت متغیر در سطح بایت کار کرده و فوق‌العاده سریع است. جدول زیر این موازنه را نشان می‌دهد:

روش کدگذاری	اندازه‌ی Postings	سرعت Decode	بار پردازشی CPU
Variable-byte	کمی بزرگ‌تر	بسیار سریع‌تر	کمتر
Gamma-code	کوچک‌تر	کندتر	بیشتر

۳) چه هزینه‌ای را آگاهانه قربانی کرده‌ایم؟

در انتخاب **D2 + P1**، ما هزینه **حداکثر فشرده‌سازی دیسک برای فایل پستینگ** را آگاهانه فدای سرعت کرده‌ایم. به عبارتی پذیرفته‌ایم که پستینگ‌ها کمی بزرگ‌تر از حالت گاما باشند، اما در عوض زمان اجرای کوئری‌ها سریع‌تر شده و هزینه‌ی **CPU** برای سرور کاهش یابد.

نتیجه‌گیری: ترم‌های نادر (بخش Tail) به دلیل تولید تعداد عظیمی از مدخل‌ها در کاتالوگ فنی، اهمیت فراوانی دارند و نادیده گرفتن آن‌ها خطاست. معماری D2+P1 با فشرده‌سازی بلوکی، رشد دیکشنری را مهار کرده و با کدگذاری بایت متغیر، سرعت پاسخ‌گویی به کوئری‌های آنلاین را تضمین می‌کند.

پاسخنامه مسئله 5.44: معمار کلمات (The Interstellar Indexer)

بخش 1) کالبدشکافی مسئله و هدف‌گذاری

Heaps’ Law : $M = k \times T^\beta$

Zipf’s Law : $f_r \propto \frac{1}{r}$

در سیستم‌های بازیابی اطلاعات مدرن، با دو قانون کلیدی روبه‌رو هستیم که در معادلات فوق مدل شده‌اند: قانون هیپ (**Heaps' Law**) که به ما می‌گوید با خواندن اسناد بیشتر، کلمات جدیدتری کشف می‌کنیم؛ بنابراین لغت‌نامه (**Vocabulary**) پیوسته بزرگ می‌شود. قانون زیف (**Zipf's Law**) نیز بیان می‌کند که توزیع این کلمات متوازن نیست. تعداد معدودی کلمه به شدت تکرار می‌شوند و مابقی کلمات (کلمات نادر) به ندرت در اسناد دیده می‌شوند.

هدف این سوال، طراحی یک ایندکسر بهینه از نظر حافظه است که اولاً لغت‌نامه را به جای آرایه‌ای از رشته‌های مجزا، به شکل یک «رشته متنی پیوسته» در کنار «اشاره‌گرهای بلوکی» فشرده کند؛ ثانیاً برای کلمات بسیار پرتکرار (که فاصله‌ی حضور آن‌ها در اسناد بسیار کوچک است) از فشرده‌سازی در سطح بیت (**Gamma Code**) استفاده نماید؛ و در نهایت برای کلمات نادر با فواصل بزرگ، فشرده‌سازی بایت متغیر (**Variable Byte**) را به کار گیرد تا سرعت پردازش تضمین شود.

بخش 2) یادآوری مفاهیم پایه (دانش پیش‌نیاز)

$$d\text{-gap}_i = \text{docID}_i - \text{docID}_{i-1}$$

$$\gamma\text{-Code}(5) \implies 5_{10} = 101_2 \implies \text{Offset} = 01, \text{Length} = 110 \implies 11001$$

$$\text{VB Code} \implies \text{Payload} = 7 \text{ bits}, \quad \text{Continuation Bit (MSB)} \in \{0, 1\}$$

برای حل این مسئله، تسلط بر ابزارهای فوق ضروری است. مفهوم **d-gap** به ما می‌گوید که به جای ذخیره شماره شناسه سند به صورت مطلق، فاصله هر سند با سند قبلی را ذخیره می‌کنیم تا اعداد کوچکتر شده و راحت‌تر فشرده شوند (مثلاً **112, 105, 100** به **7, 5, 100** تبدیل می‌شود). کدگذاری گاما (**Gamma Code**) کدی در سطح بیت است و از دو بخش طول و آفست تشکیل می‌شود. همانطور که در کادر بالا برای عدد **5** محاسبه شد، پس از تبدیل به باینری (**101**)، با حذف بیت اول، آفست (**01**) به دست می‌آید و به تعداد طول آفست، عدد **1** و در انتها یک **0** نوشته می‌شود.

در مقابل، کدگذاری بایت متغیر (**Variable Byte**) در سطح بایت عمل می‌کند؛ **7** بیت به داده تعلق دارد و بیت پرارزش نشانگر وضعیت پایان بایت است. در نهایت، مفهوم ذخیره‌سازی بلوکی مطرح است که به جای حفظ آدرس شروع تک‌تک کلمات، آن‌ها را در دسته‌های **k** تایی گروه‌بندی کرده و تنها آدرس اولین کلمه از هر دسته را ذخیره می‌کند.

بخش 3) تحلیل معماری و عبور از دام‌های سوال

Condition $i < T \implies$ Offset is in Bits (Gamma Region)

Condition $i \geq T \implies$ Offset is in Bytes (VB Region)

If $gap = 1 \implies \log_2(1) = 0 \implies \gamma\text{-Code} = 0$

این مسئله دارای دو چالش عمیق مهندسی است که تفکر خلاقانه شما را می‌طلبد. چالش اول تداخل بیتی و بایتی در فایل پستینگ است؛ ترکیب کدهای بیتی گاما و کدهای بایتی در یک آرایه واحد، موجب ایجاد مشکل آفست‌های بایتی (**Padding**) و هدررفت شدید حافظه می‌گردد. راه‌حل خلاقانه این است که معماری حافظه به دو ناحیه کاملاً ایزوله تفکیک شود. از آنجا که کلمات بر اساس فرکانس مرتب می‌شوند، دقیقاً **T** کلمه اول به ناحیه گاما تعلق دارند. همان‌طور که در معادلات استراتژیک بالا مشخص است، آرایه اشاره‌گرها (**postings_ptrs**) رفتار دوگانه‌ای به خود می‌گیرد: برای کلمات با اندیس کمتر از **T**، مقدار ذخیره‌شده به عنوان آفست بیتی، و برای مابقی مقادیر به عنوان آفست بایتی در نظر گرفته می‌شود.

چالش دوم به استثنای شکاف یک در کدهای گاما برمی‌گردد. اگر کلمه‌ای در دو سند متوالی بیاید، فاصله آن‌ها **1** خواهد بود. استخراج آفست از عدد **1** منجر به لگاریتم صفر و خطای محاسباتی می‌شود. راه‌حل این است که عدد **1** به عنوان یک استثنای منطقی در کدگذاری گاما در نظر گرفته شود؛ بدین معنا که طول آفست آن صفر بوده و مستقیماً رشته بیتی **0** را بدون محاسبه اضافی بازمی‌گردانیم.

بخش 4) پیاده‌سازی و کدنویسی گام‌به‌گام

کد زیر پیاده‌سازی بی‌نقص این معماری است. تمامی اصول **Clean Code** در آن رعایت شده و پیچیدگی‌های زمانی در بهینه‌ترین حالت ممکن مهندسی شده است.

```
indexer.py

import sys

class InterstellarIndexer:
    def __init__(self, block_size, top_t):
        # Parameters
        self.k = block_size
        self.top_t = top_t
```

```

# Outputs
self.dict_str = ""
self.block_ptrs = []
self.postings_ptrs = []

# Memory Zones (Solving Trap 1)
self.gamma_region_bits = ""
self.vb_region_bytes = []

def build_index(self, doc_term_stream):
    # Step 1: Aggregate docIDs for each term
    term_to_docs = {}
    for doc_id, term in doc_term_stream:
        if term not in term_to_docs:
            term_to_docs[term] = []
        term_to_docs[term].append(doc_id)

    # Step 2: Zipf's Law - Sort terms by frequency
    sorted_terms = sorted(term_to_docs.keys(), key=lambda t: (-len(term_to_docs[t]), t))

    dict_chars = []

    for i, term in enumerate(sorted_terms):
        # Step 3: Block storage logic
        if i % self.k == 0:
            current_offset = sum(len(s) for s in dict_chars)
            self.block_ptrs.append(current_offset)

            dict_chars.append(str(len(term)) + term)

        # Step 4: Sort docs and calculate d-gaps
        docs = sorted(list(set(term_to_docs[term])))
        gaps = [docs[0]] + [docs[j] - docs[j-1] for j in range(1, len(docs))]

        # Step 5: Hybrid Compression Routing
        if i < self.top_t:
            self.postings_ptrs.append(len(self.gamma_region_bits))
            self.gamma_region_bits += self.encode_gamma(gaps)
        else:
            self.postings_ptrs.append(len(self.vb_region_bytes))
            self.vb_region_bytes.extend(self.encode_vb(gaps))

    self.dict_str = "".join(dict_chars)

def encode_gamma(self, gaps):
    bit_str = ""
    for g in gaps:
        # Trap 2 Resolution: Handling gap = 1
        if g == 1:
            bit_str += "0"
        else:
            binary_repr = bin(g)[2:]
            length = len(binary_repr) - 1
            unary = ("1" * length) + "0"
            offset = binary_repr[1:]
            bit_str += unary + offset
    return bit_str

def encode_vb(self, gaps):
    vb_bytes = []
    for g in gaps:
        temp_bytes = []
        while True:
            temp_bytes.insert(0, g % 128)
            if g < 128:
                break
            g //= 128
            temp_bytes[-1] += 128
        vb_bytes.extend(temp_bytes)
    return vb_bytes

# --- IO Execution ---
if __name__ == "__main__":
    input_data = sys.stdin.read().split()
    if not input_data:
        sys.exit()

    N = int(input_data[0])
    T = int(input_data[1])
    k = int(input_data[2])

    stream = []
    idx = 3
    for _ in range(N):
        doc_id = int(input_data[idx])
        term = input_data[idx+1]
        stream.append((doc_id, term))
        idx += 2

    indexer = InterstellarIndexer(k, T)
    indexer.build_index(stream)

    print(indexer.dict_str)
    print(indexer.block_ptrs)
    print(indexer.postings_ptrs)

```

بخش 5) تحلیل نهایی و اثبات درستی (Complexity)

String Concatenation (Bad) $\implies \mathcal{O}(V^2)$

List Join (Good) $\implies \mathcal{O}(V)$

Sorting by Frequency $\implies \mathcal{O}(V \log V)$

1. پیچیدگی زمانی ساخت لغتنامه: همان‌طور که در روابط بالا ذکر شده است، اگر از الحاق رشته‌ها به شکل `string += term` درون حلقه استفاده می‌کردیم، پیچیدگی زمانی ساخت لغتنامه به دلیل ساختار غیرقابل تغییر در پایتون به توان دو می‌رسید. اما با استفاده از لیست و متد `join()`، این پیچیدگی به صورت کاملاً خطی کاهش یافته است که برای دآوری‌های سیستمی ایده‌آل است.

2. پیچیدگی زمانی مرتب‌سازی: برای اعمال قانون زیف، کلمات بر اساس فرکانس مرتب می‌شوند که پیچیدگی آن به صورت لگاریتمی-خطی درآمده و کاملاً منطقی و استاندارد است.

3. مدیریت حافظه (Space Complexity): با جداسازی استراتژیک مناطق حافظه اطمینان حاصل شد که هیچ بیتی هدر نرفته است و خروجی‌ها در فشرده‌ترین حالت ممکن قرار دارند.

پاسخنامه مسئله 5.45 : مهندسی دیکشنری و پستینگ‌ها (Dictionary & Postings Engineering)

تحلیل کلی صورت سوال و هدف یادگیری

دانشجوی عزیز، پیش از ورود به کدهای راه‌حل، باید درک کنیم که چرا این سوال مطرح شده است. در سیستم‌های واقعی (مانند موتورهای جستجوی سازمانی یا لاگ‌های IoT)، حافظه RAM به شدت محدود است.

طبق **قانون هیپ**، هرچه اسناد بیشتری را پردازش کنیم، کلمات جدیدتری کشف می‌شوند؛ بنابراین دیکشنری (**Vocabulary**) ما به شدت حجیم می‌شود. از سوی دیگر، طبق **قانون زیپ**، چند کلمه خاص به شدت تکرار می‌شوند و میلیون‌ها کلمه به ندرت ظاهر می‌شوند. این رفتار باعث می‌شود فاصله‌ی بین شماره‌ی اسناد (**Gaps**) برای کلمات نادر، بسیار بزرگ باشد. هدف این سوال، سنجش توانایی شما در تبدیل مفاهیم تئوری به الگوریتم‌های حافظه‌محور (**Memory-Efficient**) است.

بخش الف) ساختار دیکشنری فشرده (Dictionary-as-a-String)

1. تحلیل چالش‌ها و ارائه راه‌حل مفهومی

- مشکل ابهام در کلمات دارای عدد (**Ambiguity Bug**): اگر طول کلمه را به صورت متنی به ابتدای کلمه بچسبانیم (مثلاً برای کلمه `404error` با طول 8، رشته `8404error` ساخته شود)، در زمان استخراج نمی‌دانیم طول کلمه 8 است یا 84 یا 840!

- راه‌حل کتابخانه‌ای: برای حل این مشکل، به جای ذخیره عدد به صورت کاراکتر متنی، طول کلمه را به یک بایت حافظه خام تبدیل می‌کنیم. در زبان‌هایی مثل پایتون، تابع `chr(len)` یک عدد صحیح را به معادل تک‌کاراکتری (تک‌بایتی) آن

تبدیل می‌کند. چون طول کلمات لاگ معمولاً کمتر از **256** کاراکتر است، این روش همیشه فقط 1 بایت فضا می‌گیرد و ابهام را کاملاً از بین می‌برد. در زمان خواندن نیز با دستور **ord()** آن را به عدد برمی‌گردانیم.

- **مشکل پیچیدگی زمانی $O(N^2)$:** رشته‌ها در پایتون تغییرناپذیر (**Immutable**) هستند. اگر در یک حلقه یک میلیون بار بنویسید **string += term**، پایتون در هر بار اجرای حلقه، کل رشته‌ی قبلی را کپی کرده و در یک فضای جدید می‌نویسد. این کار پیچیدگی زمانی را به شدت افزایش می‌دهد.
- **راه‌حل بهینه:** کلمات را درون یک آرایه (لیست) قرار داده و در نهایت با استفاده از متد **join()** آن‌ها را با پیچیدگی دقیق **$O(N)$** به یک رشته متصل تبدیل می‌کنیم.

2. پیاده‌سازی گام‌به‌گام (کد بهینه)

نکته: تمامی متغیرها و توضیحات داخل کد طبق استانداردهای مهندسی نرم‌افزار به زبان انگلیسی نوشته شده‌اند.

```
dictionary_search.py

def build_and_search_dict(sorted_terms, k, query):
    # Phase 1: Build the Dictionary O(N) Time Complexity
    # Using a list builder instead of += prevents O(N^2) memory reallocation overhead.
    string_builder = []
    pointers = []
    current_length = 0

    for i in range(len(sorted_terms)):
        term = sorted_terms[i]

        # Store a pointer for the first term of every block of size k
        if i % k == 0:
            pointers.append(current_length)

        # Encode the length as a single ASCII byte to prevent alphanumeric ambiguity
        encoded_length = chr(len(term))
        string_builder.append(encoded_length)
        string_builder.append(term)

        # Advance the current_length counter (1 byte for length + length of term)
        current_length += 1 + len(term)

    dict_string = "".join(string_builder)

    # Phase 2: Search O(log(M/k) + k) Time Complexity
    # M is total terms. M/k is total blocks.
    left = 0
    right = len(pointers) - 1
    block_idx = -1

    # Step A: Binary search on block pointers
    while left <= right:
        mid = (left + right) // 2
        ptr = pointers[mid]

        # Extract the length using ord() which is strictly O(1)
        term_len = ord(dict_string[ptr])
        first_term = dict_string[ptr + 1 : ptr + 1 + term_len]

        if first_term == query:
            return mid * k
        elif first_term < query:
            block_idx = mid
            left = mid + 1
        else:
            right = mid - 1

    if block_idx == -1:
        return -1

    # Step B: Linear search within the identified block
    curr_ptr = pointers[block_idx]

    for step in range(k):
        # Prevent out of bounds if it's the last incomplete block
        if curr_ptr >= len(dict_string):
            break

        term_len = ord(dict_string[curr_ptr])
        current_term = dict_string[curr_ptr + 1 : curr_ptr + 1 + term_len]

        if current_term == query:
            return (block_idx * k) + step

        curr_ptr += 1 + term_len
```

بخش ب) فشرده‌سازی هیبریدی پستینگ‌ها (Hybrid Compression)

1. تحلیل تئوری و اثبات ریاضی

در این بخش، ما باید میان کد گاما (**Gamma**) و متغیر بایتی (**VB**) تصمیم‌گیری کنیم. مشکل اصلی اینجاست که برای قابلیت بازگشایی (**Decodability**)، باید **1** بیت به عنوان پرچم اضافه کنیم.

فرض کنید فاصله بین دو سند برابر با **G** باشد. تعداد بیت‌های لازم برای نمایش خام این عدد در مبنای دو را **L** می‌نامیم. معادلات ریاضی و طول مصرفی هر الگوریتم با احتساب **1** بیت پرچم به شرح زیر محاسبه می‌شود:

$$L = \lfloor \log_2(G) \rfloor + 1$$
$$\gamma\text{-Length (with Flag)} = (2L - 1) + 1 = 2L$$
$$\text{VB-Length (with Flag)} = 8 \times \left\lceil \frac{L}{7} \right\rceil + 1$$

تحلیل نقطه تقاطع (Crossover Point):

آیا **VB** هیچ‌گاه بهتر از گاما عمل می‌کند؟ بیایید با یک مثال ریاضی بررسی کنیم: فرض کنید یک **Gap** برابر با **20** باشد.

1. محاسبه **L**: عدد **20** در باینری **5** بیت است ($L = 5$).

2. محاسبه طول کد گاما: $10 = 5 \times 2$ بیت.

3. محاسبه طول کد **VB**: چون $L \leq 7$ است، تنها **1** بایت نیاز داریم. پس **8** بیت + **1** بیت پرچم = **9** بیت.

نتیجه‌ی شگفت‌انگیز ریاضی: برای فواصل (**Gaps**) در بازه‌ی **16** تا **127** (یعنی $5 \leq L \leq 7$)، الگوریتم **Variable Byte** علیرغم اضافه شدن بیت پرچم، مطلقاً فضای کمتری (**9** بیت) نسبت به گاما (**10** تا **14** بیت) اشغال می‌کند! این موضوع در فواصل بزرگتر نیز به صورت متناوب تکرار می‌شود و فلسفه‌ی نیاز به «فشرده‌سازی هیبریدی» را اثبات می‌کند.

2. پیاده‌سازی گام‌به‌گام

```
hybrid_compression.py

import math

def hybrid_encode_postings(gaps):
    encoding_choices = []

    for gap in gaps:
        # Gap 0 is not encoded in standard IR indexing
        if gap == 0:
            continue

        # Mathematical evaluation of L (binary length)
        L = math.floor(math.log2(gap)) + 1

        # Calculate total bits including the 1-bit decoding flag
        gamma_total_bits = 2 * L
        vb_total_bits = 8 * math.ceil(L / 7.0) + 1

        # Select the strictly optimal compression scheme for the current gap
        if vb_total_bits < gamma_total_bits:
            encoding_choices.append("vb")
        else:
            encoding_choices.append("gamma")

    return encoding_choices
```

- **طراحی ساختار داده:** یاد گرفتیم که ذخیره داده به صورت رشته، اگر اصول مدیریت حافظه (آرایه سازنده به جای الحاق مستقیم) در آن رعایت نشود، سیستم را فلج می‌کند. همچنین استفاده از تبدیل بایت (**chr** در پایتون) یک تکنیک مهندسی کلاسیک برای فرار از ابهام جداکننده‌هاست.
- **درک ریاضی فشرده‌سازی:** برخلاف تصور اولیه که گاما همیشه برای اعداد کوچک بهتر است، محاسبات دقیق فرمولی به ما نشان داد که به دلیل ماهیت بلوکی بودن **VB**، در محدوده‌های خاصی این روش عملکرد بهتری دارد. در سیستم‌های واقعی، هیچ‌چیز بر اساس «حدس» پیاده نمی‌شود، بلکه دقیقاً «مبتنی بر فرمول» است.

پاسخنامه مسئله 5.46 : فشرده‌سازی در موتور جستجو (Search Engine Compression)

مفاهیم پیش‌نیاز (Prerequisites)

- برای درک کامل این سوال و چرایی استفاده از الگوریتم‌های مختلف، تسلط بر مفاهیم زیر الزامی است:
- **قانون زیپ (Zipf's Law):** توزیع کلمات در اسناد یکسان نیست. تعداد کمی از کلمات (مثل حروف اضافه) بسیار پرکاربرد هستند و در اکثر اسناد تکرار می‌شوند، در حالی که بیشتر کلمات (مثل اصطلاحات تخصصی) کم‌کاربرد می‌باشند.
- **فواصل شناسه‌ها (Gaps):** موتورهای جستجو برای بهینه‌سازی حافظه، به جای ذخیره شماره‌ی دقیق اسناد (**DocIDs**)، فاصله‌ی بین آن‌ها را ذخیره می‌کنند تا با اعداد کوچکتری سروکار داشته باشند.
- **کدهای گاما (Gamma):** در سطح «بیت» پردازش می‌شوند. برای فشرده‌سازی اعداد بسیار کوچک (مثل 1 یا 2) فوق‌العاده کم‌حجم هستند، اما برای اعداد بزرگ به شدت ناکارآمد عمل می‌کنند.
- **کدهای بایت متغیر (VB):** در سطح «بایت» (بسته‌های 8 بیتی) پردازش می‌شوند. برای اعداد کوچک کمی هدررفت حافظه دارند، اما بهترین انتخاب برای مدیریت و فشرده‌سازی اعداد بزرگ هستند.

بخش الف) تحلیل مفهومی

پاسخ نهایی:

کلمات پرکاربرد فواصل (**Gaps**) بسیار کوچکی بین شناسه‌هایشان دارند که با کدهای بیتی گاما بهینه‌تر فشرده می‌شوند، اما کلمات کم‌کاربرد فواصل بزرگی تولید می‌کنند که کدهای سطح بایت (**VB**) برای مدیریت این اعداد بزرگ مناسب‌ترند.

تحلیل:

هنگامی که یک کلمه طبق قانون زیپف «پرکاربرد» محسوب می‌شود، یعنی تقریباً در اسناد متوالی و متعددی حضور دارد (مثلاً سند شماره 1، 2، 4 و 5). در نتیجه، فاصله بین شناسه‌ی این اسناد اعداد بسیار کوچکی مثل 1 یا 2 خواهد بود. کدهای گاما می‌توانند عدد 1 را تنها در یک بیت ذخیره کنند؛ بنابراین استفاده از آن‌ها برای بخش پرکاربرد واژگان کاملاً بهینه است.

در مقابل، اگر کلمه‌ای «کم‌کاربرد» باشد، ممکن است در سند شماره 10 و سپس در سند شماره 8050 ظاهر شود. در اینجا فاصله (**Gap**) عدد بزرگ 8040 است. نمایش چنین عدد بزرگی با کدهای گاما نیازمند بیت‌های پیشوند بسیار طولانی است که حافظه و سرعت پردازش را از بین می‌برد. در این مرحله، کدهای بایت متغیر (**VB**) وارد عمل شده و اعداد بزرگ را به شکلی ساختاریافته در بسته‌های 8 بیتی مدیریت می‌کنند.

بخش ب) پیاده‌سازی الگوریتم

برای پیاده‌سازی این سیستم، نیازمند یک ساختار شرطی هستیم که جریان دیکد کردن (**Decoding**) را بر اساس فرکانس کلمه هدایت کند. اگر فرکانس کلمه از حد آستانه زیپ عبور کند، با گاما فشرده شده است و در غیر این صورت با **VB**.

کد پاسخ (پیاده‌سازی بهینه با زمان اجرای $O(1)$ برای هدایت جریان):

```
search_engine_compression.py

def decode_gamma(posting_ptr):
    # mock implementation for gamma decoding
    # returns list of docIDs
    return [1, 2, 4, 5]

def decode_vb(posting_ptr):
    # mock implementation for variable byte decoding
    # returns list of docIDs
    return [10, 8050, 15000]

def dynamic_posting_decoder(term_frequency, zipf_threshold, posting_ptr):
    """
    routes the decoding process based on term frequency and zipf law
    time complexity: O(1) for routing
    """
    # check if term is high frequency (zipf head)
    if term_frequency >= zipf_threshold:
        # small gaps -> gamma encoded
        decoded_doc_ids = decode_gamma(posting_ptr)
        return decoded_doc_ids

    # term is low frequency (zipf tail)
    else:
        # large gaps -> vb encoded
        decoded_doc_ids = decode_vb(posting_ptr)
        return decoded_doc_ids
```

نکته طلایی (Takeaway)

قاعده کلی فشرده‌سازی سیستم‌های بازیابی اطلاعات:

فرکانس تکرار یک کلمه، رابطه مستقیمی با نوع الگوریتم فشرده‌سازی آن دارد.

- فرکانس بالا مساوی است با **Gap کوچک** که نیازمند **فشرده‌سازی بیتی (مانند گاما)** است.
- فرکانس پایین مساوی است با **Gap بزرگ** که نیازمند **فشرده‌سازی بایتی (مانند VB)** است.

پاسخنامه مسئله 5.47 : طراحی هسته موتور جستجوی تعبیه‌شده (Embedded Search Engine)

مفاهیم پیش‌نیاز (Prerequisites)

- قانون هیپ (Heaps' Law):** تعداد واژه‌های یکتا M با رشد تعداد توکن‌ها N به صورت $M = K \times N^\beta$ زیاد می‌شود؛ پس اندازه دیکشنری را پیش از دیدن کل داده می‌توان تخمین زد.
- دیکشنری رشته‌ای با ذخیره‌سازی بلوکی (Dictionary-as-a-String + Blocking):** همه واژه‌ها پشت سر هم در یک رشته قرار می‌گیرند. به جای یک اشاره‌گر برای هر واژه، فقط یک اشاره‌گر برای هر بلوک k تایی نگه می‌داریم و در عوض قبل از هر واژه یک بایت طول می‌گذاریم.
- قانون زیپ (Zipf's Law):** فرکانس واژه با رتبه‌اش نسبت عکس دارد؛ تعداد کمی واژه بخش بزرگی از جستجوها و توکن‌ها را می‌سازند.

- **کدهای گاما و بایت متغیر (γ / VB):** گاما در سطح بیت کار می‌کند و برای **Gap**‌های کوچک بهترین است؛ **VB** در سطح بایت کار می‌کند و برای **Gap**‌های بزرگ مناسب‌تر است.

بخش الف) ارزیابی حافظه دیکشنری با قانون هیپ

گام اول: تخمین تعداد واژه‌های یکتا

با جای‌گذاری پارامترهای جدول در قانون هیپ:

$$M = K \times N^{\beta} = 40 \times (10^6)^{0.5}$$
$$M = 40 \times 1,000 = 40,000 \text{ terms}$$

گام دوم: حجم رشته و اشاره‌گرها

در روش بلوکی، هر واژه علاوه بر 8 بایت متن خودش، یک بایت نشانگر طول هم دارد؛ اما اشاره‌گر 3 بایتی فقط برای ابتدای هر بلوک $k = 4$ تایید ذخیره می‌شود:

$$\text{String} = 40,000 \times (8 + 1) = 360,000 \text{ bytes}$$
$$\text{Pointers} = \left\lceil \frac{40,000}{4} \right\rceil \times 3 = 10,000 \times 3 = 30,000 \text{ bytes}$$
$$\text{Total} = 360,000 + 30,000 = 390,000 \text{ bytes}$$

گام سوم: مقایسه با سقف حافظه

MAX_RAM برابر $400 \text{ KB} = 400 \times 1,024 = 409,600$ بایت است (حتی با تعریف دهدهی 400,000 بایت هم نتیجه تغییر نمی‌کند). چون $390,000 < 409,600$ ، دیکشنری در حافظه جا می‌شود و **Exception** پرتاب نمی‌شود؛ حدود 19,600 بایت هم حاشیه امن باقی می‌ماند.

چرا بلوک‌بندی در این سؤال تعیین‌کننده است؟

اگر همین دیکشنری را بدون بلوک‌بندی (یک اشاره‌گر برای هر واژه و بدون بایت طول) ذخیره کنیم:

$$40,000 \times 8 + 40,000 \times 3 = 440,000 \text{ bytes} > 409,600$$

یعنی سیستم خطای حافظه می‌داد. بلوک‌بندی با $k = 4$ با حذف سه‌چهارم اشاره‌گرها 90,000 بایت صرفه‌جویی می‌کند و فقط 40,000 بایت بابت نشانگرهای طول هزینه دارد. همچنین اگر پیکره دو برابر شود ($N = 2 \times 10^6$)، تعداد واژه‌ها به $\approx 56,569$ و حافظه لازم به $\approx 551,546$ بایت می‌رسد و تابع باید **MemoryError** پرتاب کند.

مرتب‌سازی بر اساس قانون زیپ: پس از قبول‌شدن بررسی حافظه، واژه‌ها به ترتیب نزولی فرکانس مرتب می‌شوند تا پرتکرارترین واژه‌ها در ابتدای رشته و در همان چند بلوک اول قرار بگیرند و هنگام جستجو با احتمال بیشتری در **Cache** پردازنده باشند. برای واژه‌های هم‌فرکانس، ترتیب الفبایی را معیار دوم گذاشته‌ایم تا خروجی همیشه یکسان باشد.

کد پاسخ بخش الف:

```
import math

def build_optimized_dict(terms, N=10**6, K=40, beta=0.5, avg_len=8,
                        k=4, ptr_bytes=3, len_bytes=1, max_ram=400 * 1024):
    # 1. memory check with Heaps' law: M = K * N^beta
    M = K * N ** beta
    string_bytes = M * (avg_len + len_bytes)          # every term + its length byte
    pointer_bytes = math.ceil(M / k) * ptr_bytes      # one pointer per block
    needed = string_bytes + pointer_bytes
    if needed > max_ram:
        raise MemoryError(f"dictionary needs {needed:,} B > MAX_RAM {max_ram:,} B")

    # 2. Zipf: most frequent terms first (ties -> alphabetical)
    ordered = sorted(terms, key=lambda tf: (-tf[1], tf[0]))

    # 3. one long string + one pointer per block of k terms
    buf, pointers = bytearray(), []
    for i, (term, _freq) in enumerate(ordered):
        if i % k == 0:
            pointers.append(len(buf))
            raw = term.encode('utf-8')
            if len(raw) > 255:
                raise ValueError(f"term longer than 255 bytes: {term!r}")
            buf.append(len(raw))          # 1-byte length marker
            buf += raw
    return bytes(buf), pointers
```

برای نمونه، ورودی `[('the', 900), ('of', 700), ('index', 40)]` به رشته `\x03the\x02of\x05index` و آرایه اشاره‌گرهای `[0]` تبدیل می‌شود (هر سه واژه در بلوک اول هستند).

تله طراحی:

دیکشنری رشته‌ای معمولاً الفبایی مرتب است تا بتوان روی اشاره‌گرهای بلوک جستجوی دودویی انجام داد. وقتی واژه‌ها را بر اساس فرکانس مرتب می‌کنیم، این امکان از بین می‌رود؛ پس برای پیدا کردن یک واژه یا باید یک نگاشت جداگانه **term** → **block** (مثلاً جدول درهم‌سازی) داشت، یا جستجوی خطی از ابتدای رشته انجام داد که به لطف قانون زیپ برای اکثر پرس‌وجوها خیلی زود به نتیجه می‌رسد.

بخش ب) فشرده‌سازی هیبریدی لیست‌های پستینگ

گام اول: محاسبه فاصله‌ها

اولین شناسه سند به همان شکل ذخیره می‌شود (قبل از آن سندی نیست که فاصله نسبت به آن حساب شود) و بقیه به‌صورت اختلاف با شناسه قبلی. چون کد گاما عدد 0 را نمی‌تواند نمایش دهد، شناسه‌ها باید اکیداً صعودی و از 1 به بالا باشند؛ تابع در غیر این صورت خطا می‌دهد.

گام دوم: کلمات پرتکرار ($r \leq 10$) با کد گاما

مثال: `doc_ids = [3, 4, 7, 20, 21]` با رتبه زیپ $r = 3$. فاصله‌ها `[3, 1, 3, 13, 1]` هستند و هر کدام این‌طور کد می‌شوند (طول کد گاما $1 + \lceil \log_2 g \rceil$ بیت است):

فاصله	دودویی	offset	کد گاما
3	11	1	101
1	1	—	0
13	1101	101	1110101

با کنار هم گذاشتن کدها، $15 = 1 + 7 + 3 + 1 + 3$ بیت به دست می‌آید: **101 0 101 1110101 0**. این بیت‌ها باید در بایت‌های 8 بیتی پک شوند، پس یک بیت پرکننده لازم است:

101010111101010 + 1 ⇒ 10101011 11010101 = [171, 213]

یعنی پنج شناسه سند فقط در 2 بایت ذخیره شدند (در برابر 20 بایت برای اعداد 32 بیتی).

تله الگوریتمی: بیت‌های پرکننده باید 1 باشند، نه 0

در کد گاما، عدد 1 با یک تک بیت 0 کد می‌شود. اگر بایت آخر را با صفر پر کنیم (11010100)، رمزگشا آن صفر اضافه را یک فاصله‌ی 1 می‌خواند و لیست را [3, 1, 3, 13, 1, 1] برمی‌گرداند؛ یعنی سند 22 به اشتباه به نتایج اضافه می‌شود. اما رشته‌ای از 1 ها که به صفر پایانی نرسد، یک کد ناتمام است و رمزگشا آن را کنار می‌گذارد. راه دیگر، ذخیره تعداد پستینگ‌ها (df) در دیکشنری است.

گام سوم: سایر کلمات (r > 10) با کد بایت متغیر

مثال: doc_ids = [824, 829, 215406] با رتبه $r = 250$. فاصله‌ها [824, 5, 214577] هستند. هر عدد به گروه‌های 7 بیتی شکسته می‌شود و بیت پرارزش آخرین بایت هر عدد 1 می‌شود:

824 $\Rightarrow$ 00000110 10111000
5 $\Rightarrow$ 10000101
214,577 $\Rightarrow$ 00001101 00001100 10110001

مجموعاً 6 بایت. اگر همین لیست را با گاما کد می‌کردیم، فقط عدد 214,577 به تنهایی $2 \times 17 + 1 = 35$ بیت لازم داشت؛ به همین دلیل برای دم توزیع زیپ که فاصله‌های بزرگ دارد، **VB** انتخاب بهتری است و رمزگشایی بایت‌محور آن هم سریع‌تر است.

نکته: این پیاده‌سازی از قرارداد کتاب (بایت پرارزش اول، بیت 1 روی آخرین بایت) پیروی می‌کند. کد درسنامه فصل ۵ بایت کم‌ارزش را اول می‌گذارد و بیت ادامه را روی بایت‌های غیرآخر 1 می‌کند. هر دو درست‌اند به شرطی که رمزگذار و رمزگشا یک قرارداد داشته باشند.

کد پاسخ بخش ب:

```
encode_postings.py

def gaps_of(doc_ids):
    if doc_ids[0] < 1 or any(b <= a for a, b in zip(doc_ids, doc_ids[1:])):
        raise ValueError("doc_ids must be strictly increasing and >= 1")
    # the first docID is stored as-is, the rest as differences
    return [doc_ids[0]] + [b - a for a, b in zip(doc_ids, doc_ids[1:])]

def gamma_bits(g):
    offset = bin(g)[3:]                                # binary without the leading 1
    return '1' * len(offset) + '0' + offset            # unary(length) + offset

def vb_bytes(g):
    out = []
    while True:
        out.insert(0, g % 128)
        if g < 128:
            break
        g //= 128
    out[-1] += 128                                     # high bit marks the last byte
    return out

def encode_postings(doc_ids, zipf_rank):
    gaps = gaps_of(doc_ids)
    if zipf_rank <= 10:                                # Zipf head -> small gaps -> gamma
        bits = ''.join(gamma_bits(g) for g in gaps)
        bits += '1' * (-len(bits) % 8)                 # pad with 1s, never with 0s
        return bytes(int(bits[i:i + 8], 2) for i in range(0, len(bits), 8))
    return bytes(b for g in gaps for b in vb_bytes(g)) # Zipf tail -> VB
```

نکته طلایی (Takeaway)

- **قانون هیپ** می گوید قبل از ساخت ایندکس چقدر حافظه لازم است؛ بلوک بندی دیکشنری می تواند مرز «جا می شود / جا نمی شود» را جابه جا کند.
- **قانون زیپ** می گوید کدام واژه ها داغ هستند: آن ها را جلوتر بچینید و پستینگ هایشان را با گاما در سطح بیت فشرده کنید.
- در کدهای بی تی همیشه به **مرز بایت** فکر کنید: پرکننده اشتباه، داده اشتباه تولید می کند.

پاسخنامه مسئله 5.48 : معماری موتور جستجوی لاگ های توزیع شده

(Distributed Log Search Engine)

بخش الف) معماری واژه نامه و دام مقیاس پذیری

💡 مفهوم اولیه (قانون Heaps):

قانون هیپ به ما می گوید با خواندن متن های جدید، چقدر کلمه جدید (که قبلاً ندیده ایم) به واژه نامه اضافه می شود. فرمول پایه آن $M = k \times N^\beta$ است.

در نمودار لگاریتمی، این فرمول به یک معادله خط راست تبدیل می شود. در این معادله خط، **شیب خط همان β است**:

$$\log_{10}(M) = \log_{10}(k) + \beta \times \log_{10}(N)$$

■ گام 1: محاسبه شیب خط (پارامتر β)

در صورت سوال دو نقطه روی نمودار به ما داده شده است: نقطه اول $(x_1 = 0, y_1 = 2)$ و نقطه دوم $(x_2 = 4, y_2 = 8)$. برای پیدا کردن شیب خط (β) از فرمول ساده ریاضی شیب استفاده می کنیم:

$$\beta = \frac{y_2 - y_1}{x_2 - x_1} = \frac{8 - 2}{4 - 0} = \frac{6}{4} = 1.5$$

پس مقدار β برای این سیستم برابر با **1.5** به دست آمد.

■ گام 2: تحلیل معنای $\beta = 1.5$ (کشف دام سوال)

در تمام زبان های طبیعی (مثل انگلیسی، فارسی و...)، مقدار β **همیشه بین 0.4 تا 0.6** است. یعنی سرعت تولید کلمات جدید همیشه از سرعت کل کلمات کمتر است. اما اینجا $\beta = 1.5$ (یعنی بزرگتر از 1) شده است! این از نظر منطقی یعنی چه؟

نکته کلیدی: این یعنی به ازای هر 1 کلمه ای که پردازش می شود، بیش از 1 کلمه جدید تولید می شود! این حالت فقط زمانی رخ می دهد که کلمات، متن های عادی نباشند، بلکه **رشته های کاملاً تصادفی، هش ها (Hashes) یا شناسه سنسورها (UUID)** باشند که هیچ کلمه تکراری در آن ها وجود ندارد.

■ گام 3: نتیجه گیری برای معماری واژه نامه

حالا که فهمیدیم داده‌های ما رشته‌های تصادفی بدون اشتراک (مثل هش‌ها) هستند، باید ببینیم روش‌های فشرده‌سازی پیشنهادی چگونه عمل می‌کنند:

- **Blocked Storage و Dictionary-as-a-String:** اساس کار این روش‌ها **کدگذاری پیشوندی (Front Coding)** است. یعنی اگر کلماتی مثل `automat`، `automatic` و `automation` داشته باشیم، بخش مشترک `automat` را فاکتور می‌گیرد تا فضا ذخیره شود.
- **نتیجه‌گیری نهایی:** چون کلمات ما هش‌های کاملاً نامرتب هستند، هیچ پیشوند مشترکی بین آن‌ها وجود ندارد! بنابراین این الگوریتم‌ها هیچ چیزی را نمی‌توانند فاکتور بگیرند. بدتر از آن، استفاده از این روش‌ها نیازمند ذخیره کردن پوینترها (**Pointers**) و طول رشته‌هاست که باعث می‌شود **حجم واژه‌نامه از حالت عادی هم بیشتر شود** و حافظه **RAM** به سرعت پر شده و سیستم **Crash** کند.

بخش ب) پارادوکس فشرده‌سازی Postings

💡 مفهوم اولیه (قانون Zipf و Gaps):

- **قانون زیپف:** می‌گویند فرکانس یک کلمه با رتبه آن رابطه عکس دارد. ($cf_i = \frac{c}{i}$)
- **فاصله یا Gap:** در فایل پستینگ، به جای ذخیره شماره اسناد، فاصله بین آن‌ها را ذخیره می‌کنیم تا اعداد کوچکتر شوند. میانگین این فاصله از تقسیم کل اسناد (یا توکن‌ها) بر فرکانس کلمه به دست می‌آید.

■ گام 1: محاسبه فرکانس کلمه رتبه 50,000

طبق جدول، کلمه اول (رتبه $i = 1$) فرکانس $cf_1 = 10,000,000$ دارد. در قانون زیپف، ثابت c برابر است با فرکانس کلمه اول ضرب در رتبه‌اش:

$$c = 1 \times 10,000,000 = 10,000,000$$

حالا فرکانس کلمه رتبه 50,000 را حساب می‌کنیم:

$$cf_{50,000} = \frac{10,000,000}{50,000} = 200$$

پس کلمه **[kernel_panic]** کلاً 200 بار در کل سیستم دیده شده است.

■ گام 2: محاسبه میانگین فاصله (Average Gap)

کل توکن‌های سیستم $N = 10^8 = 100,000,000$ است. این کلمه 200 بار تکرار شده است. پس میانگین فاصله بین هر بار دیده شدن این کلمه چقدر است؟

$$\text{Gap} = \frac{100,000,000}{200} = 500,000$$

عدد **Gap** ما بسیار بزرگ و برابر 500,000 است.

■ گام 3: محاسبه فضای مصرفی با Variable Byte (VB)

در روش **VB**، ما از **7** بیت هر بایت برای داده و **1** بیت برای کنترل استفاده می‌کنیم. باید ببینیم عدد **500,000** به چند بیت نیاز دارد.

- **1** بایت (**7** بیت داده): حداکثر عدد $2^7 - 1 = 127$
- **2** بایت (**14** بیت داده): حداکثر عدد $2^{14} - 1 = 16,383$
- **3** بایت (**21** بیت داده): حداکثر عدد $2^{21} - 1 = 2,097,151$

چون $16,383 < 500,000 \leq 2,097,151$ ، پس ما به **3 بایت** کامل نیاز داریم. **3** بایت یعنی **24 بیت** فضای ذخیره‌سازی.

■ گام 4: محاسبه فضای مصرفی با کد گاما (gamma-code)

فرمول طول کد گاما برای عدد G برابر است با: $2 \lfloor \log_2(G) \rfloor + 1$.

برای عدد **500,000** باید لگاریتم پایه **2** آن را حساب کنیم. می‌دانیم که $2^{18} = 262,144$ و $2^{19} = 524,288$. پس $\lfloor \log_2(500,000) \rfloor = 18$.

$$\text{Length} = 2(18) + 1 = 36 + 1 = 37 \text{ bits}$$

پس روش کد گاما به **37 بیت** فضا نیاز دارد.

■ گام 5: نتیجه‌گیری و رد استدلال مهندس ارشد

حالا دو عدد را مقایسه می‌کنیم:

- روش **Variable Byte** برای این فاصله بزرگ: **24 بیت**
- روش **gamma-code** برای این فاصله بزرگ: **37 بیت**

نتیجه‌گیری: استدلال مهندس ارشد کاملاً غلط و یک **دام ذهنی** است!

برخلاف تصور اولیه، کدهای بیتی (مثل گاما) برای **اعداد بزرگ به شدت ناکارآمد هستند**. دلیل آن بخش **Unary** کد گاما است که به صورت خطی رشد می‌کند (برای عدد **500,000** باید **18** عدد **1** پشت سر هم بنویسیم!). در حالی که **Variable Byte** با وجود هدر رفتن بیت‌های کنترلی، ساختار لگاریتمی بهتری برای اعداد خیلی بزرگ دارد.

کد گاما فقط برای فاصله‌های بسیار کوچک (**Gap** های کوچک یعنی کلمات پرکاربرد) از **VB** بهتر عمل می‌کند.